

中国科学技术大学

University of Science and Technology of China

专业博士学位论文



论文题目

面向 SIMD 架构智能处理器的

编译优化技术研究

作者姓名

郑璠韬

类别领域

电子信息

校内导师

周学海 教授

实践导师

陈天石 研究员

完成时间

二〇二五年三月

中国科学技术大学

工程博士学位论文



面向 SIMD 架构智能处理器的 编译优化技术研究

作者姓名： 郑鎏韬

类别领域： 电子信息

校内导师： 周学海 教授

企业导师： 陈天石 教授

完成时间： 二〇二五年六月九日

University of Science and Technology of China
A dissertation for doctor's degree



Research on Compilation Optimization Techniques for SIMD Architecture Intelligent Processors

Author: Liutao Zheng

Speciality: Electronics and Engineering

Supervisor: Prof. Xuehai Zhou

Advisor: Prof. Tianshi Chen

Finished time: June 9, 2025

创新性及应用说明

本文围绕智能处理器在深度学习领域的编译优化难题,从工程实践与理论创新双维度展开系统性研究,取得了多项突破性成果。在工程实践方面,针对 SIMD 架构特性创新设计了 Triton 语言编译优化方案,提出数据驻留与流水线并行技术,在寒武纪 MLU370 平台实现算子性能最高提升 4.65 倍;研发的 FusionFrame 框架通过存储感知调度优化,使 ResNet-50 的片外内存访问延迟降低 52.7%,系统级验证显示卷积神经网络和大语言模型任务分别获得 3.1 倍和 2.43 倍加速。在理论创新层面,提出了面向 SIMD 的 Linalg 中间表示体系,攻克了编译转换过程中信息丢失的难题;提出基于向量描述的地址推导系统,解决冗余输入爆炸问题;构建统一分析成本模型,实现跨平台性能平均提升 35.2%。研究成果已应用于寒武纪深度学习系统,相比 Nvidia T4 GPU 实现大语言模型场景下的端到端执行时间缩减 33.64%,卷积神经网络场景下的性能提升 62.18%,相关技术获多项发明专利,为国产 AI 芯片生态建设提供了核心编译支撑,推动了深度学习编译理论体系的发展与工程实践创新。

摘 要

随着人工智能技术的快速发展，深度学习作为其核心驱动力，已在计算机视觉、自然语言处理、语音识别等多个领域取得突破性进展并得到广泛应用。然而，随着模型复杂度的不断提升和计算规模的持续扩大，如何充分发挥智能处理器的计算潜力以提升 AI 计算效率，已成为当前研究的重要课题。当前 AI 编译优化面临三大核心挑战：首先，现有领域特定语言和算子编译器主要针对 GPU 的 SIMT 架构优化，难以有效适配 SIMD 架构处理器的粗粒度向量操作和独特数据访问模式；其次，深度神经网络任务在硬件加速器上的执行效率受限于片外与片上内存之间的数据搬运开销；最后，图融合指令生成过程中存在冗余输入爆炸和上采样计算效率低下等关键问题。针对这些挑战，本文从深度学习编译器的性能优化与多后端支持的角度出发，提出了一系列创新性解决方案，并在实际系统中进行了充分的实验验证。研究聚焦于面向智能处理器（SIMD 架构）的深度学习编译与调度优化，主要贡献包括：

1. 面向 SIMD 架构智能处理器的 Triton 语言编译性能优化。本文提出了面向 SIMD 架构的 Triton 语言编译性能优化方法。针对现有领域特定语言和编译器（如 Triton）主要针对 GPU 的 SIMT 架构优化，难以有效适配 SIMD 架构处理器的粗粒度向量操作和独特数据访问模式的问题，设计了一套全新的编译流程。该流程采用 Linalg 方言作为中间抽象层，通过多阶段优化架构，实现了从 Triton-Python 算子到硬件无关 Linalg 表示，再到 MLU-IR 中间表示的高效转换。提出了指针分析、计算语义抽象等关键技术，解决了 SIMD 架构编译过程中信息丢失的问题。针对常见二元算子，提出了数据片上驻留和软件流水线并行两项关键优化技术，显著提升了计算效率。该框架有效克服了现有编译器在适配 SIMD 架构时的不足，为深度学习模型在多样化硬件平台上的高效执行提供了强有力的支持。

2. 面向 SIMD 架构智能处理器的算子融合数据流评估框架。本文设计并实现了面向 SIMD 架构智能处理器的融合数据流框架 FusionFrame。该框架采用以存储为中心的融合机制，通过灵活的调度空间表示和统一的分析成本模型，优化了片上内存访问和分块策略。针对深度神经网络（DNN）任务在硬件加速器上的执行效率优化问题，FusionFrame 提出了一种基于存储层次感知的调度优化方法。该方法通过循环分块和操作融合相结合的策略，实现了计算与存储的协同优化：在计算层面，通过细粒度的循环分块策略，有效增强了数据局部性；在存储层面，通过算子融合机制，最小化了片外与片上内存之间的数据搬运开销。通过系统化的调度空间探索和精确的性能评估，FusionFrame 能够生成多种优化方案

并选择最优调度策略。实验结果表明,该框架在减少 DRAM 访问和降低延迟方面显著优于现有方法,特别是在卷积神经网络和 Transformer 类模型中展现出优越性能,为 DNN 调度提供了新的理论支撑和实践工具。

3. 面向 SIMD 架构处理器的图融合指令生成系统。针对图融合指令生成中的关键挑战,本文提出了一套完整的优化方法,重点解决了冗余输入爆炸和上采样计算效率低下两大核心问题。在冗余输入优化方面,针对算子融合过程中冗余数据传播放大现象,提出了一种基于精确地址模式表达的形状推导方法,通过深度融合图的推导机制,有效抑制了冗余数据的指数级增长。在上采样计算优化方面,提出了基于池化索引的重排机制和固定偏移的索引计算优化方法,通过深入分析上采样操作的计算特征,设计针对性的优化策略,显著提升了上采样结构的执行效率。特别是在填充操作和卷积计算的融合场景中,提出的优化策略实现了显著的性能提升。该方法不仅解决了冗余输入爆炸问题,还通过上采样计算的优化,为融合算子的高效执行提供了全面的技术支持。

4. 基于 MLU 平台的深度学习系统级验证。在系统层面,本文工作已在寒武纪深度学习系统中得到实际应用和验证。基于 MLU 平台的系统级设计验证表明,本文提出的优化方案在大语言模型和传统卷积神经网络场景下均取得了显著的性能收益。

本文的研究成果不仅丰富了深度学习编译与调度的理论体系,也为实际应用中的性能优化提供了有力的技术支撑。通过上述创新性工作,本文有效解决了深度学习系统软件优化中的关键问题,推动了国产深度学习计算平台的技术进步和广泛应用。

关键词: 深度学习; 编译优化; 调度优化; 图编译; 指令生成

ABSTRACT

With the rapid development of artificial intelligence technology, deep learning, as its core driving force, has achieved breakthrough progress and widespread application in various fields such as computer vision, natural language processing, and speech recognition. However, as model complexity continues to increase and computational scale expands, how to fully leverage the computational potential of intelligent processors to enhance AI computing efficiency has become a crucial research topic. Current AI compilation optimization faces three core challenges: (1) existing domain-specific languages and operator compilers are primarily optimized for GPU's SIMT architecture, making it difficult to effectively adapt to the coarse-grained vector operations and unique data access patterns of SIMD architecture processors; (2) the execution efficiency of deep neural network tasks on hardware accelerators is limited by the data transfer overhead between off-chip and on-chip memory; (3) critical issues such as redundant input explosion and inefficient upsampling computation exist in graph fusion instruction generation. To address these challenges, this paper proposes a series of innovative solutions from the perspective of performance optimization and multi-backend support for deep learning compilers, with thorough experimental validation in practical systems. The research focuses on deep learning compilation and scheduling optimization for intelligent processors (SIMD architecture), with main contributions as follows:

1. Triton Language Compilation Performance Optimization for SIMD Architecture Intelligent Processors. This paper proposes a Triton language compilation performance optimization method for SIMD architecture. Addressing the challenge that existing domain-specific languages and compilers (e.g., Triton) are primarily optimized for GPU's SIMT architecture and difficult to adapt to the coarse-grained vector operations and unique data access patterns of SIMD architecture processors, we designed a novel compilation flow. This flow employs Linalg dialect as an intermediate abstraction layer and achieves efficient transformation from Triton-Python operators to hardware-independent Linalg representation and then to MLU-IR intermediate representation through a multi-stage optimization architecture. Key technologies such as pointer analysis and computational semantic abstraction are proposed to solve the irreversible transformation problem in SIMD architecture compilation. For common binary operators, two key optimization techniques - on-chip data residency and software pipelining parallelism - are proposed, significantly improving computational efficiency.

This framework effectively overcomes the limitations of existing compilers in adapting to SIMD architecture, providing strong support for efficient execution of deep learning models on diverse hardware platforms.

2. Operator Fusion Dataflow Evaluation Framework for SIMD Architecture Intelligent Processors. This paper designs and implements FusionFrame, a fusion dataflow framework for intelligent processors. The framework adopts a storage-centric fusion mechanism and optimizes on-chip memory access and tiling strategies through flexible scheduling space representation and a unified analysis cost model. For the execution efficiency optimization of deep neural network (DNN) tasks on hardware accelerators, FusionFrame proposes a storage hierarchy-aware scheduling optimization method. This method achieves co-optimization of computation and storage through a combination of loop tiling and operation fusion strategies: at the computation level, fine-grained loop tiling effectively enhances data locality; at the storage level, operator fusion minimizes data transfer overhead between off-chip and on-chip memory. Through systematic scheduling space exploration and precise performance evaluation, FusionFrame can generate multiple optimization schemes and select the optimal scheduling strategy. Experimental results demonstrate that the framework significantly outperforms existing methods in reducing DRAM access and latency, particularly showing superior performance in convolutional neural networks and Transformer models, providing new theoretical support and practical tools for DNN scheduling.

3. Graph Fusion Instruction Generation System for SIMD Architecture Processors. Addressing the key challenges in graph fusion instruction generation, this paper proposes a comprehensive optimization method, focusing on two core issues: redundant input explosion and inefficient upsampling computation. For redundant input optimization, we propose a shape derivation method based on precise address pattern expression to effectively suppress the exponential growth of redundant data through a deep fusion graph derivation mechanism. For upsampling computation optimization, we propose a pooling index rearrangement mechanism and fixed-offset index computation optimization method. By deeply analyzing the computational characteristics of upsampling operations, we design targeted optimization strategies that significantly improve the execution efficiency of upsampling structures. Particularly in fusion scenarios involving padding operations and convolution computations, the proposed optimization strategies achieve remarkable performance improvements. This method not only solves the redundant input explosion problem but also provides comprehensive technical support for efficient execution of fusion operators through upsampling computation

optimization.

4. System-Level Validation of Deep Learning on MLU Platform. At the system level, this work has been practically applied and validated in the Cambricon deep learning system. System-level design validation based on the MLU platform demonstrates that the proposed optimization schemes achieve significant performance gains in both large language models and traditional convolutional neural network scenarios.

The research outcomes of this paper not only enrich the theoretical system of deep learning compilation and scheduling but also provide strong technical support for performance optimization in practical applications. Through these innovative works, this paper effectively addresses key issues in deep learning system software optimization, promoting technological advancement and widespread application of domestic deep learning computing platforms.

Key Words: Deep Learning; Compilation Optimization; Scheduling Optimization; Graph compiler; Instruction Generator

目 录

第 1 章 绪论.....	1
1.1 研究背景与研究意义.....	1
1.1.1 深度学习算法的发展.....	1
1.1.2 智能处理器.....	3
1.1.3 深度学习编译器.....	5
1.2 国内外研究现状.....	9
1.2.1 智能处理器的发展现状.....	9
1.2.2 深度学习编译器的发展现状.....	11
1.3 本文研究问题与难点.....	14
1.3.1 智能处理器的算子编译性能优化问题	15
1.3.2 智能处理器的图融合策略优化与性能建模问题	15
1.3.3 智能处理器的图融合指令生成优化问题	16
1.4 本文内容与贡献.....	17
1.4.1 面向 SIMD 架构智能处理器的 Triton 语言编译性能优化 ..	17
1.4.2 面向 SIMD 架构智能处理器的算子融合数据流评估框架 ..	18
1.4.3 面向 SIMD 架构智能处理器的图融合指令生成系统	19
1.4.4 基于 MLU 平台的深度学习系统级设计与验证.....	19
1.5 本文组织结构.....	20
第 2 章 面向 SIMD 架构智能处理器的 Triton 语言编译性能优化	22
2.1 引言.....	22
2.2 相关背景和研究动机.....	23
2.2.1 MLIR 概述.....	23
2.2.2 Triton 语言概述	25
2.2.3 SIMD 架构芯片编译器设计问题与挑战	27
2.3 面向 SIMD 架构智能处理器的 Triton 语言编译器设计	31
2.3.1 硬件无关表示设计.....	32
2.3.2 硬件相关表示设计.....	36
2.4 面向 SIMD 架构智能处理器的 Triton 程序编译性能优化	38
2.4.1 任务调度优化.....	38
2.4.2 二元算子性能优化.....	41
2.5 实验设计与评估.....	44
2.5.1 实验目标.....	44
2.5.2 实验环境.....	45
2.5.3 Triton 语言编译性能优化消融实验.....	45

2.5.4 Triton 语言在 MLU 平台上的编程效率评估	48
2.5.5 Triton 语言跨平台的性能对比	49
2.6 讨论	50
2.7 本章小结	50
第 3 章 面向 SIMD 架构智能处理器的算子融合数据流评估框架	52
3.1 引言	52
3.2 相关背景和研究动机	53
3.2.1 调度表示和原语	53
3.2.2 融合操作的分块	53
3.2.3 硬件映射探索	54
3.2.4 数据流调度优化问题建模	54
3.3 计算图融合数据流的调度空间	55
3.3.1 以存储为中心的表示	56
3.3.2 以存储为中心的表示原语	58
3.4 基于量化分析的性能评估模型	59
3.4.1 以存储为中心的融合机制	59
3.4.2 分块融合算子	60
3.4.3 分块数据形状计算和内存分配	61
3.4.4 性能评估	63
3.5 实验评估与分析	63
3.5.1 实验目标	63
3.5.2 实验环境	64
3.5.3 L1 缓存对不同融合数据流和分块维度的影响实验	65
3.5.4 不同 SotA 融合数据流的对比实验	67
3.5.5 针对调度空间的不同维度与 SotA 工作的对比实验	69
3.6 讨论	71
3.7 本章小结	71
第 4 章 面向 SIMD 架构智能处理器的图融合指令生成系统	73
4.1 引言	73
4.2 相关背景和研究动机	74
4.2.1 深度融合下的地址推导问题	74
4.2.2 深度融合下的冗余输入爆炸问题	74
4.2.3 神经网络中上采样操作的性能优化问题	76
4.3 图融合指令生成地址推导	79
4.3.1 访存地址表达	79
4.3.2 地址推导与数据复用整体算法概览	80
4.3.3 地址推导的基本流程	81
4.3.4 数据复用优化	83

4.3.5 访存模式合并方案.....	85
4.4 上采样计算模式优化.....	87
4.5 实验评估与分析.....	90
4.5.1 实验目标.....	90
4.5.2 实验环境.....	90
4.5.3 数据复用的消融实验.....	91
4.5.4 上采样性能优化的消融实验.....	92
4.5.5 融合算子指令生成系统的性能与 SotA 对比	93
4.6 讨论.....	95
4.7 本章小结.....	95
第 5 章 基于 MLU 平台的深度学习系统级设计与验证	96
5.1 引言.....	96
5.2 寒武纪深度学习系统软件设计	96
5.2.1 寒武纪系统软件整体架构.....	96
5.2.2 算子编译层架构.....	98
5.2.3 图编译层架构.....	99
5.3 系统层面收益评估与分析	100
5.3.1 MLU 架构下的系统软件收益评估.....	101
5.3.2 跨处理器平台的收益评估.....	101
5.4 本章小结.....	102
第 6 章 总结与展望	103
6.1 研究总结.....	103
6.1.1 算子编译器的研究内容与本文贡献.....	103
6.1.2 计算图编译器的研究内容与本文贡献	104
6.1.3 系统级应用与评估.....	105
6.2 研究展望.....	106
参考文献.....	108

插图清单

图 1.1	深度学习算法与智能处理器架构的协同优化需要编译器作为桥梁	6
图 1.2	深度学习任务的整体流程	7
图 1.3	TVM 张量表达式优化流程, 多后端调度支持 ^[22]	12
图 1.4	TensorFlow XLA ^[37] 编译流程	13
图 1.5	以 Apollo ^[38] 架构为例展示典型的计算图编译流程	14
图 1.6	本文研究思路	17
图 2.1	矩阵乘法计算使用标量计算表示与 Linalg 方言表示的区别	24
图 2.2	Triton 程序分块与 CUDA 线程分块的对比 ^[50]	26
图 2.3	现有 Triton 基于 GPU 后端的下降流程 ^[50]	27
图 2.4	寒武纪 MLU ^[51] (左图) 和 Nvidia GPU ^[52] (右图) 的硬件抽象模型	29
图 2.5	Triton 面向 SIMD 架构智能处理器的下降流程	31
图 2.6	Triton-IR 和 Linalg-IR 对于指针操作的表达方式对比	34
图 2.7	Triton-IR 中对输入指针参数的处理	34
图 2.8	Triton-IR 和 Linalg-IR 对于加法计算的表达方式对比	35
图 2.9	Linalg-IR 下降为 MLU-IR 的下降流程	36
图 2.10	MLU UnionN 类型任务的映射关系 ^[51]	38
图 2.11	GPU 中的任务调度模型 ^[52]	39
图 2.12	对于一块 6 x 128 大小的数据块, 不同架构下 Triton 算子的任务划分方式	41
图 2.13	数据驻留技术在卷积计算中的优化方法	43
图 2.14	流水线技术在卷积计算中的优化方法	44
图 2.15	MLU 平台上 Triton 语言任务调度优化前后的性能对比	47
图 2.16	MLU 平台上二元算子性能优化前后的性能对比	48
图 2.17	MLU 平台上 Triton 与 BANGC 语言的代码效率与性能对比	49
图 2.18	Triton 语言编写的算子在 MLU 与 GPU 平台上的性能对比	49
图 3.1	以存储为中心的调度空间表示	56
图 3.2	FusionFrame 的整体架构	59
图 3.3	以存储为中心的融合机制示意图	61
图 3.4	分块输出维度的示意图, 活动分块在橙色中标记, 表示为 <i>Act_tile</i>	62
图 3.5	针对 Cambricon-Acc 的验证结果	64

图 3.6	选定的具有相同架构和不同参数的 ResNet-50 残差块工作负载的 详细说明	65
图 3.7	基于图中的调度表示, 数据搬运量和延迟的实验结果	67
图 3.8	三种融合数据流: 逐层、DeFiNES-like 和 FusionFrame 的性能对比	68
图 3.9	不同调度维度对性能影响的 SotA 比较	69
图 4.1	卷积算子融合时, 相邻时间片处理的数据	76
图 4.2	编码器-解码器结构 ^[84]	76
图 4.3	卷积和反卷积计算方式的对比 ^[85]	77
图 4.4	上采样操作中的填充和计算模式	77
图 4.5	将长度为 14 的数据块按照 3 的长度拆成 5 块	79
图 4.6	一个典型的神经网络片段的拆分数据块的推导及其 Interval 描述	82
图 4.7	<i>access</i> 模式及其 Interval 描述	83
图 4.8	<i>reuse</i> 模式及其 Interval 描述	83
图 4.9	<i>reuse_by_next</i> 模式及其 Interval 描述	84
图 4.10	<i>compute</i> 模式及其 Interval 描述	84
图 4.11	数据复用案例	85
图 4.12	同一数据存在多个访存模式的示例	85
图 4.13	将不同访存模式的数据块进行合并	86
图 4.14	合并之后重新计算 <i>reuse</i> 和 <i>compute</i> , 得到最终可复用的数据块访 存模式	87
图 4.15	上采样计算模式优化	88
图 4.16	上采样重排优化	89
图 4.17	内存复用优化的消融实验	92
图 4.18	上采样优化的消融实验	93
图 4.19	本工作与 SotA Chimera 的指令生成性能对比	94
图 5.1	寒武纪系统软件整体架构	97
图 5.2	本工作应用场景的整体工作流程	97
图 5.3	Triton 编译器整体架构	98
图 5.4	MagicMind 图编译器整体架构	99
图 5.5	MLU 370 平台上手写算子库 (CNL) 与基于 Triton 和 MagicMind 加速库的性能对比	101
图 5.6	MLU 370 与 GPU T4 的端到端执行对比	102

表格清单

表 2.1 测试算子类型及规模^[53] 46

表 3.1 以存储为中心的表示原语 58

表 3.2 卷积的输入形状和输出形状映射 62

表 3.3 不同目标硬件规格 64

表 3.4 相关 SotA 调度框架比较 70

表 4.1 Range 和 Pattern 的对应关系 80

表 4.2 Range 和 Pattern 的表达式抽象 80

表 4.3 访存模式合并后的 Interval 表示 86

第1章 绪 论

1.1 研究背景与研究意义

人工智能技术的快速发展主要得益于三个关键要素的协同进步：算法创新、算力提升和数据积累。首先，深度学习算法的突破性进展为人工智能应用提供了强大的理论基础和实现手段。从卷积神经网络到 Transformer 架构，算法创新不断突破传统方法的性能瓶颈，为计算机视觉、自然语言处理等领域带来了革命性变革。其次，智能处理器的快速发展为人工智能计算提供了强大的硬件支持。从中央处理器（CPU）到智能处理器，计算架构的演进显著提升了深度学习模型的训练和推理效率。最后，海量数据的积累为人工智能模型的训练和优化提供了丰富的素材，使得模型能够学习到更加复杂的特征和模式。然而，要充分发挥这些要素的潜力，实现人工智能技术的广泛应用，还需要系统软件的有效支持。深度学习编译器作为连接算法与硬件的关键桥梁，通过自动化的代码生成和优化技术，能够显著提升开发效率，充分利用硬件计算能力，从而推动人工智能技术的落地应用。

1.1.1 深度学习算法的发展

深度学习算法的发展历程可以追溯到卷积神经网络（CNN）的提出与广泛应用。CNN 作为深度学习早期的重要代表，最早由 Yann LeCun 等人于 1980 年代末提出，并在 1998 年通过 LeNet-5 模型在手写数字识别任务中取得了显著成功^[1]。CNN 通过卷积层、池化层和全连接层的多层结构，实现了对图像等网格数据的高效特征提取。其核心特点在于权值共享和局部感知机制，能够有效处理空间不变性任务，如图像分类、目标检测和语义分割。权值共享机制通过减少参数数量，降低了模型的计算复杂度，而局部感知机制则通过卷积核的滑动操作，捕捉图像中的局部特征，从而避免了传统全连接网络中参数爆炸的问题。

CNN 的发展历程可以分为几个关键阶段。在早期，CNN 主要用于简单的图像分类任务，如手写数字识别和人脸识别。随着计算能力的提升和大规模数据集的出现，CNN 在 2012 年迎来了突破性进展。AlexNet 在 ImageNet 图像分类挑战赛中以显著优势获胜，证明了深度 CNN 在复杂视觉任务中的强大能力^[2]。此后，CNN 的架构不断演进，出现了 VGGNet、GoogLeNet 和 ResNet 等一系列经典模型。VGGNet 通过增加网络深度提升了特征提取能力^[3]，而 ResNet 则通过引入残差连接解决了深层网络的梯度消失问题^[4]。这些模型的成功推动了 CNN 在计算机视觉领域的广泛应用。

在应用场景方面，CNN 不仅在图像分类任务中表现出色，还在目标检测、语

义分割、图像生成和视频分析等领域取得了显著成果。目标检测任务中，R-CNN 系列模型通过将 CNN 与区域建议网络结合，实现了对图像中目标的精确定位和分类^[5]。语义分割任务中，全卷积网络（FCN）通过将全连接层替换为卷积层，实现了像素级的分类^[6]。此外，CNN 还被广泛应用于医学影像分析、自动驾驶和智能监控等领域。例如，在医学影像分析中，CNN 能够自动识别病变区域，辅助医生进行诊断^[7]；在自动驾驶领域，CNN 用于实时检测道路上的行人、车辆和交通标志^[8]。

然而，尽管 CNN 在图像处理任务中表现出色，其算子种类繁多，网络结构复杂，尤其在处理长距离依赖关系时表现不佳。例如，在自然语言处理（NLP）任务中，文本数据通常具有长距离依赖特性，即句子中相隔较远的词可能存在语义关联。CNN 的局部感知机制难以捕捉这种长距离依赖关系，这限制了其在序列数据（如文本）中的应用。为了解决这一问题，研究者们开始探索新的架构，最终催生了 Transformer 模型的出现。

Transformer 的核心创新在于引入了自注意力机制（Self-Attention），能够动态计算输入序列中任意两个元素之间的关联度，从而捕捉长距离依赖关系^[9]。与 CNN 不同，Transformer 不依赖固定的卷积核，而是通过多头注意力机制（Multi-Head Attention）并行处理全局信息，显著提升了模型的表达能力。Transformer 的编码器-解码器架构为大规模预训练模型（如 BERT 和 GPT）提供了灵活的框架，推动了 NLP 领域的快速发展^[10]。

尽管 Transformer 最初是为 NLP 任务设计的，但其强大的特征提取能力很快被应用于计算机视觉（CV）领域。Vision Transformer（ViT）通过将图像分割为固定大小的 patch，并将其作为输入序列，成功实现了与 CNN 相当的图像分类性能^[11]。与 CNN 相比，ViT 的架构更加简洁，且在大规模数据集上表现出更强的扩展性。此外，Swin Transformer 等变体通过引入局部窗口机制，进一步提升了 Transformer 在目标检测和图像分割等任务中的表现^[12]。

基于 Transformer 架构的大规模语言模型（LLM）和视觉语言模型（VLM）是当前深度学习的前沿方向。LLM 如 GPT-3 通过海量参数和预训练技术，实现了对自然语言的强理解能力和生成能力^[13]；而 VLM 如 CLIP 则通过对比学习方法，将图像与文本进行联合建模，实现了多模态任务的零样本学习能力^[14]。然而，这些模型的训练和推理对计算资源提出了极高的要求，尤其是在推理阶段，KV 缓存优化和分阶段处理（如 Prefill 和 Decode 阶段）成为提升效率的关键。Prefill 阶段主要涉及大规模矩阵运算，是计算瓶颈所在；而 Decode 阶段则由于需要频繁访问 KV 缓存，成为访存瓶颈^[15]。

深度学习算法的发展将继续围绕 Transformer 架构展开，探索更高效的计算范式和多模态融合技术。未来的研究方向可能包括：（1）优化 Transformer 的计算

瓶颈，如通过稀疏注意力机制降低计算复杂度^[16]；(2) 结合 CNN 与 Transformer 的优势，设计混合架构以适应不同任务需求^[17]；(3) 推动 LLM 和 VLM 在实际应用中的落地，如自动驾驶、医疗诊断和智能交互等领域。这一发展历程不仅体现了技术架构的演进，也反映了计算范式的革新，为人工智能的广泛应用提供了坚实的基础，同时也为未来的研究指明了方向。

1.1.2 智能处理器

智能处理器的出现源于传统 CPU 架构在人工智能计算中的局限性。CPU 作为通用计算处理器，其设计初衷是优化单线程任务的执行效率，通过指令流水线、分支预测和数据缓存等技术实现快速顺序操作。这种架构在处理复杂逻辑和低延迟任务时表现出色，如操作系统调度和数据库管理等。然而，随着深度学习和大规模并行计算的兴起，CPU 在矩阵运算和并行数据处理方面的不足日益凸显。研究表明，CPU 的并行计算能力难以满足 AI 任务对高吞吐量的需求，这促使了新型处理器架构的探索^[18]。

智能处理器的核心设计理念是面向大规模并行计算优化。与 CPU 不同，智能处理器将计算资源重点配置在数据处理单元上，如浮点运算单元和张量核心，而非复杂的数据缓存和流控制机制。这种设计策略使得处理器能够同时执行数千个线程，显著提升并行任务的处理效率。更重要的是，智能处理器采用计算掩盖内存访问延迟的技术，通过优化数据流和计算模式来减少对昂贵大容量缓存的依赖。研究表明，这种架构在深度学习任务中能够实现显著的性能提升^[19]。从指令集架构的角度，智能处理器主要分为 SIMT（单指令多线程）和 SIMD（单指令多数据）两种类型，它们在深度学习计算中展现出不同的性能特征和适用场景。

SIMT 架构以 NVIDIA GPU 和 AMD GPU 为代表，其核心特征是单个指令可以同时控制多个线程的执行。这种架构在处理图形渲染和深度学习矩阵运算等高度并行任务时表现出色，具有优异的灵活性和可扩展性。以 CUDA 编程模型为例，其线程级并行机制允许开发者通过简单的代码实现高效的并行计算。以下代码展示了 SIMT 架构下向量加法的典型实现：

```
1 __global__ void vectorAdd(float* A, float* B, float* C, int N) {  
2     int idx = blockIdx.x * blockDim.x + threadIdx.x; // Thread Index  
3     if (idx < N) {  
4         C[idx] = A[idx] + B[idx]; // single instruction, multiple threads.  
5     }  
6 }
```

Listing 1.1 基于 SIMT 架构的并行向量加法实现（以 NVIDIA CUDA 为例）

上述代码展示了 SIMT 架构下并行向量加法的典型实现。在 CUDA 编程模型中，

每个线程独立执行相同的指令流，但处理不同的数据元素。具体而言，代码通过 `blockIdx.x` 和 `threadIdx.x` 的组合计算全局线程索引，实现了线程级并行（Thread-Level Parallelism）。这种编程范式充分利用了 GPU 的硬件特性：首先，线程调度由硬件自动管理，开发者无需显式处理线程调度；其次，内存地址计算和访问由硬件自动完成，程序员无需手动计算内存地址和进行复杂的指针操作，大大简化了并行编程的复杂度；最后，指令执行顺序的调度由硬件自动优化，开发者可以专注于算法逻辑的实现，而无需考虑指令级并行优化。

然而，SIMT 架构在处理深度学习任务时面临线程分支分散问题（control divergence），这可能导致计算资源的浪费和效率下降^[20]。具体而言，由于 SIMT 架构中所有线程执行相同的指令流，当线程束（warp）中的线程执行不同分支时（如部分线程执行 if 分支，其他线程执行 else 分支），硬件需要串行执行所有分支路径。在 if 分支执行期间，else 分支的线程必须等待；反之，当 else 分支执行时，if 分支的线程则处于空闲状态。这种执行方式虽然保持了 SIMT 的架构特性，但会导致计算单元利用率降低。此外，如果多个线程访问相同数据，还可能造成不必要的寄存器/访存资源浪费。

相比之下，SIMD 架构在深度学习计算中展现出独特优势。以寒武纪 MLU 和华为 Ascend IPU 为代表的 SIMD 处理器，通过单条指令同时操作多个数据元素，特别适合处理规则的数据并行任务。这种架构不仅能够显著提升计算吞吐量，还通过减少指令解码和调度的开销来优化计算效率。以下代码展示了 SIMD 架构下向量加法的典型实现：

```
1 mlu_entry void vectorAdd(float* A, float* B, float* C, int N) {
2     __nram__ float a[128], b[128], c[128];
3     __memcpy(a, A, 128 * sizeof(float), GDRAM2NRAM);
4     __memcpy(b, B, 128 * sizeof(float), GDRAM2NRAM);
5     __bang_add(c, a, b, 128); // single instruction, multiple data.
6     __memcpy(C, c, 128 * sizeof(float), NRAM2GDRAM);
7 }
```

Listing 1.2 SIMD 架构下的向量加法实现（以 MLU BANGC 为例）

SIMD 架构在内存管理和计算模式上与 SIMT 架构存在显著差异，主要体现在显式内存管理、数据访问模式和向量化计算三个方面。首先，在内存管理方面，SIMD 架构要求程序显式管理片上存储空间，包括为每个变量（如 a、b、c）分配固定大小的片上存储区域（例子中为 128 个 float 类型数据），这种精细化的内存控制机制能够优化数据局部性并减少访存开销。其次，在数据访问模式上，SIMD 架构采用显式的数据搬运指令（如 `__memcpy`）实现全局内存与片上高速缓存（NRAM）之间的数据传输，这种设计需要开发者根据数据块的维度和连续性特征进行精确控制，以实现高效的数据流管理。最后，在计算模式上，SIMD 架构采用向量化计算指令（如 `__bang_add`）直接操作 128 个数据元素，避免了

SIMT 架构中基于线程索引的循环处理方式，这种粗粒度的向量操作不仅简化了编程模型，还显著提升了计算密度和能效比。研究表明，SIMD 架构有效解决了 SIMT 架构中的线程分支分散问题，在深度学习计算场景中表现出更好的性能^[21]。具体而言，SIMD 架构通过固定宽度的向量化计算，避免了线程调度开销，同时通过优化的内存访问模式，提高了数据吞吐率。

智能处理器的发展经历了从计算优化到存储优化的演进过程。在早期深度学习时代，处理器设计主要聚焦于提升卷积神经网络（CNN）的计算效率，通过增加计算单元数量、优化矩阵乘法运算等方式来加速模型训练和推理。然而，随着大规模语言模型（LLM）的兴起，计算资源不再是唯一瓶颈，存储和访存带宽成为关键制约因素。LLM 的推理过程需要频繁访问庞大的 KV 缓存，这导致传统以计算为中心的架构难以充分发挥性能。为此，新一代智能处理器开始采用创新的存储架构，如分层缓存系统、近存计算技术等，以优化 KV 缓存的访问效率，降低访存延迟。同时，处理器还引入了智能预取、数据压缩等技术，进一步提升带宽利用率，满足 LLM 对高吞吐量、低延迟的需求。

智能处理器的演进反映了人工智能计算需求的变迁。从早期的计算优化到当前的存储和带宽优化，再到未来的可持续性和智能化发展，智能处理器不断突破传统架构的限制。随着 AI 技术的快速发展，智能处理器将在模型训练、推理加速和多模态计算等领域发挥更加重要的作用，推动计算技术的持续革新和人工智能应用的广泛落地。

1.1.3 深度学习编译器

在深度学习技术发展的早期阶段，模型部署主要依赖于手工编写硬件指令集代码，在这种开发模式下，随着深度学习技术的快速发展，算法快速迭代与硬件架构复杂化之间的矛盾日益凸显，同时跨平台部署对软件可移植性的迫切需求也愈发强烈。

首先，深度学习算法的快速演进要求开发者能够在多样化的智能处理器架构上高效实现新算法。然而，现代智能处理器的多级存储层次（包括全局内存、共享内存和寄存器文件）和多核编程模型（如线程块、线程束和计算单元）显著增加了程序开发的复杂度。开发者不仅需要深入理解算法逻辑，还必须掌握硬件架构细节，包括内存访问模式优化、数据局部性管理和并行计算调度等。这种跨领域的专业知识需求大大提高了开发门槛，使得算法实现与性能优化成为一项耗时且容易出错的工作。

其次，智能处理器架构的多样性和快速迭代对软件的可移植性和可重用性提出了严峻挑战。从智能处理器架构层面来看，不同硬件平台在指令集抽象、计算单元组织和内存层次结构等方面存在显著差异。以 NVIDIA GPU 和寒武纪 MLU

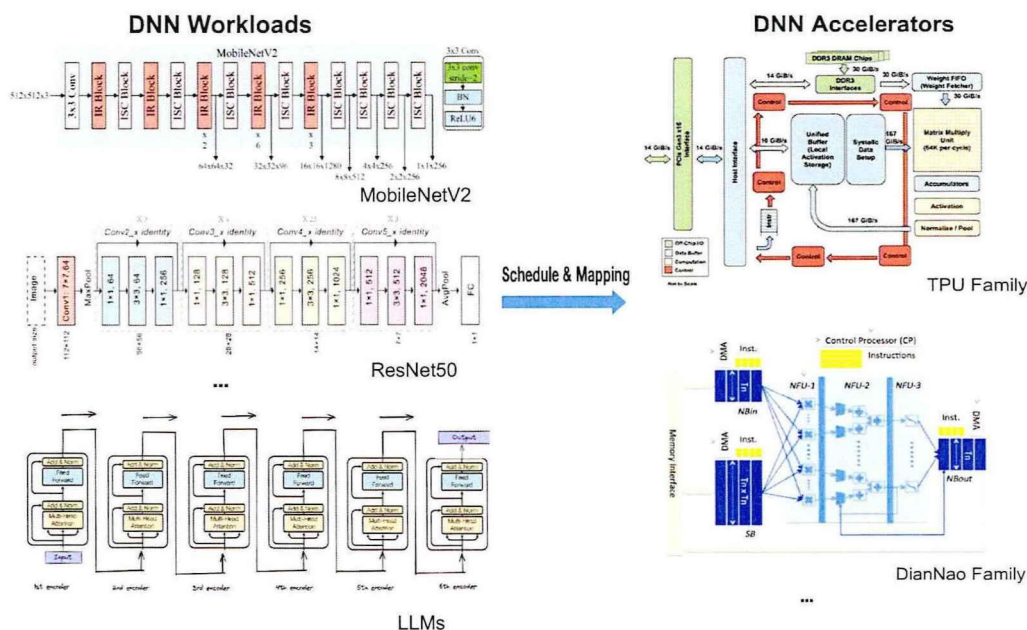


图 1.1 深度学习算法与智能处理器架构的协同优化需要编译器作为桥梁

为例，前者采用线程级并行的 SIMT 模型，通过 CUDA 编程模型实现大规模并行计算；后者则采用数据级并行的 SIMD 模型，通过向量化指令集实现高效计算。这种架构差异导致针对特定平台优化的代码难以直接移植到其他平台，增加了软件开发成本。从技术演进角度来看，硬件架构的快速迭代（遵循摩尔定律提出的芯片性能每 18 个月翻倍的规律）进一步加剧了软件维护的复杂性。开发者需要持续跟踪硬件更新，频繁调整代码以适应新硬件特性，这不仅增加了开发成本，也影响了软件系统的长期可维护性。

为了解决上述问题，深度学习编译器应运而生，其核心使命在于实现程序性能的自动化优化和跨平台代码生成，如图 1.1 所示。这一目标的实现主要面临两个关键挑战：

- 在性能优化维度，编译器需要构建完整的硬件优化体系，涵盖内存访问模式优化、并行计算调度和指令级优化等多个层面。这要求编译器具备强大的程序分析能力，能够准确识别程序的计算特征和数据访问模式，并自动应用最优的优化策略。
- 在跨平台支持维度，编译器需要建立层次化的中间表示（IR）体系，通过多级转换机制将高层算法描述映射到不同硬件平台的指令集。这种分层设计不仅确保了代码的可移植性，还通过硬件特定的后端优化实现了性能的最大化。

通过系统性地解决这两个核心问题，深度学习编译器显著降低了算法开发的复杂度，提高了代码的可维护性，并加速了新算法在多样化硬件平台上的部署进程。

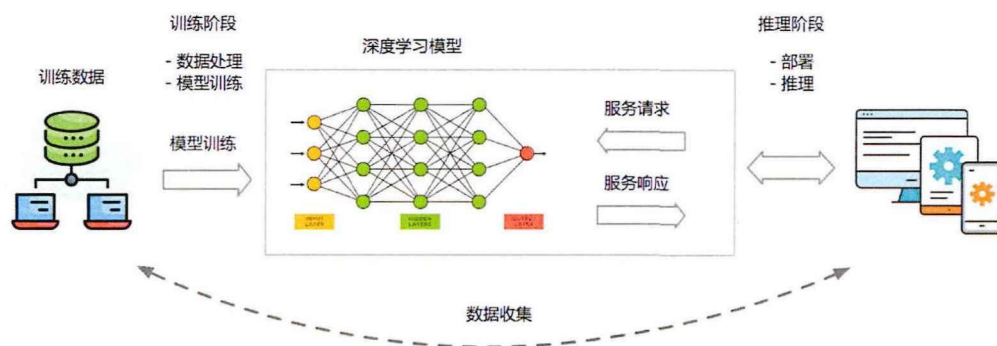


图 1.2 深度学习任务的整体流程

深度学习编译器的发展源于训练与推理场景对硬件优化的差异化需求。深度学习任务的整体流程如图 1.2 所示，可以将深度学习任务分为训练阶段和推理阶段。作为连接算法与硬件的关键桥梁，编译器需要针对训练阶段的灵活性和推理阶段的确定性分别设计优化策略。训练过程要求保存每个算子的中间结果用于反向传播，并支持动态调整模型结构，这使得算子级别的独立优化成为重点；而推理过程则可以利用模型固定的特点，通过全局优化（如算子融合、内存复用等）显著提升执行效率。

针对不同的深度学习场景，现代深度学习编译器可分为算子编译器和计算图编译器两类技术路线。算子编译器专注于单个算子的高效实现，通过自动向量化、指令调度等技术提升基础算子的硬件利用率，主要服务于对灵活性要求较高的训练场景；计算图编译器则从全局视角优化整个模型的计算图，通过算子融合、内存规划等技术减少数据搬运开销，更适用于追求极致性能的推理场景。这两类编译器相互补充，共同构成了完整的深度学习编译技术栈。

算子编译器主要面向硬件厂商提供的底层编程语言，其核心目标是实现高效的单算子级优化。以 NVIDIA GPU 的 CUDA 语言和寒武纪 MLU 的 BANG C 语言为代表的底层编程语言，虽然在指令集的基础上进行了语法层面的封装，但仍要求开发者深入理解硬件体系结构。具体而言，开发者需要手动进行数据块拆分、内存管理、指令调度等底层优化，这不仅增加了开发难度，也限制了代码的可移植性和复用性。

为了降低开发门槛并提升代码的可移植性，学术界提出了高层次抽象编程语言的解决方案。这类方案通过引入更高级的编程抽象，使开发者能够专注于算法逻辑的实现，而无需过多关注底层硬件细节。以 TVM^[22] 为代表的开源深度学习编译器框架，采用类 Python 语法进行算法描述，并提供了自动内存管理、指令调度等优化功能。这种高层次抽象不仅显著降低了开发难度，还通过中间表示 (IR) 层的设计实现了跨平台部署能力，使得同一份代码可以在不同硬件架构上高效运行。

与算子编译器专注于单个算子的优化不同,计算图编译器着眼于整个深度学习模型的全局优化。其核心功能是将深度学习模型从框架描述转换为特定硬件平台的高效代码实现。这一过程通常包括模型导入、计算图优化、硬件相关优化和代码生成等步骤。首先,计算图编译器从主流框架(如 TensorFlow、PyTorch)中导入模型,并将其转换为中间表示(IR),通常称为计算图(Graph IR)。计算图是深度学习模型的计算和控制流的抽象表示,与硬件无关,便于进行全局优化。在计算图优化阶段,编译器会应用一系列优化技术,如算子融合、代数简化、死代码消除等,以减少计算冗余并提高计算效率。随后,编译器将优化后的计算图转换为低阶 IR,进行硬件相关的优化,如内存分配、并行化、循环优化等,最终生成针对特定硬件平台的高效代码^[23]。

深度学习编译器的设计架构通常分为前端和后端两部分,中间通过多级 IR 连接。前端负责与硬件无关的优化,后端则专注于硬件相关的优化和代码生成。前端的主要任务是将深度学习模型转换为计算图,并应用图级优化技术。这些优化技术包括节点级优化(如节点消除和节点替换)、块级优化(如代数简化和算子融合)以及数据流级优化(如公共子表达式消除和静态内存规划)。后端的主要任务是将高阶 IR 转换为低阶 IR,并执行硬件相关的优化,如硬件内在映射、内存延迟隐藏、循环优化等。此外,深度学习编译器还广泛采用自动调优技术,通过机器学习方法优化算子的性能参数,以进一步提高代码的执行效率^[23]。

深度学习编译器的出现极大地简化了深度学习模型的部署过程,尤其是在边缘设备和嵌入式系统中的应用。传统的深度学习框架在部署模型时,往往需要依赖硬件厂商提供的优化库(如 cuDNN、MKL-DNN),这些库虽然能够提供高效的算子实现,但无法充分利用深度学习芯片的潜力。深度学习编译器通过自动化的代码生成和优化技术,能够更好地利用硬件资源,提高模型的性能和效率。此外,深度学习编译器还支持多种硬件平台,包括 CPU、GPU、TPU 以及定制加速器,为深度学习模型的跨平台部署提供了便利^[24]。

深度学习编译器的发展离不开与传统编译器的技术融合。传统编译器注重于优化寄存器使用和指令集匹配,其优化往往偏向于局部。而深度学习编译器的优化则涉及全局的改写,包括内存优化、算子融合等。深度学习编译器还引入了多级 IR 的设计,高阶 IR 用于表示计算和控制流,低阶 IR 用于硬件相关的优化和代码生成。这种分层设计使得深度学习编译器能够更好地适应不同硬件平台的需求,同时保留了优化的灵活性和可扩展性^[25]。

深度学习编译器的研究还涉及多种优化技术,如整数集分析、多面体模型和机器学习驱动的自动调优。整数集分析是一种用于优化循环迭代范围的技术,通过表示迭代变量之间的依赖关系,实现程序的分析 and 变换。多面体模型则是一种基于线性约束的优化方法,能够处理复杂的循环嵌套和内存访问模式^[26]。此外,

深度学习编译器还广泛采用机器学习方法来自动优化算子的性能参数,如 TVM 中的 AutoTVM 技术,通过机器学习模型预测不同参数配置的性能,从而找到最优的优化方案^[27]。

深度学习编译器的未来发展将更加注重与 AI 芯片的协同设计。随着 AI 芯片的灵活性不断提高,编译器在硬件优化中的作用将越来越重要。未来的 AI 芯片设计将更加注重与编译器的协同优化,以实现更高的性能和能效。此外,深度学习编译器还将继续探索多模态优化技术,以支持更复杂的深度学习模型和应用场景^[28]。

综上所述,深度学习编译器通过自动化的代码生成和优化技术,极大地简化了深度学习模型的部署过程,并提高了模型的性能和效率。其核心功能包括模型导入、计算图优化、硬件相关优化和代码生成,设计架构通常分为前端和后端两部分,通过多级 IR 连接。深度学习编译器的发展离不开与传统编译器的技术融合,未来还将更加注重与 AI 芯片的协同设计和多模态优化技术的探索。

1.2 国内外研究现状

1.2.1 智能处理器的发展现状

智能处理器的发展经历了从通用计算到专用计算的演进过程,其架构设计不断适应人工智能计算的需求变化。本节将从 CPU、SIMT 类型处理器和 SIMD 类型处理器三个维度,系统分析各类处理器的技术特点、优势与局限性。

1. CPU 架构的演进与局限

CPU 作为传统计算的核心,其设计理念始终围绕单线程性能优化展开。通过指令流水线、分支预测和数据缓存等技术的协同优化,CPU 在处理复杂逻辑和低延迟任务时展现出显著优势。然而,随着深度学习和大规模并行计算的兴起,CPU 架构在 AI 计算中的局限性日益凸显。

为应对 AI 计算需求,现代 CPU 通过集成 SIMD 指令集扩展和多核架构来提升并行处理能力。Intel 的 AVX-512 指令集支持 512 位宽的数据并行处理,显著加速了矩阵运算;AMD 的 Ryzen 系列和 Intel 的 Core i9 系列处理器通过多核架构和超线程技术,在 AI 推理任务中表现出色。然而,CPU 的指令集架构在处理大规模并行计算时仍存在效率瓶颈,其性能难以与专用 AI 加速器相媲美。

2. SIMT 类型处理器

SIMT(单指令多线程)架构以 GPU 为代表,通过大规模并行化设计显著提升了计算吞吐量。NVIDIA 的 CUDA 核心(CUDA core)是 SIMT 类型的典型代表,其核心创新在于将相同指令广播给多个线程,每个线程独立处理一个数据,从而实现高效的数据并行计算。

从 Tesla 架构到最新的 Hopper 架构, NVIDIA GPU 不断优化 SIMT 执行效率。Tesla 架构引入流式多处理器 (SM) 和线程束 (Warp) 调度机制, 显著提升了图形渲染和通用计算性能。Volta 架构则进一步集成 Tensor Core, 专门用于加速深度学习中的矩阵乘法运算, 支持混合精度计算, 显著提升了 AI 模型的训练和推理速度。AMD 的 RDNA 架构也采用类似设计, 通过优化线程调度和内存访问, 提升了图形处理和 AI 计算性能。

SIMT 架构的优势在于其编程灵活性和可扩展性, 能够动态分配计算资源以适应不同任务需求。然而, 该架构在处理深度学习任务时仍面临指令译码和调度效率差等挑战, 这在一定程度上限制了其计算效率的进一步提升。具体而言, 指令译码效率低主要体现在: (1) SIMT 架构需要为每个线程单独译码指令, 导致译码开销随线程数量线性增长^[29]; (2) 深度学习任务中频繁的矩阵运算需要大量重复指令, 但 SIMT 架构无法有效复用已译码指令^[30]。在调度效率方面, 主要问题包括: (1) 线程分支发散导致部分计算单元闲置, 降低了硬件利用率^[31]; (2) 内存访问延迟难以被有效掩盖, 导致计算单元等待数据的时间增加^[32]; (3) 复杂的线程调度机制引入了额外的开销, 特别是在处理大规模并行任务时, 调度器的负担显著增加^[33]。这些因素共同制约了 SIMT 架构在深度学习计算中的性能表现。为了解决这些问题, SIMD 架构的深度学习处理器应运而生。

3. SIMD 类型处理器

SIMD (单指令多数据) 架构通过单条指令同时操作多个数据元素, 特别适合处理规则的数据并行任务。寒武纪 MLU 和华为 Ascend IPU 是 SIMD 类型的典型代表, 其设计目标是通过高效的向量化计算提升 AI 任务执行效率。

寒武纪 MLU 采用专用向量处理单元 (VPU), 通过优化数据通路和指令集, 显著提升了深度学习任务中的矩阵运算性能。MLU370 处理器支持 128 个并行计算单元, 可在单个时钟周期内完成大规模矩阵乘法运算。华为 Ascend IPU 则集成专用张量处理核心 (TPC), Ascend 910 处理器支持 512 个并行计算单元, 显著提升了 AI 模型训练效率。

SIMD 架构的优势在于其高效的数据并行处理能力, 通过减少指令解码和调度开销, 显著优化了计算效率。然而, 该架构在生态建设和编程易用性方面仍存在不足, 其接受度和普及度远不及 SIMT 类型的 GPU。

4. 智能处理器的发展趋势

智能处理器的发展现状反映了从通用计算到智能计算的演进过程。CPU 通过集成 SIMD 指令集和多核架构提升了数据并行处理能力, 但在处理 AI 任务时仍存在效率瓶颈; SIMT 类型处理器通过 GPU 架构和 CUDA 编程模型解决了大规模并行计算问题, 但在线程分支 divergence 和内存访问延迟方面仍有优化空间; SIMD 类型处理器通过专用向量化计算单元提升了 AI 任务执行效率, 但在

生态建设和编程易用性方面存在不足。

未来,智能处理器将继续朝着高性能、低功耗和智能化的方向发展。一方面,处理器架构将更加注重计算与存储的协同优化,通过近存计算、智能预取等技术提升访存效率;另一方面,处理器设计将更加注重能效优化,通过稀疏计算、混合精度等技术降低功耗。此外,智能处理器还将更加注重生态建设,通过完善编程模型和开发工具链,降低开发门槛,推动 AI 技术的广泛应用。

1.2.2 深度学习编译器的发展现状

深度学习编译器作为连接深度学习模型与硬件平台的关键技术,近年来在学术界和工业界都取得了突破性进展。根据其功能定位和技术特点,深度学习编译器可分为算子编译器和深度学习计算图编译器两大类,这两类编译器在优化策略和应用场景上各具特色,共同推动了深度学习技术的快速发展。

1. 算子编译器

算子编译器是指将用户基于领域专用语言(domain-specific language)撰写的算法描述程序翻译成硬件指令的软件工具。其技术演进呈现出从底层硬件编程到高层抽象优化的显著趋势,核心目标是通过不断提升硬件适配能力和优化自动化程度,实现开发者的负担减轻以及计算任务的高效执行。算子编译器的技术演进过程可划分为三个主要阶段:手工优化阶段、半自动优化阶段和全自动优化阶段。

在手工优化阶段,开发者需要直接使用硬件汇编指令进行编程,这不仅要求开发者深入理解硬件架构细节,还极大地增加了编程复杂度。这一阶段的典型特征是高度依赖开发者的硬件知识储备,特别是在内存管理、数据对齐和并行化等方面需要手动优化,才能实现高性能计算。随后,基于汇编指令封装的 C/C++ 接口在一定程度上降低了编程门槛,开发者可以通过高级语言进行硬件编程,典型的例子是 NVIDIA CUDA C 和寒武纪 MLU BANG C 语言,其在数据类型抽象和函数封装等方面获得了显著便利。然而,这种编程模式仍然要求开发者具备深厚的硬件知识储备,特别是在内存管理、数据对齐和并行化等方面需要手动优化,才能实现高性能计算。

随着硬件架构的不断演进和多样化发展,用户对跨平台支持的需求日益增长,这推动了算子编译器进入半自动优化阶段。TVM 等深度学习编译器的出现标志着这一阶段的开始,通过引入多层中间表示(IR)和自动优化技术,实现了从硬件细节中抽象出高效的编程接口。TVM 将计算和调度分离,计算通过张量表达式表示,调度则是针对计算的一系列变换,如图 1.3所示。这种设计借鉴了 Halide^[28]等图像处理领域的优化技术,引入了循环变换、线程绑定、计算局部性等调度原语。这些调度原语能够有效地优化张量计算的执行效率,特别是在



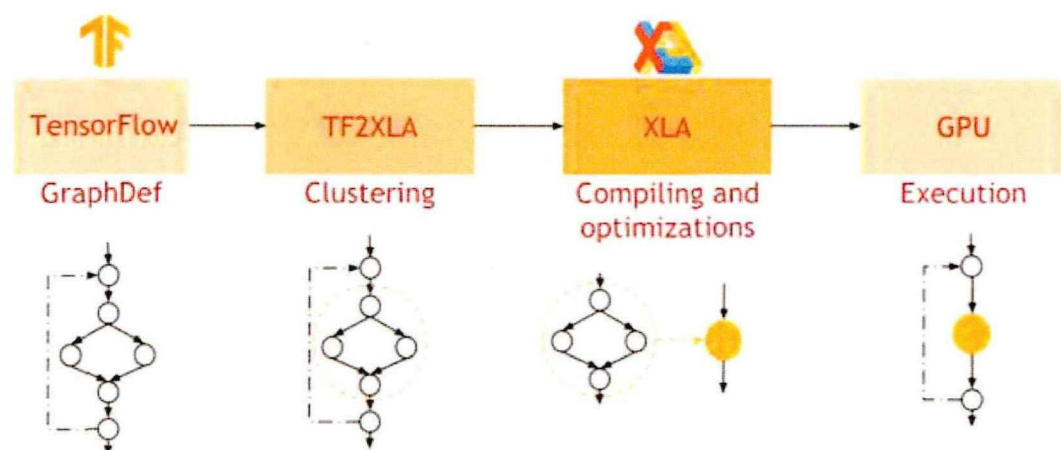
图 1.3 TVM 张量表达式优化流程，多后端调度支持^[22]

GPU 等并行计算硬件上，通过合理的线程绑定和循环变换，可以显著提升计算吞吐量并减少内存访问开销。

进入全自动优化阶段，算子编译器的优化技术经历了从手工模板到自动搜索的显著演进。早期 AutoTVM^[34] 依赖手工编写的优化模板，虽能提升特定硬件下的算子性能，但受限于模板的固定性，难以适应多样化硬件架构。随着自动调度搜索技术的发展，Ansr^[35] 通过自动探索调度空间和多层次调度原语（如循环变换、线程绑定等），实现了更灵活的优化策略，显著提升了算子执行效率和对不同硬件平台的适配能力。Triton^[36] 语言则进一步降低了用户感知的硬件信息，提供纯 Python 语法给用户用于描述算法，同时将整体编译流程抽象成更多层次的中间表示，以捕捉更多的优化机会。这种演进不仅提高了编译器的自动化程度，还显著降低了开发者的编程负担，为深度学习算子的跨平台部署提供了有力支持。

从技术演进的角度来看，算子编译器的发展呈现出明显的抽象化与自动化趋势。这一趋势主要体现在两个方面：首先，在编程接口层面，编译器通过提供更高层次的编程语言抽象，显著降低了开发者的硬件知识门槛，使得用户能够专注于算法设计而非底层实现细节；其次，在优化机制层面，编译器通过自动化的性能优化技术，将硬件相关的底层优化任务完全隐藏在编译过程中，实现了优化过程的透明化。这种发展模式不仅提升了开发效率，还通过统一的编程接口实现了跨平台兼容性，使得用户能够以单一代码库适配多种硬件架构，有效解决了异构计算环境下的开发难题。

然而，算子编译器在优化范围上存在固有局限性，其优化粒度局限于单个算子内部，无法跨越算子边界进行全局优化。这种局限性导致每个算子的输入输出都需要进行频繁的片上片外数据搬运，造成了严重的内存墙问题。为了突破这一限制，学术界和工业界逐步发展出计算图编译器技术，通过将优化范围扩展到整个计算图层面，实现跨算子的全局优化。计算图编译器通过算子融合、内存复用等技术，有效减少了算子间的数据搬运开销，显著提升了深度学习模型的整体执

图 1.4 TensorFlow XLA^[37] 编译流程

行效率。

2. 深度学习计算图编译器

深度学习计算图编译器作为连接深度学习模型与硬件平台的关键技术,其核心任务是通过多层次优化将高层的计算图表示转换为高效的底层硬件代码。当前主流计算图编译器在架构设计和优化策略上呈现出两大显著特征:多层次中间表示(IR)设计和算子融合技术,这些创新显著提升了计算图的执行效率和硬件适配能力。

在架构设计层面,多层次 IR 设计已成为现代深度学习计算图编译器的标准范式。以 TensorFlow XLA 为例,其采用分层架构设计,将编译过程划分为高层图表示和底层算子表示两个主要阶段。如图 1.4所示,XLA 首先将高层数据流图转换为基于代数表示的中间表示(XLA HLO),在此层次上实施算子融合、内存操作消除等优化;随后将优化后的 HLO 表示进一步转换为 LLVM 中间表示或直接映射到 TPU 平台。这种分层设计不仅保持了计算图的高层抽象特性,还实现了与底层硬件的深度适配,为跨平台优化提供了灵活的基础架构。

在优化策略层面,算子融合技术^[38-40]已成为提升计算图执行效率的关键手段。该技术通过将图层优化与算子层优化相结合,有效解决了传统分层优化中存在的优化粒度不匹配问题。以 Apollo 架构为例,如图 1.5所示,其创新性地提出了开放式多层规约式融合架构,通过将不同融合方式实现为独立 Layer,在各级 Layer 中分别实施基于 polyhedral 优化的循环融合、访存密集型算子融合等优化策略。这种分层融合机制不仅能够根据硬件特性动态调整优化策略,还能有效减少内存访问开销,显著提升计算图的整体执行效率。

随着深度学习模型的复杂度和硬件平台的多样性不断增加,计算图编译器在自动并行化、内存优化等方面也取得了显著进展。未来,计算图编译器将继续朝着智能化、自适应化的方向发展,通过引入机器学习驱动和优化策略和跨层次

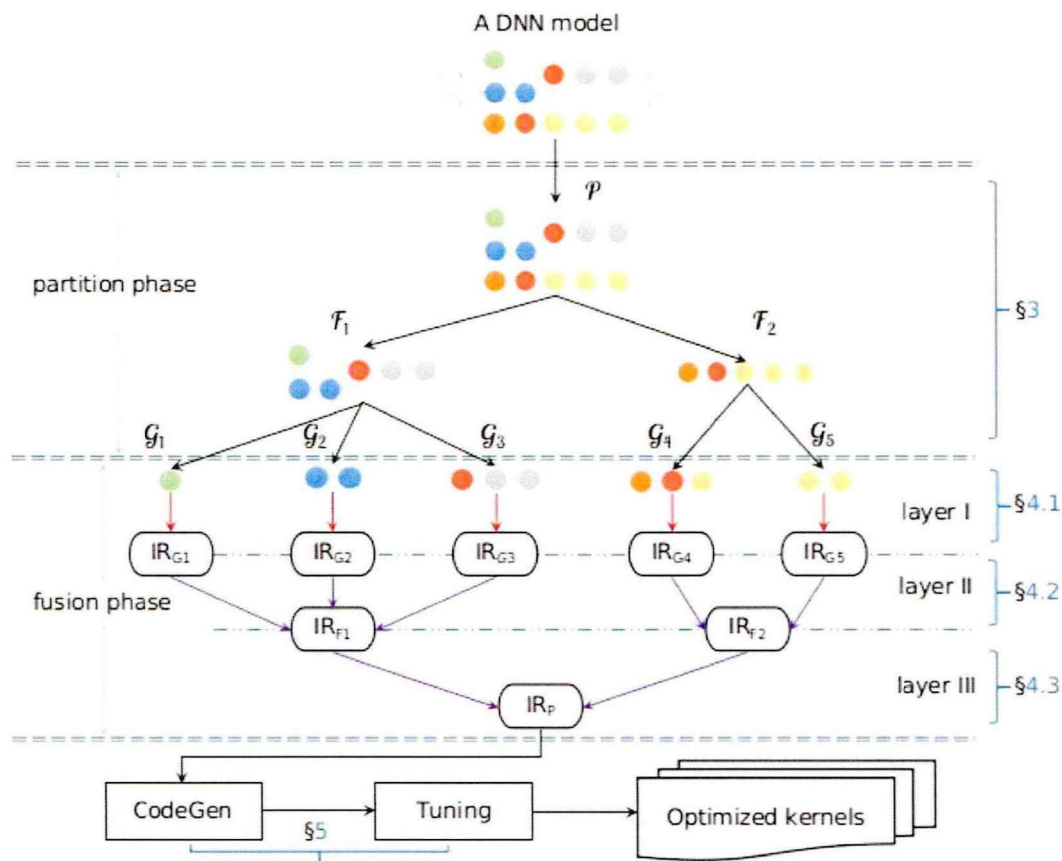


图 1.5 以 Apollo^[38] 架构为例展示典型的计算图编译流程

协同优化机制，进一步提升编译效率和生成代码的性能。

3. 未来发展趋势与挑战

算子编译器和深度学习计算图编译器在深度学习生态系统中各司其职，共同推动着深度学习技术的发展。算子编译器通过高层次抽象语言降低了编程门槛，但其优化能力主要局限于单个算法；计算图编译器则通过全局优化策略有效解决了访存瓶颈等问题，显著提升了模型执行效率。展望未来，随着硬件架构的日益多样化和模型复杂度的不断提升，深度学习编译器将朝着更高效、更灵活的方向发展。特别是在异构计算、自动调优、跨平台部署等领域，深度学习编译器将面临新的机遇与挑战，为深度学习模型的部署和优化提供更强有力的技术支持。

1.3 本文研究问题与难点

本文聚焦于智能处理器编译优化这一核心问题，从算子编译器和计算图编译器两个关键层面展开深入研究，得到”芯片设计与试制”国家级重大项目支持。在算子编译器层面，指令集适配和内存访问优化构成了主要技术挑战，这些问题

的解决直接关系到编译器对多样化硬件架构的适配能力和代码生成效率；在计算图编译器层面，计算图融合和内存层次优化则成为亟待突破的核心瓶颈，这些问题的有效处理对于提升复杂计算图的执行性能具有决定性影响。这两个层面的优化问题相互关联、层层递进，共同构成了智能处理器编译优化的完整技术体系。深入研究和解决这些问题，不仅对提升智能处理器的计算性能至关重要，也为推动智能计算系统的整体发展提供了关键支撑。

1.3.1 智能处理器的算子编译性能优化问题

智能处理器的算子编译性能优化面临着多方面的挑战。首先，SIMD 架构的处理器在任务调度和运算指令的操作数粒度上与传统的 SIMT 架构存在显著差异。SIMD 架构的编程粒度更粗，要求编译器能够有效地分析和优化数据的连续性，以充分利用其访存优势。然而，现有的编译器技术在处理这种粗粒度的向量操作时，往往无法有效地进行优化，导致性能瓶颈。

其次，现有的领域特定语言（Domain Specific Language）和编译器，如 TVM 和 Triton，虽然在 GPU 后端表现出色，但在适配 SIMD 架构的专用加速器时，存在明显的不足。这些编程语言和编译器大多针对 GPU 设计，缺乏对 SIMD 架构的深度优化和适配，限制了其在国产智能计算硬件上的应用和发展。例如，TVM 在处理 GPU 任务时，能够通过多面体编译技术对循环进行激进的优化，但在面对 SIMD 架构时，这种优化策略往往失效，因为 SIMD 架构的指令集和数据访问模式与 GPU 有显著不同。

此外，SIMD 架构的访存行为与 SIMT 架构也存在显著差异。对于 SIMT 架构下的访存行为而言，每条访存指令操作一个数，由硬件进行连续性分析，将不同数据进行访存合并从而达到高效访存的目的。然而，对于 SIMD 架构的访存而言，每条访存指令操作一块数据，从而就要求软件进行数据的连续性分析，进而加速访存连续的数据块。这对编译器设计提出了新的要求，需要编译器能够有效地分析和优化数据的连续性，以充分利用 SIMD 架构的访存优势。

综上所述，为了充分发挥国产 SIMD 架构处理器的性能潜力，扩展 Triton 语言对 SIMD 架构处理器后端的支持，设计一套全新的多层次 IR 系统显得尤为必要。这套系统需要能够设计任务调度方式、提供中间抽象层、优化数据流和控制流，以确保在不同硬件平台上的高效执行。

1.3.2 智能处理器的图融合策略优化与性能建模问题

深度神经网络（DNN）的硬件加速面临计算与存储协同优化的重大挑战。随着 DNN 在计算机视觉和自然语言处理等领域的广泛应用，其复杂的计算流程和数据依赖关系对硬件加速器提出了严峻挑战，特别是在内存层次结构设计和计

算能力优化方面。

现有工作在算子融合策略和硬件建模方面存在显著不足，亟需系统性优化与创新。从融合策略层面而言，现有工作主要存在两个关键问题：其一，基于循环分块策略的优化方法虽然能够在一定程度上改善 DNN 执行的延迟和内存占用，但其优化粒度局限于单算子层面，在处理复杂操作时会导致频繁的片外与片上内存数据搬运，严重制约了性能提升；其二，尽管部分研究提出了融合算子的优化策略以增强数据局部性，通过驻留算子间中间数据于片上空间来优化性能，但在处理大卷积权重的 CNN 时，现有框架仍难以有效解决权重超出片上内存容量的瓶颈问题，其分块策略和硬件映射能力仍有待提升。

从硬件建模层面而言，现有框架的局限性主要体现在建模方法的系统性和全面性不足。当前研究往往过度关注 DRAM 访问优化，而忽视了片上数据搬运对整体性能的影响，未能充分考虑多级内存层次结构的协同优化，导致硬件性能无法得到充分发挥。这种片面的建模方法限制了调度框架对复杂硬件架构的适配能力，难以实现最优的性能调优。

因此，开发一种兼具完备算子融合策略和系统化硬件性能建模能力的融合数据流框架，已成为当前研究领域亟待解决的关键问题。

1.3.3 智能处理器的图融合指令生成优化问题

智能处理器的图融合指令生成面临两个核心挑战：地址推导问题和上采样优化问题。在地址推导方面，首要难点在于深度融合场景下的冗余输入爆炸问题。随着融合深度的增加，特别是在包含滑窗类算子（如卷积）的情况下，相邻时间片之间的数据重叠会导致冗余输入呈爆炸性增长，这不仅浪费计算资源，还可能导致内存带宽瓶颈。其次，访存模式合并问题也是地址推导的关键挑战。在计算图存在分支结构时，同一数据可能被多个算子使用，产生多个访存模式，如何高效合并这些模式以减少重复访存成为亟待解决的问题。

在上采样优化方面，传统方法主要针对下采样操作设计，难以有效应对上采样操作中特有的数据搬运和计算模式。上采样操作中大量的零值填充和无效计算严重影响了整体性能。现有存储层次优化方案虽然通过优先使用高层级存储来减少存储层级间的数据搬运，但仍存在显著局限性：一方面，高层级存储的芯片面积代价较高，难以大规模扩展；另一方面，有限的存储空间往往成为性能瓶颈，特别是在处理大规模特征图时尤为明显。这些问题导致上采样操作的计算开销往往占据整个网络推理时间的显著比例，成为制约模型性能的关键瓶颈。

1.4 本文内容与贡献

本文的研究内容是面向 SIMD 架构智能处理器的编译技术研究。针对这一研究问题，本文以编译器的两种类型作为划分，从算子编译器和计算图编译器分别展开相关研究。本文整体的研究路线如图 1.6所示。

- (1) 针对智能处理器中的 SIMD 架构下 Triton 程序的性能优化问题，本文提出了一种全新的 Triton 语言编译流程，通过引入 Linalg 方言作为中间抽象层，解决了 Triton 在适配 SIMD 架构时的编译问题。该方案实现了数据连续性分析、向量操作融合等优化，显著提升了访存效率和代码可移植性。
- (2) 针对智能处理器中的算子融合策略与性能建模问题，本文提出了融合数据流框架 FusionFrame。该框架通过以内存为中心的调度表示，实现了算子融合、分块策略优化和多级内存层次映射，有效减少了数据搬运和 DRAM 访问，显著提升了深度学习模型的执行效率。
- (3) 针对智能处理器中的融合算子指令生成问题，本文设计了一套完整的图融合指令生成系统。该系统创新性地解决了地址推导中的冗余输入爆炸和访存模式合并问题，并提出了面向上采样这一典型场景的性能优化方法，通过优化冗余数据搬运和计算时间，为智能处理器中的融合算子指令生成提供了新的思路。

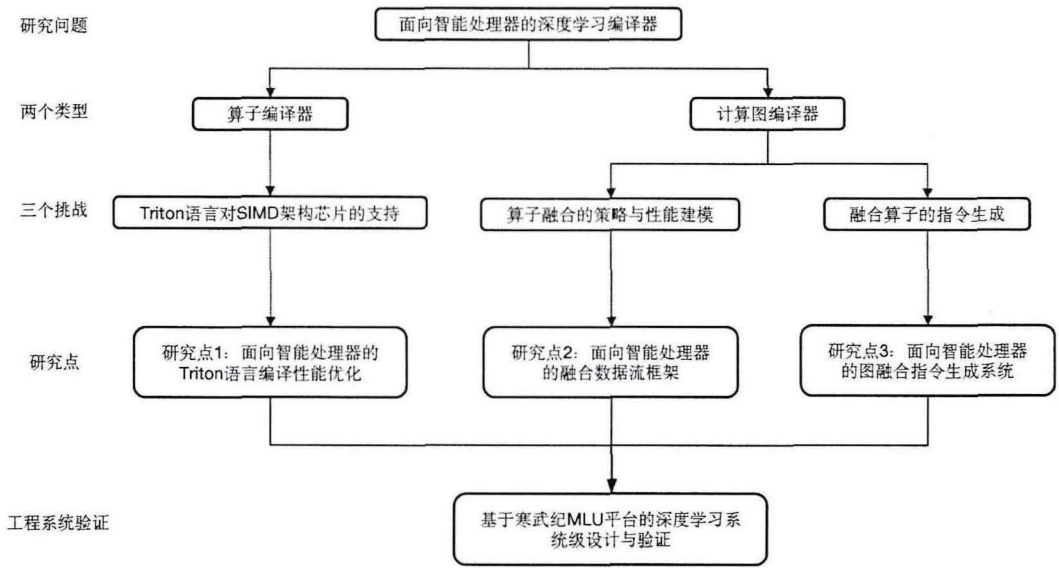


图 1.6 本文研究思路

1.4.1 面向 SIMD 架构智能处理器的 Triton 语言编译性能优化

本文提出了一种全新的 Triton 语言编译流程，旨在扩展 Triton 对 SIMD 架构处理器后端的支持。该流程借鉴了 IREE 的设计理念，采用 Linalg 方言作为连接

高层次表示与 SIMD 硬件指令集的中间抽象层。通过这一设计，本文不仅解决了 Triton 在适配 SIMD 架构时的编译问题，还为国产智能计算硬件的发展提供了强有力的支持。

首先，本文通过 Linalg 方言对 Triton 的高层次表示进行抽象，保留向量操作的语义信息。这一步骤将高级 Python 代码转换为低级的 IR 表示，为后续的优化和转换打下基础。生成的 Triton-IR 代码随后会经历一系列的简化和优化过程，确保 IR 代码既高效又适应目标硬件的特性。

其次，本文在中间表示层进行一系列的优化，如数据连续性分析、向量操作的融合等。通过这种系统性地跟踪向量数据的生成过程的方法，针对生成过程进行深入分析，可以准确获取向量数据的连续性信息，从而可以通过连续访存来显著提升性能。这种方法不仅提高了访存效率，还为后续的优化和硬件映射提供了坚实的基础。

最后，本文将优化后的中间表示转换为 SIMD 架构的硬件指令集，确保编译后的代码能够在国产智能计算硬件上高效执行。通过引入 *tensor_view* 操作和 *linalg_ext* 等方言，本工作成功地将 Triton-IR 中的高级抽象转换为适合 SIMD 架构的低级实现，同时保持了代码的清晰性和优化潜力。这种转换策略为深度学习模型在多样化硬件平台上的高效执行提供了强有力的支持。

综上所述，本文通过设计一套全新的 Triton 语言编译流程，成功解决了 Triton 在适配 SIMD 架构时的编译问题，并为国产智能计算硬件的发展提供了强有力的支持。这一工作不仅提高了代码的可移植性和可维护性，而且为深度学习编译器技术的进步提供了坚实的基础。

1.4.2 面向 SIMD 架构智能处理器的算子融合数据流评估框架

本文提出了一种面向 SIMD 架构智能处理器的融合数据流评估框架 FusionFrame，旨在解决现有算子融合框架在算子融合、分块策略和硬件映射方面的挑战。FusionFrame 通过定义全面的算子融合优化空间，提出了一种以内存为中心的算子融合策略表示方法，能够有效捕捉算子融合和内存层次结构之间的关系。

FusionFrame 采用以存储为中心的融合机制，遍历计算图以融合算子并将中间数据分层次映射到每个内存级别。该机制首先将所有算子融合并分块，将内部分块循环映射到最内层、最快的内存（如 L0 缓存），然后识别并组合外部公共循环，将其映射到更高的内存级别。这种分层映射策略显著提高了数据局部性，减少了片外与片上内存之间的数据搬运。

在分块策略方面，FusionFrame 支持在输出通道维度上进行分块，解决了现有框架在处理大卷积权重时的局限性。通过选择适当的分块维度和大小，FusionFrame 能够有效利用片上内存容量，避免了融合终止的问题。此外，FusionFrame

还提供了详细的分块形状计算方法和内存需求评估模型，确保了内存容量优化的精确性和高效性。

在硬件映射方面，FusionFrame 通过系统化的方法将 DNN 融合算子映射到多级内存层次结构，显著减少了 DRAM 访问和延迟。通过案例研究，FusionFrame 展示了其在减少 DRAM 访问和降低延迟方面的显著优势，特别是在具有多级内存层次结构的硬件平台上。这些优势源于 FusionFrame 灵活的融合数据流表示和分层融合算子策略，突显了其在优化 DNN 融合数据流中的重要性。

综上所述，FusionFrame 通过其创新的算子融合策略表示方法和以内存为中心的融合机制，显著提升了 DNN 算子融合和优化效果，为智能计算系统的进一步发展提供了坚实的技术支撑。本文的研究成果不仅丰富了 DNN 算子融合的理论体系，也为实际应用中的性能优化提供了有力的工具和方法。

1.4.3 面向 SIMD 架构智能处理器的图融合指令生成系统

本文提出了一套完整的图融合指令生成系统，针对地址推导和上采样优化两大核心问题提出了创新性解决方案。在地址推导方面，系统首先提出了一种基于 Interval 和 Region 数据结构的访存模式表达方法，能够精确描述数据块的拆分方式和访存地址。其次，系统设计了一种基于图融合的地址推导流程，通过入口算子选择、数据拆分、原子操作分解和复用优化等步骤，实现了高效的地址推导。特别地，系统提出了一种创新的访存模式合并方案，能够有效处理计算图分支结构导致的多个访存模式问题，显著减少了重复访存。

在上采样优化方面，系统突破了传统优化方法的局限，提出了一种基于池化索引的重排机制和固定偏移的索引计算优化方法。该方案通过深入分析上采样操作特有的数据搬运模式，设计针对性的优化策略，有效解决了上采样操作中大量零值填充带来的性能瓶颈问题。系统特别针对智能处理器中常见的填充操作和卷积操作在融合场景进行了优化，展示了其在实际应用中的有效性。这些创新性解决方案共同构成了一个高效的图融合指令生成系统，为提升智能处理器的性能提供了强有力的技术支持。

1.4.4 基于 MLU 平台的深度学习系统级设计与验证

本文提出的创新性工作已在寒武纪深度学习系统中得到实际应用，有效解决了系统软件优化中的关键问题。在寒武纪系统软件的整体架构中，本文工作主要应用于算子编译层和图编译层两个核心模块，通过分层架构设计和模块化实现，显著提升了系统的计算效率和易用性。

在算子编译层，本文的工作应用于 Triton 算子的编译器，实现了从高级语言到机器指令的高效转换。该框架采用多阶段优化架构，通过指针分析、计算语义

抽象等创新技术,将 Triton-Python 算子转换为硬件无关的 Linalg 表示,并进一步优化为 MLU-IR 中间表示。针对常见二元算子,本文提出了数据驻留和指令流水两项关键优化技术,在实际应用中取得了显著的性能提升。

在图编译层,本文的工作应用于 MagicMind 图编译器,通过计算图优化和指令生成技术,显著提升了系统整体性能。该编译器采用模块化设计,实现了从模型描述到指令生成的全流程优化。本文解决了地址推导等关键技术难题,并探索了针对上采样这一典型性能瓶颈场景中的性能优化方案,取得了显著的性能提升。

本文工作有效解决了寒武纪系统软件中的三个关键问题。首先,通过 Triton 编译器和 MagicMind 图编译器的协同工作,实现了对主流深度学习算法框架的良好支持,解决了深度学习框架多样性的问题。其次,通过图优化和指令生成技术,有效提升了计算效率,解决了计算图复杂性的问题。最后,通过模块化设计和优化策略,提高了系统软件的易用性和可扩展性,为国产深度学习计算平台的发展提供了重要的技术支持。

这些创新性工作已在寒武纪深度学习系统中得到实际应用,为国产深度学习计算平台的发展提供了重要的技术支持。通过实验评估验证,本文提出的方法在实际系统中取得了显著的性能提升,为深度学习系统软件的优化提供了新的思路和方法。

1.5 本文组织结构

本文围绕智能处理器的高效调度与优化展开研究,针对深度学习计算图中的关键性能瓶颈,系统性地提出了融合算子指令生成系统的优化方案。全文共分为六章,各章节内容安排如下:

第一章为绪论,阐述了深度学习计算图优化的重要性,分析了当前研究面临的挑战,并概述了本文的研究目标和主要贡献。

第二章为面向 SIMD 架构智能处理器的 Triton 语言编译性能优化。本章详细探讨了 Triton 语言在智能处理器上的编译优化策略,重点分析了其在 SIMD 架构上的性能瓶颈及优化方法。通过引入新的编译优化技术和硬件抽象机制,本章有效提升了 Triton 语言在智能处理器上的执行效率,解决了传统 SIMT 架构编译器在 SIMD 架构上的适配难题。

第三章为面向 SIMD 架构智能处理器的算子融合数据流评估框架。本章提出了 FusionFrame 框架,通过创新的三维调度空间建模和以存储为中心的表示方法,实现了 DNN 调度性能的显著提升。该框架从算子融合、循环分块和硬件映射三个维度构建了完整的调度空间,并开发了高效的分析模型,为融合数据流优

化提供了系统性的解决方案。

第四章为面向 SIMD 架构智能处理器的图融合指令生成系统。本章针对深度学习计算图中的关键性能瓶颈，提出了融合算子指令生成系统的优化方案。通过引入基于向量描述的地址推导系统，有效解决了深度融合场景下的地址推导难题。同时，针对上采样操作的特殊计算模式，提出了基于池化索引的重排机制和固定偏移的索引计算优化方法。

第五章为基于 MLU 平台的深度学习系统级设计与验证。本章系统地阐述了寒武纪深度学习系统软件的设计与优化方法，重点探讨了分层架构设计、算子编译优化和图编译优化等关键技术。作为寒武纪深度学习加速库的核心组成部分，本工作通过创新的系统架构设计和优化策略，显著提升了整体系统软件的性能表现。基于严格的系统级评估，本章通过在大语言模型和传统卷积神经网络等典型应用场景下的实验验证，充分证实了本研究内容所提出方法的有效性。

第六章为总结与展望。本章总结了本文的主要研究成果，并探讨了未来可能的研究方向，包括硬件适配的进一步扩展和更广泛的应用场景。

通过上述章节的系统性研究，本文为智能处理器的深度学习计算图优化提供了全面的解决方案，显著提升了计算效率和性能。

第2章 面向 SIMD 架构智能处理器的 Triton 语言编译性能优化

2.1 引言

智能处理器，尤其是基于 SIMD 架构的处理器，在现代计算领域中扮演着至关重要的角色。这些处理器通过单指令多数据的方式，能够高效地处理大规模并行计算任务，特别适用于深度学习和人工智能领域。然而，现有的编程模型和编译器技术在应对这些处理器的复杂性和多样性时，面临着诸多挑战。

首先，SIMD 架构的处理器在任务调度和运算指令的操作数粒度上与传统的 SMT 架构存在显著差异。SIMD 架构的编程粒度更粗，要求编译器能够有效地分析和优化数据的连续性，以充分利用其访存优势。此外，SIMD 架构的指令粒度是基于向量表达的粗粒度指令，这使得传统的编译流程在处理这些架构时，面临着不可逆转换问题，导致编译复杂性和性能优化的机会被浪费。具体而言，SIMD 架构的处理器在执行任务时，通常以向量为单位进行操作，这要求编译器在生成代码时，能够准确地识别和利用数据向量的连续性，以减少访存开销和提高计算效率。然而，现有的编译器技术在处理这种粗粒度的向量操作时，往往无法有效地进行优化，导致性能瓶颈。

其次，现有的领域特定语言（Domain Specific Language）和编译器，如 TVM 和 Triton，虽然在 GPU 后端表现出色，但在适配 SIMD 架构的专用加速器时，存在明显的不足。这些编程语言和编译器大多针对 GPU 设计，缺乏对 SIMD 架构的深度优化和适配，限制了其在国产智能计算硬件上的应用和发展。例如，TVM 在处理 GPU 任务时，能够通过多面体编译技术对循环进行激进的优化，但在面对 SIMD 架构时，这种优化策略往往失效，因为 SIMD 架构的指令集和数据访问模式与 GPU 有显著不同。此外，Triton 虽然在 GPU 后端表现优异，但其编译流程并未针对 SIMD 架构进行优化，导致在国产智能计算硬件上的性能表现不尽如人意。

针对上述挑战，本章提出了一种全新的 Triton 语言编译流程，旨在扩展 Triton 对 SIMD 架构处理器后端的支持。该流程借鉴了 IREE 的设计理念，采用 Linalg 方言作为连接高层次表示与 SIMD 硬件指令集的中间抽象层。通过这一设计，本章不仅解决了 Triton 在适配 SIMD 架构时的编译问题，还为国产智能计算硬件的发展提供了强有力的支持。具体而言，本章提出的编译流程包括以下几个关键步骤：首先，通过 Linalg 方言对 Triton 的高层次表示进行抽象，保留向量操作的语义信息；其次，在中间表示层进行一系列的优化，如数据连续性分析、向量操作

的融合等；最后，将优化后的中间表示转换为 SIMD 架构的硬件指令集，确保编译后的代码能够在国产智能计算硬件上高效执行。

针对二元算子性能优化，本章提出了数据驻留技术和流水线处理技术。数据驻留技术通过优化数据在计算单元和存储单元之间的调度，减少数据搬运开销；流水线处理技术则通过重叠计算和访存操作，提高硬件资源利用率。通过实验验证，这些优化技术在国产智能计算硬件上取得了显著效果，能够有效提升 SIMD 架构的计算性能。这些优化策略为深度学习编译器的设计提供了新的思路，特别是在处理大规模矩阵运算和卷积计算时，能够显著减少数据交换和加载时间，提高计算资源的利用率。

综上所述，本章的研究不仅为 Triton 语言在 SIMD 架构上的应用提供了新的解决方案，还为国产智能计算硬件的发展提供了重要的技术支持。通过本章的工作，期望能够推动领域特定语言和编译器技术在国产智能计算硬件上的广泛应用，为深度学习和人工智能领域的发展做出贡献。

2.2 相关背景和研究动机

2.2.1 MLIR 概述

深度学习编译器设计的核心在于如何高效地将高级算法描述转换为在特定硬件上执行的低级代码。领域特定编程语言在这一过程中扮演着至关重要的角色，它们提供了一种针对特定计算模式（如深度学习）优化的编程抽象。然而，现有的编译方法，如多面体编译（Polyhedral Compilation）和调度语言（Scheduling Languages），虽然在某些方面表现出色，但也存在显著的局限性。

多面体编译是一种高级编译技术，它通过将程序的控制流表示为多面体（polyhedral）的形式，使得编译器能够在编译时对程序进行深入的分析和优化。这种方法的核心在于将循环嵌套的迭代空间表示为一个多面体，并通过线性和整数规划技术来探索可能的变换和调度策略。多面体编译的优势在于能够对程序进行激进的变换和优化，如循环展开（unrolling）、循环融合（fusion）、循环重排（interchange）和数据局部性优化（data locality optimization）^[41-43]。然而，多面体编译也存在一些局限性。首先，多面体编译的复杂性随着程序中语句的数量和迭代域的大小而增加，这使得编译过程变得非常计算密集。其次，多面体编译通常需要程序的控制流和数据访问模式是仿射的（affine），这限制了多面体编译在处理不规则或稀疏计算模式时的应用^[44]。此外，多面体编译的自动调优过程可能需要解决复杂的整数线性规划问题，这进一步增加了编译的复杂性和成本^[45]。

调度语言是一种将算法描述与实现细节分离的编程范式。通过使用调度语

言, 开发者可以独立地定义算法的逻辑和性能优化策略。这种方法的典型代表包括 Halide 和 TVM, 它们允许开发者通过语法层面的分离来优化算法的性能^[28,46]。例如, TVM 通过 Relay 和 TensorIR IR 层, 提供了灵活且高效的模型编译和优化功能^[22]。尽管调度语言在简化编程和优化方面具有显著优势, 但它们也面临一些挑战, 如依赖特定硬件特性和处理复杂稀疏计算模式时的灵活性不足^[47]。

为了解决这些问题, MLIR (Multi-Level Intermediate Representation) 提供了一个统一的框架, 支持不同层次的中间表示 (IR), 从而简化编译流程并提高优化的灵活性。MLIR 的设计允许编译器在不同的抽象层次上进行优化, 更好地适应各种应用特定架构的特性。此外, MLIR 的可扩展性使得开发者能够根据特定需求定制 IR, 推动了领域特定编程语言的发展和创新。

MLIR 的核心优势在于支持多种抽象层次的 IR, 允许开发者针对不同的硬件平台和应用需求定义和优化领域特定编程语言。MLIR 的设计允许所有 IR 元素, 如属性、操作和类型, 进行定制化, 同时这些元素可以被简化为基本的概念。这种灵活性使得 MLIR 能够解析、表示并实现任何操作的 IR 转换。MLIR 的一个关键特性是方言 (dialect) 之间的转换能力。方言是指特定于某个领域或硬件平台的 IR 表示。每个方言都是 MLIR 生态系统中的一个独立实体, 拥有自己的操作、类型和语义规则。这些方言可以被设计为支持特定的编译任务或优化目标, 从而使得 MLIR 能够适应广泛的应用场景和硬件架构。方言之间的转换机制极大地扩展了 MLIR 的应用范围。例如, MLIR 项目中常用的方言, Linalg, 对线性代数操作, 如点对点操作、矩阵乘法和卷积等, 进行硬件无关的抽象, 提供了一种简洁的方式来描述 DNN 中常见的张量操作。

<pre> 1 // matmul 2 for (i = 0 : m) { 3 for (j = 0 : n) { 4 for (l = 0 : k) { 5 C[m][n] += A[m, k] * B[k, n]; 6 } 7 } 8 } </pre>	<pre> 1 // Linalg matmul 2 %result = linalg.generic # 3 matmul_trait 4 ins(%A, %B : tensor<?x?xf32,>, 5 tensor<?x?xf32,>) 6 outs(%C : tensor<?x?xf32,>) { 7 ^bb0(%a: f32, %b: f32, %c: f32) : 8 %d = arith.mulf %a, %b: f32 9 %e = arith.addf %c, %d: f32 10 linalg.yield %e : f32 11 } </pre>
--	--

图 2.1 矩阵乘法计算使用标量计算表示与 Linalg 方言表示的区别

以矩阵乘法算子 $A \times B = C$ 为例, 其中 A 和 B 分别是 $m \times k$ 和 $n \times k$ 的矩阵。在传统编译方法中, 这个算子会被表示为多个嵌套循环中的标量乘法和加法操作, 如图 2.1 左所示。图 2.1 右是 Linalg 方言中矩阵乘算子的定义, 可以看见通过这种结构化的表示, 可以将复杂的标量循环操作简化为抽象的 *linalg.matmul* 表

达。Linalg 方言的一个关键特性是它能够在转换过程中保留操作的语义信息，这意味着在对 Linalg 算子进行拆分 (tiling) 和融合 (fusion) 后，仍然可以保持矩阵乘法的语义。这种信息保留对于 SIMD 架构的优化至关重要，因为它允许编译器在不同层次上进行有效的优化，而不会丢失关键的语义信息。现有工作如 IREE 编译器^[48]使用 Linalg 作为中间表示，将 TensorFlow 和 PyTorch 模型通过硬件无关的转换为 Linalg 方言，并基于 Linalg 方言进行硬件相关的转换至 GPU 指令集。基于 MLIR 的编程语言，例如 Triton，已经展示了在不同硬件平台上实现高效模型执行的潜力，为开发者提供了一种直观且简洁的方式来编写 GPU 内核。同时，Triton 语言在 PyTorch^[49]中作为 GPU 硬件的官方指令生成语言，获得了社区的广泛认可。

2.2.2 Triton 语言概述

在深度学习领域，智能处理器 (GPGPU) 已成为研究和部署深度神经网络 (DNNs) 的主流平台。随着硬件架构的不断演进，传统的编程模型已难以满足日益增长的性能需求。为了应对这一挑战，学术界和工业界对领域特定编程语言和编译器产生了浓厚兴趣，旨在简化 GPU 编程并提高性能。Triton 语言正是在这样的背景下应运而生，作为一种专为智能处理器设计的编程语言，核心目标是简化 DNNs 的编程模型，同时提供高性能的计算能力。DNN 模型的层级结构生成了大量高度可并行化的工作，非常适合多核处理器执行。尽管 GPU 因计算能力而成为研究和部署 DNNs 的流行平台，但它们在优化局部性和并行性方面仍面临挑战。特别是当现有优化原语无法高效实现计算时，这一问题尤为突出。此外，GPU 架构的快速发展和专业化，如 NVIDIA 和 AMD 微架构中张量核心 (Tensor Core) 的添加，以及各种深度学习处理器架构的提出，进一步增加了编程的复杂性。

Triton 语言通过创新的编程范式，旨在解决这些挑战。它基于分块算法的编程范式，重构了传统的“单程序，多数据” (SPMD) 执行模型，提出了一种新的变体，即程序分块 (Blocked Program) 而非线程分块 (Blocked Threads)。如图 2.2 所示，这种设计的关键优势在于能够提供块结构化的迭代空间，为程序员在实现稀疏操作时提供了比现有编程语言更大的灵活性。同时，它还给编译器提供了更多数据局部性和并行性的优化机会。在 Triton 语言的设计中，编译器利用块级数据流分析技术，静态地根据目标程序的控制和数据流结构来调度迭代块。这种系统能够自动应用广泛的优化，例如自动数据合并、线程交错、预取、自动向量化、张量核心感知指令选择、共享内存分配/同步和异步复制调度等。这些优化策略不仅提高了程序的执行效率，还增强了其在不同硬件平台上的可移植性。

针对 GPU 后端，Triton 已经拥有了成熟的编译器多层次 IR 解决方案。Triton

CUDA Programming Model
(Scalar Program, Blocked Threads)

```
#pragma parallel
for(int m = 0; m < M; m++){
  #pragma parallel
  for(int n = 0; n < N; n++){
    float acc = 0;
    for(int k = 0; k < K; k++){
      acc += A[m, k] * B[k, n];
    }
    C[m, n] = acc;
  }
}
```

Triton Programming Model
(Blocked Program, Scalar Threads)

```
#pragma parallel
for(int m = 0; m < M; m += MB){
  #pragma parallel
  for(int n = 0; n < N; n += NB){
    float acc[MB, NB] = 0;
    for(int k = 0; k < K; k += KB){
      acc += A[m:m+MB, k:k+KB]
        @ B[k:k+KB, n:n+NB];
    }
    C[m:m+MB, n:n+NB] = acc;
  }
}
```

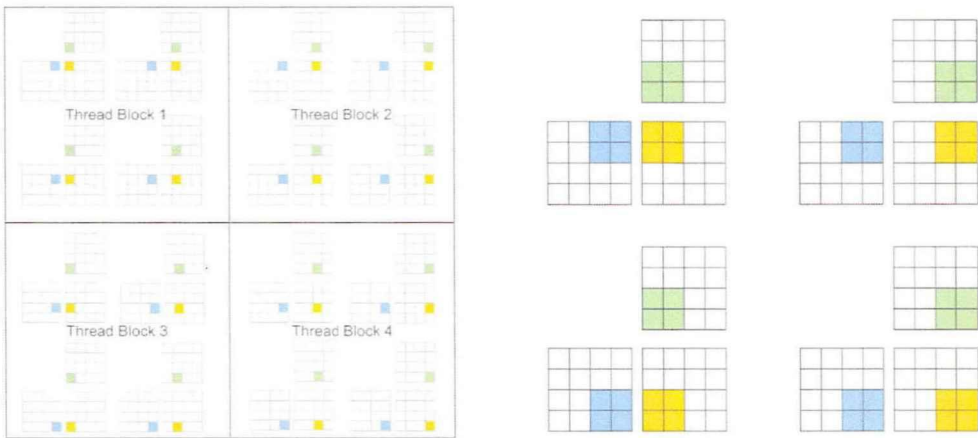
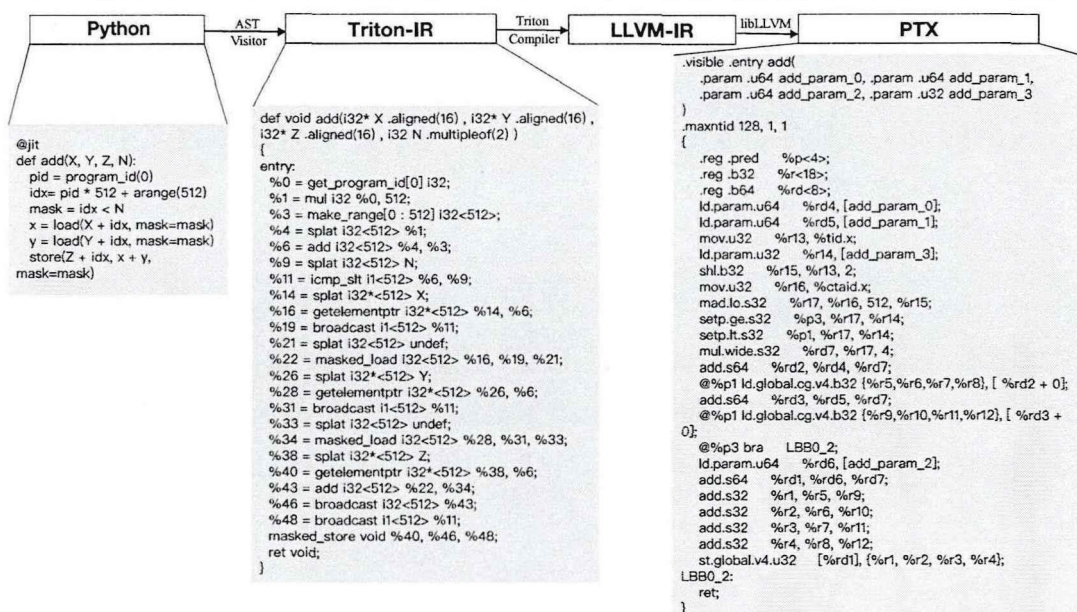


图 2.2 Triton 程序分块与 CUDA 线程分块的对比^[50]

的多层次 IR 系统设计如图 2.3 所示，核心是 Triton-IR ——一种基于 MLIR 的中间表示 (IR)，在这种表示中，多维数据块被视为一等公民。这种设计不仅提高了代码的可读性和可维护性，还为编译器的优化和转换提供了强大的基础。Triton 编译器的下降流程始于对 Python 函数的抽象语法树 (AST) 的遍历。利用 `@triton.jit` 装饰器，编译器能够即时生成 Triton-IR，采用通用的静态单赋值 (SSA) 构造算法。这一步骤将高级 Python 代码转换为低级的 IR 表示，为后续的优化和转换打下基础。生成的 Triton-IR 代码随后会经历一系列的简化和优化过程。编译器后端自动执行这些操作，确保 IR 代码既高效又适应目标硬件的特性。优化后的 IR 代码进一步被转换为高质量的 LLVM-IR，这是 LLVM 编译器基础设施中用于表示程序的中间语言。最终，LLVM-IR 被转换为 PTX (Parallel Thread Execution) 代码，这是一种专为 NVIDIA GPU 设计的低级汇编语言，确保了在现代 NVIDIA GPU 上的高效执行。

Triton 编译器在针对 NVIDIA GPU 的优化和代码生成方面表现出色，编译流程的设计充分利用了 GPU 的单指令多线程 (SIMT) 架构特性。然而，这种优化策略并不总是能够直接应用于其他类型的硬件，特别是那些采用单指令多数

图 2.3 现有 Triton 基于 GPU 后端的下降流程^[50]

据 (SIMD) 架构的领域特定架构 (Domain Specific Architecture), 例如谷歌的 TPU^[19]和寒武纪的 MLU^[21]。目前, Triton 语言的官方实现中尚不支持除 GPU 以外的其他芯片架构。

2.2.3 SIMD 架构芯片编译器设计问题与挑战

现有的领域特定编程语言, 如 TVM, 虽然在深度学习编译领域取得了一定的进展, 但它们大多针对 GPU 后端设计, 对 SIMD 架构的专用加速器的适配不足。这种局限性不仅制约了处理器架构的发展, 还影响了国内智能计算硬件的发展和深度学习应用的本土化适配。国产深度学习处理器, 如昇腾 910 系列和寒武纪 MLU 系列芯片, 全部采用的是 SIMD 架构。这些芯片的多样性和特殊性要求编程语言不仅要具备高度的可移植性, 还需要能够充分利用这些硬件的特性, 以实现最优的性能表现。SIMD 架构芯片和 SIMT 架构芯片在指令的操作数粒度上存在显著差异, 这带来了以下明显的区别:

SIMD 架构中间表示问题: 从形式化定义的角度, 设 I 为指令集, S 为标量指令集, V 为向量指令集。对于 SIMT 架构, 指令集 I 主要由标量指令集 S 组成, 即

$$I = S \quad (2.1)$$

而对于 SIMD 架构, 指令集 I 主要由向量指令集 V 组成, 即

$$I = V \quad (2.2)$$

这种架构差异导致 SIMD 架构中单指令的操作数粒度较大, 并且对具体的神经网络运算有抽象, 例如卷积计算可以通过专门的卷积指令实现。而 SIMT 架构中

单指令的操作数是标量，通过循环执行乘法和加法来完成卷积操作。

在 Triton 的现有设计中，编译流程涉及三个层次的中间表示：Triton-IR (T)、LLVM-IR (L) 和硬件指令集 (I)。对于 SIMT 架构，编译器 C 将 T 转换为 L ，再转换为 S ，即

$$C(T) = L \rightarrow S \quad (2.3)$$

这一步骤在 GPU 上表现良好，因为 LLVM-IR 和 GPU 指令集都是基于标量语义的。然而，当将这一下降流程应用于 SIMD 架构时，由于 SIMD 架构的指令粒度是基于向量表达的粗粒度指令，因此如果复用针对 GPU 的编译流程的话，就需要先将粗粒度的 Triton-IR 转换为细粒度的 LLVM-IR，再转换为粗粒度的 SIMD 指令。

这种转换过程可以用数学形式表示为：设 f 为从 L 到 V 的映射函数，即

$$f(L) = V \quad (2.4)$$

由于 L 是标量表示而 V 是向量表示， f 是一个多对一的映射，即多个 LLVM-IR 标量指令可能被映射到同一个 SIMD 向量指令上。这种映射会导致以下问题：

- 信息丢失：向量指令的并行性和数据连续性信息在标量表示中丢失，
- 性能下降：由于标量指令无法充分利用 SIMD 架构的并行处理能力，导致性能下降。

这种“粗到细再到粗”的转换过程正是编译器设计中需要避免的经典不可逆转换问题 (*Irreversible Transformation Problem*)，在不同粒度的 IR 之间转换过程中的信息丢失是不可恢复的。这种转换不仅增加了编译的复杂性，还可能导致性能优化的机会被浪费。因此，为了解决这个问题，需要重新设计一套多层次 IR 系统，以保证粗粒度的向量语义在下降到硬件指令的过程中得以保留。

SIMD 架构下的二元算子性能优化问题：SIMD 架构下的二元算子性能优化可归结为两个核心问题：任务调度优化和计算效率优化。前者关注如何将多个计算任务高效映射到硬件执行单元，后者则着重于最大化单个任务的执行效率。

1. 任务调度问题建模

SIMD 与 SIMT 架构在任务调度方式上存在本质区别，这种差异主要体现在硬件抽象模型和编程范式两个层面。从形式化定义的角度，设计算任务为 T ，处理器架构为 P ，任务到硬件的映射函数为 M 。对于 SIMD 架构，其任务调度可形式化为：

$$M_{SIMD}(T) = \{C_1(B_1), C_2(B_2), \dots, C_n(B_n)\} \quad (2.5)$$

其中每个计算核 C_i 处理一个数据块 B_i ，数据块大小 $|B_i|$ 由片上 RAM 空间决定，且数据块之间相互独立。这种调度方式体现了 SIMD 架构的批处理特性，即单个

指令同时作用于多个数据元素。相比之下，SIMT 架构的任务调度可形式化为：

$$M_{SIMT}(T) = \{Th_1(x_1), Th_2(x_2), \dots, Th_m(x_m)\} \quad (2.6)$$

其中每个线程 Th_j 处理一个标量数据 x_j ，线程被组织成线程块：

$$Block_k = \{Th_{k1}, \dots, Th_{kl}\} \quad (2.7)$$

且线程之间可以通过共享内存进行通信。这种调度方式体现了 SIMT 架构的细粒度并行特性。

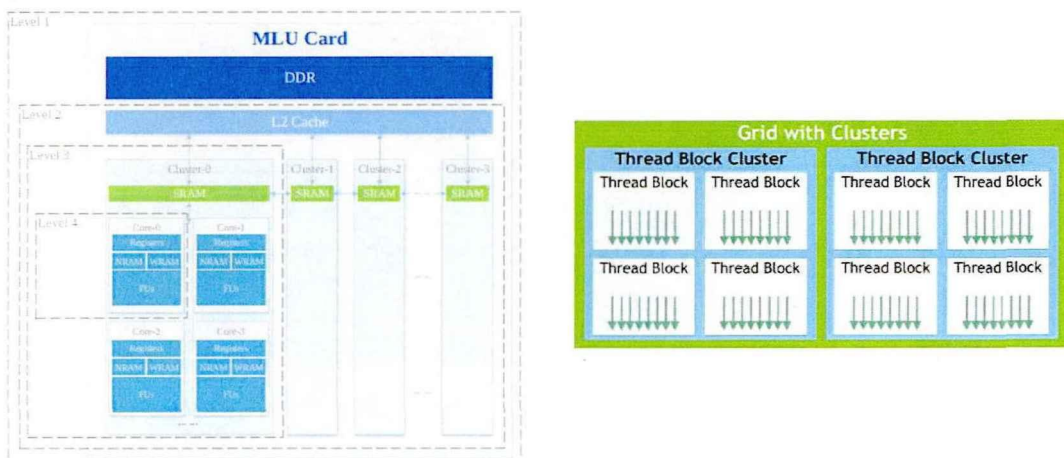


图 2.4 寒武纪 MLU^[51]（左图）和 Nvidia GPU^[52]（右图）的硬件抽象模型

如图 2.4 所示，以寒武纪 MLU 和 Nvidia GPU 为例，两者在编程模型上存在显著差异。寒武纪 MLU 采用计算核（core）作为基本编程单元，每个 core 能够处理一块数据，数据量大小取决于 core 内片上 RAM 空间的大小，并通过多核扩展出 cluster。这种设计使得 SIMD 架构更适合处理规则的大规模数据并行任务。而 Nvidia GPU 采用线程（Thread）作为基本编程单元，每个 thread 处理一个标量数据，基于多线程扩展出线程块（Thread Block）。这种设计使得 SIMT 架构在处理不规则并行任务时更具灵活性。

这种架构差异带来了一个重要挑战：如何将 Triton 的计算任务以恰当的方式映射到 SIMD 架构的硬件上。设 f 为从 M_{SIMT} 到 M_{SIMD} 的转换函数，目标为找到一个函数 f ，使得：

$$M_{SIMD}(T) = f(M_{SIMT}(T)) \quad (2.8)$$

且满足计算正确性约束，即转换后的 SIMD 映射结果与原始 SIMT 映射结果在语义上等价：

$$\forall x \in T, M_{SIMD}(x) = M_{SIMT}(x) \quad (2.9)$$

在具体实现中，还需要考虑硬件资源的约束条件。数据块拆分需要满足片上 RAM 大小的约束：

$$|B_i| \leq RAM_{on-chip} \quad (2.10)$$

若 $|B_i| > RAM_{on-chip}$, 则需要将 B_i 拆分为多个子块 $\{B_{i1}, B_{i2}, \dots, B_{ik}\}$, 满足:

$$\bigcup_{j=1}^k B_{ij} = B_i \quad \text{且} \quad \forall j \neq l, B_{ij} \cap B_{il} = \emptyset \quad (2.11)$$

最终目标是最大化硬件利用率, 尽可能充分利用 SIMD 架构的计算核:

$$\max \left(\frac{|\text{active cores}|}{|\text{total cores}|} \right) \quad (2.12)$$

2. 计算效率优化问题建模

在计算效率优化方面, 本工作需要同时考虑访存时间和计算时间的优化。访存时间可表示为:

$$T_{mem} = \sum_{i=1}^n \left(\frac{|B_i|}{BW} + L \right) \quad (2.13)$$

其中 BW 为内存带宽, L 为访存延迟。计算时间则取决于最慢的计算核:

$$T_{comp} = \max_{i \in [1, n]} \left(\frac{|B_i|}{IPC \cdot f} \right) \quad (2.14)$$

其中 IPC 为每个周期执行的指令数, f 为处理器频率。

3. 综合优化目标

将任务调度和计算效率优化两个问题结合起来, 可以得到综合优化目标:

$$\text{minimize } \alpha \cdot T_{total} + \beta \cdot (1 - U) \quad (2.15)$$

其中:

- $T_{total} = T_{mem} + T_{comp}$ 为总执行时间。
- $U = \frac{|\text{active cores}|}{|\text{total cores}|}$ 为硬件利用率。
- α 和 β 为权重系数。

约束条件主要包括以下三个方面:

首先, 数据块大小需要满足片上 RAM 的容量限制:

$$|B_i| \leq RAM_{on-chip}, \quad \forall i \in [1, n] \quad (2.16)$$

其次, 数据完整性需要得到保证, 即所有数据块的总和应等于完整数据集, 且各数据块之间互不重叠:

$$\bigcup_{i=1}^n B_i = T \quad \text{且} \quad B_i \cap B_j = \emptyset, \quad \forall i \neq j \quad (2.17)$$

最后, 计算结果的正确性必须得到保证, 即 SIMD 架构下的计算结果应与原始 SIMT 架构下的结果完全一致:

$$\forall x \in T, M_{SIMD}(x) = M_{SIMT}(x) \quad (2.18)$$

综上所述,为了充分发挥国产 SIMD 架构处理器的性能潜力,扩展 Triton 对 SIMD 架构处理器后端的支持,设计一套全新的多层次 IR 系统显得尤为必要。这套系统需要能够:

- **设计任务调度方式:** 针对 Triton 语言特性,将 Triton 对任务的抽象方式映射到 SIMD 架构上,并确保硬件利用率。
- **提供中间抽象层:** 在 Triton-IR 和 SIMD 指令之间引入一个或多个中间抽象层,以适配不同硬件的指令集粒度。
- **优化数据流和控制流:** 通过编译期的优化手段,尽可能优化 SIMD 架构下的算子性能。

2.3 面向 SIMD 架构智能处理器的 Triton 语言编译器设计

为了解决 Triton 适配 SIMD 架构后端的问题,并扩展 Triton 对 SIMD 架构处理器后端的支持,本工作设计了一套全新的 Triton 语言编译流程。该流程的中间表示借鉴了 IREE^[48] 的设计理念,采用 Linalg 方言作为连接高层次表示与 SIMD 硬件指令集的中间抽象层。基于此,本工作提出了一种从 Triton-IR 到 Linalg-IR,再到硬件指令集级 IR 的全新编译架构。

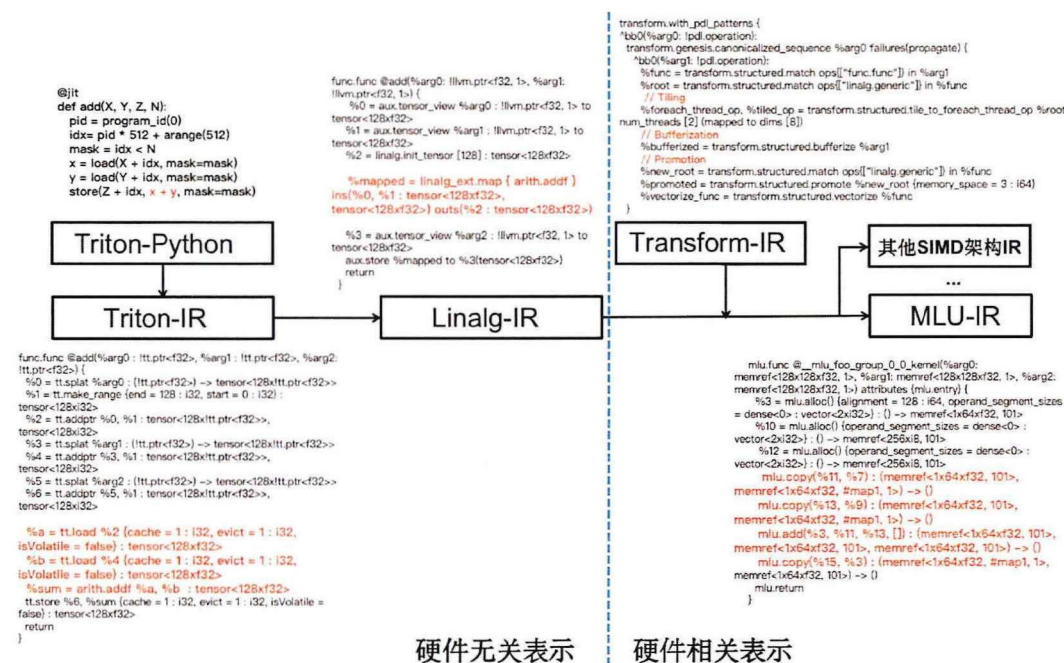


图 2.5 Triton 面向 SIMD 架构智能处理器的下降流程

图 2.5 以 Triton 语言中的 $x + y$ 运算为例,展示了这一下降过程。本章复用 Triton 官方提供的抽象语法树 (AST) 解析流程,将 Triton-Python 转换为 Triton-IR。从 Triton-IR 开始的下降过程可分为硬件无关表示部分和硬件相关表示部分。

对于硬件无关表示部分,主要关注的是如何保留高层次的基于张量的计算

语义, 无需考虑片上空间的拆分、数据搬运等细节。此部分主要涉及从 Triton-IR 到 Linalg-IR 层面的转换。通过这一转换, 确保了高层次的计算语义在下降过程中得以保留, 为后续的硬件相关表示部分奠定了基础。

对于硬件相关表示部分, 主要关注的是保留硬件指令集的语义, 需设计转换流程以将高层次的计算表达映射到硬件指令集。此部分主要涉及从 Linalg-IR 通过 Transform-IR 描述的转换过程, 转换为硬件指令集层面的 IR。这一过程确保了高层次的计算语义能够有效地映射到硬件指令集, 从而实现对 SIMD 架构处理器后端的支持。图 2.5 中以寒武纪 MLU 的硬件指令集 IR, MLU-IR, 为例展现了这一过程, 同时这一过程亦可扩展到其他 SIMD 架构的指令集 IR。

2.3.1 硬件无关表示设计

本节主要讨论了将 Triton-IR 转换为 Linalg-IR 的过程, 分别讨论了 Triton-IR 的访存语义和计算语义的操作在转换过程中的实现细节。

1. 访存语义的优化

首先, 针对访存语义的操作, 本节提出了一种通过跟踪并分析 Triton-IR 中的指针操作来推断访存数据连续性信息的方法, 从而实现访存性能的优化。Triton-IR 作为一种高度抽象的中间表示 (IR), 其设计理念在于不依赖于输入参数的具体形状, 而是统一采用指针来描述输入数据的基地址。然而, 这种设计带来的问题是指针表示的自由度极高, 可以指向一个标量, 也可以指向一块向量, 因此需要对指针指向的数据进行深入分析。

具体而言, 需要对指针的具体操作进行跟踪, 通过分析 IR 中是如何将指针信息转换为向量数据信息的过程, 从而获取指针所指向的数据信息。分析的核心思想基于一个前提, 即 Triton-IR 中对访存行为的描述都是基地址 + 偏移的方式, 由此可以通过分析出 IR 中是如何对基地址处理的, 以及如何对偏移进行处理的, 从而得出数据块的连续性信息。为了系统化地描述这一分析过程, 本章节引入以下定义: 设 P 为指针变量, B 为基地址, O 为偏移量, T 为数据类型, V 为向量数据, n 为向量长度。

基于上述定义, 本节提出了一种针对 Triton-IR 指针操作的连续性分析算法, 其具体实现如算法 2.1 所示。

上述算法中的关键操作可解释如下:

AnalyzeBase 操作作为连续性分析的基础步骤, 主要负责对基地址操作序列进行静态分析, 通过解析 Triton-IR 中的基地址操作语义, 生成对应的基地址向量表示。该操作通过追踪指针变量的定义和使用链, 提取并重构基地址的计算过程, 从而建立完整的基地址向量表示。

AnalyzeOffset 操作则专注于偏移向量的生成过程分析, 通过解析 Triton-IR

算法 2.1 Triton-IR 指针操作的连续性分析

```
Input: Triton-IR operations
Output: Continuity information of the data
1 Step 1: Analyze base address;
2  $B_{vec} \leftarrow \text{AnalyzeBase}(B)$  // 分析基地址操作
3 ;
4 Step 2: Analyze offset;
5  $O_{vec} \leftarrow \text{AnalyzeOffset}(\text{start}, \text{end})$  // 分析偏移量操作
6 ;
7 Step 3: Analyze address calculation;
8  $V_{addr} \leftarrow \text{AnalyzeAddress}(B_{vec}, O_{vec})$  // 分析地址计算操作
9 ;
10 Step 4: Check continuity;
11 if  $C(V_{addr})$  then
12   return [BaseAddress, Offset, Size, Stride] // 地址连续数据信息
13 ;
14 else
15   return [BaseAddress, 0, 1, 1] // 地址离散数据信息
16 ;
17 end
```

中显式或隐式包含的数据块起始点 (start) 和终止点 (end) 信息, 结合循环迭代变量和索引计算表达式, 推导出精确的偏移量表示。该操作特别关注偏移量的生成模式, 包括线性增长、固定步长等特征, 为后续连续性判断提供关键依据。

AnalyzeAddress 操作作为连续性分析的核心环节, 负责对地址计算过程进行语义分析。该操作将 *AnalyzeBase* 和 *AnalyzeOffset* 的分析结果进行整合, 通过模拟基地址向量与偏移向量的计算过程, 生成最终的地址向量表示。在此过程中, 该操作不仅验证地址计算的正确性, 还记录地址生成模式的关键特征, 为连续性判断提供必要的信息支持。

连续性分析的核心在于验证地址向量的等距特性。通过分析偏移生成操作可以确定偏移向量的步长 *stride*。结合基地址分析结果, 可以推导出最终地址向量 V_{addr} 的连续性。连续性判断函数 $C(V_{addr})$ 通过验证相邻地址的差值是否恒等于 *stride* 来判断数据的连续性。

该连续性分析算法具有通用性强、可扩展性好的特点, 能够精确推导内存访问模式并自动识别连续访问模式。该方法不仅适用于各种类型的基地址、偏移和地址计算操作, 还支持添加新的操作类型分析, 同时能够处理复杂的访问模式分析。

以图 2.6 中的 Triton-IR 为例, 展示了一段典型的指针操作 IR, 将指针参数转换为向量数据。为了能够准确分析出向量数据的连续性, 必须通过详细分析这段 IR 中每个操作的具体行为, 追溯这块向量数据的生成过程。从图 2.6 中可以看出, 从 *%tensor_result* 向前追溯到函数入口参数, 可以发现是由 *tt.splat* 指令将函数入口参数的基地址扩展为向量形式; 随后, 利用 *tt.make_range* 指令生成一个

偏移向量；最后，通过 `tt.addptr` 指令将基地址与偏移向量相加，从而得到向量数据。这一过程如图 2.7 所示，明确展示了此过程会生成一块连续的向量数据。

```

1 // Triton-IR
2 func.func @add(%arg0 : !tt.ptr<f32>, %arg1 : !tt.ptr<f32>, %arg2: !tt.ptr<f32>)
3   // Splat base address
4   %0 = tt.splat %arg0 : (!tt.ptr<f32>) -> tensor<128x!tt.ptr<f32>>
5   // Get offset vector
6   %offset = tt.make_range {end = 128 : i32, start = 0 : i32} : tensor<128xi32>
7   // Base address + offset
8   %tensor_result = tt.addptr %0, %offset : tensor<128x!tt.ptr<f32>>, tensor<128xi32>

1 // Linalg-IR
2 func.func @add(%arg0: !llvm.ptr<f32, 1>, %arg1: !llvm.ptr<f32, 1>)
3   %0 = aux.tensor_view %arg0 : !llvm.ptr<f32, 1> to tensor<128xf32>

```

图 2.6 Triton-IR 和 Linalg-IR 对于指针操作的表达方式对比

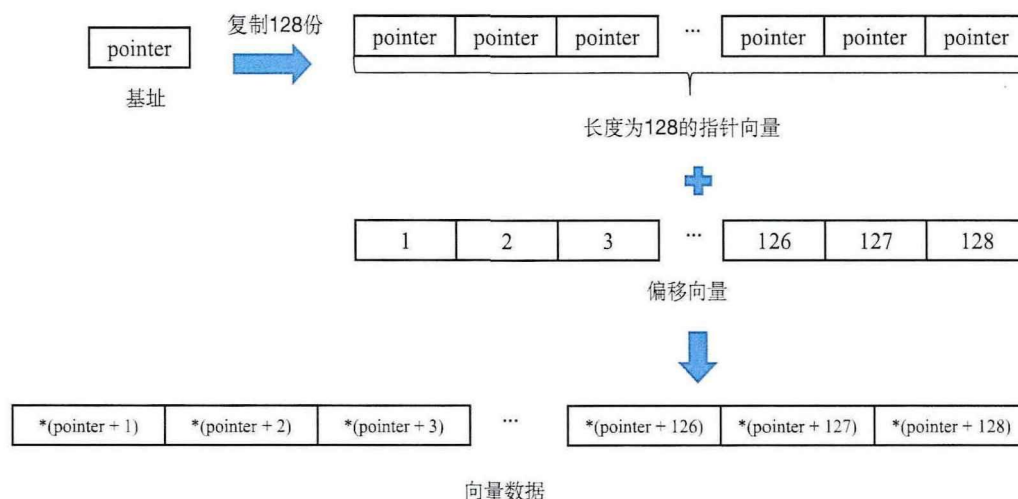


图 2.7 Triton-IR 中对输入指针参数的处理

通过上述连续性分析方法，Linalg-IR 这一层次能够直接将输入指针转换为连续的数据向量表示。这一转换过程在图 2.6 中以 Linalg-IR 形式进行了详细展示。具体而言，转换过程首先涉及将 Triton-IR 中的函数参数类型从 `tt.ptr` 转换为 `llvm.ptr`，即 LLVM 方言中的指针类型，从而继承 `tt.ptr` 的指针语义。随后，为了将指针向量转换为向量语义，本工作进一步扩展了 MLIR 的 *Auxiliary* 方言，定义了 `tensor_view` 操作。该操作专门用于承接指针输入并将其转换为连续向量数据的输出操作，从而实现了从指针到向量数据的平滑过渡。

通过这种系统性地跟踪向量数据的生成过程的方法，针对生成过程进行深入分析，可以准确获取向量数据的连续性信息，从而可以通过连续访存来显著提升性能。这种方法不仅提高了访存效率，还为后续的优化和硬件映射提供了坚实

的基础。

2. 计算语义的封装

本章对计算语义相关表示的处理进行了进一步的优化，旨在提升编译器在不同硬件平台上的高效代码生成能力。本章基于 Linalg-IR，通过将相同计算语义的操作使用 Linalg 方言的扩展进行统一封装，不仅提供了简洁的表达方式，还使得这些操作能够复用 Linalg 方言中对操作进一步优化的能力，如循环拆分等优化。

<pre> 1 // Triton-IR 2 tt.load %input1 3 tt.load %input2 4 %result = arith.addf %input1, %input2 5 tt.store %result </pre>	<pre> 1 // Linalg-IR 2 %result = linalg_ext.map { arith.addf } %input1, %input2 </pre>
--	---

图 2.8 Triton-IR 和 Linalg-IR 对于加法计算的表达方式对比

具体而言，本章以向量加法操作为例，详细阐述了这一优化过程。如图 2.8 所示，在 Triton-IR 中，基于向量语义进行描述，首先通过 *tt.load* 指令加载参数，然后利用 *arith.addf* 指令执行计算操作，最后通过 *tt.store* 指令将计算结果存储回内存。这一过程详细描述了具体硬件操作的执行流程，体现了对底层硬件操作的直接映射。然而，在 Linalg-IR 这一层次，作为算法的高级抽象表达，不再需要考虑具体的 load/store 行为，而是直接通过 *linalg.map* 操作来描述上述计算过程。这一转变不仅简化了 IR 的结构，还为编译器在不同硬件平台上的高效代码生成提供了坚实基础。鉴于当前社区中 Linalg 方言的高度可扩展性，本章在 Linalg 方言的基础上扩展了点对点类算子（Pointwise）的定义。通过使用 *linalg.map* 操作，并在其中明确具体的计算逻辑，例如 *arith.addf* 操作，为向量类算子提供了一种统一的表达方式。

本章节通过统一封装和扩展 Linalg 方言，不仅简化了 IR 的结构，还显著提升了编译器在不同硬件平台上生成高效代码的能力。这一设计策略的核心在于对计算语义的高效表达和优化，为编译器在多硬件平台上的高效代码生成提供了坚实的基础。具体而言，本章充分利用了 MLIR 丰富的方言体系，包括指针语义（LLVM）、张量语义（Linalg）、循环语义（SCF）、控制流语义（Func）、数值语义（Math）以及辅助语义（Auxiliary）等，这些方言能够很好地承接各种硬件无关的语义表达。通过这种系统化的方法，本章不仅为硬件无关的算法表达提供了强大的支持，还为进一步的硬件相关优化奠定了基础。

2.3.2 硬件相关表示设计

MLU-IR 的设计是针对寒武纪 MLU 芯片进行的封装。对于 MLU 这一 SIMD 架构的处理器而言，MLU-IR 主要描述的是 SIMD 指令级别的操作，如 *mlu.copy*，*mlu.add* 等，操作数来自硬件的片上空间，直接映射到硬件的指令集，这要求开发者对 MLU 的内存层次结构和执行单元有深入的理解。MLU-IR 的设计重点在于高效地利用 MLU 的硬件特性，包括片上存储、SIMD 执行单元和并行处理能力。例如，*mlu.alloc* 操作用于分配 MLU 的片上内存，而 *mlu.copy* 则用于在片上空间和全局内存之间做数据搬运。这些操作的实现细节被抽象化，使得开发者可以专注于算法的实现，而不必处理底层的硬件编程复杂性。



图 2.9 Linalg-IR 下降为 MLU-IR 的下降流程

为了将 Linalg-IR 有效地下降为 MLU-IR，需要在 Linalg 层面进行一系列精细的变换，将不包含数据依赖信息的张量描述转换为描述片上数据依赖的 *memref* 描述。如图 2.5 所示，这一转换流程包括关键步骤：拆分（tile）、缓冲（bufferize）和提升（promote）。这些步骤对于优化 SIMD 架构上的执行至关重要。

拆分 (Tile)：由于片上空间有限，张量数据需要进行拆分，以便在片上放下。拆分的本质是循环处理一整块数据，每次循环处理的数据量大小需要用拆分后的数据块大小（tile size）来描述。拆分后数据块的大小直接影响性能，因为它决定了计算的并行度和内存访问模式。这一过程通过定义每个张量维度的拆分块大小开始，并将原始计算循环转换为嵌套循环，其中外层循环迭代拆分块，内层循环在每个拆分块内迭代。特殊注意边界条件的处理，确保拆分块大小可能不完全除尽维度大小时的正确性。在图 2.9 所示的 *Tile* 过程中，初始的两个 *linalg.conv_2d* 操作符，分别称为 *conv_2d_1* 和 *conv_2d_2*，沿着维度 *H* 和 *W* 进行融合和拆分，对应的拆分块大小分别为 *OH* 和 *OW*。随后，第三个 *linalg.conv_2d* 操作符，*conv_2d_3*，沿着维度 *C* 进行融合和拆分，拆分块大小为 *OC*。在这个过程中使用了 mlir 中的 *SCF* 方言构造 for 循环。

缓冲 (Bufferize)：缓冲化是将基于张量的操作转换为基于缓冲区（buffer）的操作的过程。这一转换对于将高级张量操作转换为适合在硬件上执行的形式至关重要，尤其是对于那些通常操作缓冲区（连续内存区域）而非抽象张量类型的

硬件。缓冲化的关键目标包括降低张量抽象、引入显式内存分配和释放、使用原地计算 (inplace) 以及为代码生成做准备。它通过将张量操作转换为等效的缓冲区操作、引入缓冲区的分配和释放, 并使用重写模式系统地替换张量操作为缓冲区操作。如图 2.9 所示, 在 *Bufferize* 过程中, 每个操作数的内存空间是基于调度表示使用 *memref* 类型定义的。例如, 调度表示指定 *conv_2d_1* 和 *conv_2d_2* 操作符在 L0 缓存上执行, 因此它们的操作数被分配在 L0 缓存中。这种明确的内存分配策略确保了数据在执行过程中的高效访问, 减少了内存访问延迟, 从而提高了整体计算性能。

提升 (Promote): 提升是 Linalg 方言中设计的一种机制, 旨在通过将数据从较低级别的内存层次结构 (例如全局内存) 提升到较高级别 (例如共享或局部内存) 来优化内存访问模式。这对于提高访问全局内存相对较慢的系统的性能非常有用。提升的关键目标包括改善内存局部性、启用数据重用、优化循环嵌套。提升转换通过将 linalg 操作的张量操作数提升到更快的内存中的缓冲区来工作, 涉及在局部内存中创建临时缓冲区, 将必要的数据复制到此缓冲区, 并修改计算以使用缓冲区而不是原始张量。如图 2.9 所示, 在 *Promote* 过程中, 插入的 *linalg.copy* 操作用于将数据在不同内存空间之间搬运。

为了实现这些复杂的变换, MLIR 中的变换方言 (Transform Dialect) 提供了一个结构化和可扩展的方式来描述和应用变换。变换方言的设计目标是促进 MLIR 框架内的复杂代码变换和优化, 使得编译器优化更加高效和易于维护。变换方言允许模块化变换不同操作的组合, 且可以扩展新的操作用于性能优化。除此之外, 它以声明式方法将变换行为进行抽象, 允许用户简单地调用各种变换, 在此基础上可以方便地实现自动调优 (auto-tuning) 的机制。通过使用变换方言, 可以将拆分、缓冲和提升等操作抽象化, 从而在 Linalg-IR 和 MLU-IR 之间构建一个高效且可维护的转换路径。这种方法不仅提高了代码的可读性和可维护性, 而且为未来的优化和硬件适配提供了强大的支持。

通过这些步骤, 可以将 Linalg-IR 中的高级抽象有效地映射到 MLU-IR, 为 SIMD 架构的芯片生成优化的代码。这种多层次的 IR 转换策略不仅提高了代码的可移植性和可维护性, 而且为深度学习编译器技术的进步提供了坚实的基础。通过引入 *tensor_view* 操作和 *linalg_ext* 等方言, 本工作成功地将 Triton-IR 中的高级抽象转换为适合 SIMD 架构的低级实现, 同时保持了代码的清晰性和优化潜力。这种转换策略为深度学习模型在多样化硬件平台上的高效执行提供了强有力的支持。

2.4 面向 SIMD 架构智能处理器的 Triton 程序编译性能优化

本章节主要围绕 SIMD 架构下的二元算子性能优化展开，重点解决任务调度优化和计算效率优化这两个核心问题：前者关注如何将多个计算任务高效映射到硬件执行单元，后者则着重于最大化单个任务的执行效率。首先，针对如何高效地将 Triton 任务映射到 SIMD 架构的执行单元上这一问题，本工作提出了一种新的任务映射策略，旨在确保 Triton 算子能够充分利用 SIMD 架构的并行处理能力，从而实现高效的代码执行。通过这一策略，Triton 程序能够更好地适应 SIMD 架构的特性，显著提升其在智能处理器上的执行效率。其次，针对如何优化单个任务在芯片上的执行效率这一问题，本工作提出了一种通用的二元算子性能优化方法。该方法不仅能够针对多种通用硬件平台进行优化，还能在不同硬件环境下实现性能的显著提升。通过这一优化方法，Triton 算子在编译时的性能得到了有效提升，进一步增强了其在多样化硬件平台上的执行效率。

2.4.1 任务调度优化

本节提出了一种面向 SIMD 架构的任务调度优化方案，通过重构 Triton 语言的核心调度参数，实现了针对 SIMD 架构的计算效率大幅度的提升。该方案的核心在于理解 SIMT 与 SIMD 架构在计算任务抽象方面的差异，并针对性地进行任务调度参数的重映射。

1. 架构差异与任务抽象

现代并行计算架构主要采用 SIMT（单指令多线程）和 SIMD（单指令多数据）两种并行计算模型。以 NVIDIA GPU 为代表的 SIMT 架构采用细粒度任务调度，每个线程处理单个数据元素；而以寒武纪 MLU 芯片为代表的 SIMD 架构则采用粗粒度任务调度，每个计算单元并行处理多个数据元素。在 MLU 架构中，

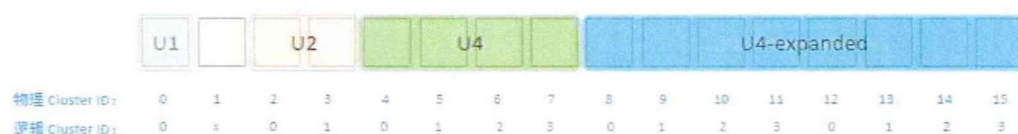


图 2.10 MLU UnionN 类型任务的映射关系^[51]

计算任务被抽象为一个三维网格 $[\text{taskDimX}, \text{taskDimY}, \text{taskDimZ}]$ ，每个任务对应一个物理计算核（core）。这种任务类型被称为 **BLOCK** 任务，是任务映射的最小颗粒度。进一步地，任务划分的颗粒度可扩展至 cluster 级别，形成 Union 任务类型。如图 2.10 所示，在物理上拥有 16 个 cluster 的情况下，通过逻辑 cluster ID 可以对任务使用的计算单元进行编码。例如，在 Union2 任务中，逻辑 cluster ID 为 0 和 1 分别描述当前任务使用的 2 个 cluster 计算单元。

相比之下，如图 2.11 所示，GPU 通过 $[\text{GridId}, \text{BlockId}, \text{ThreadId}]$ 三级层次结

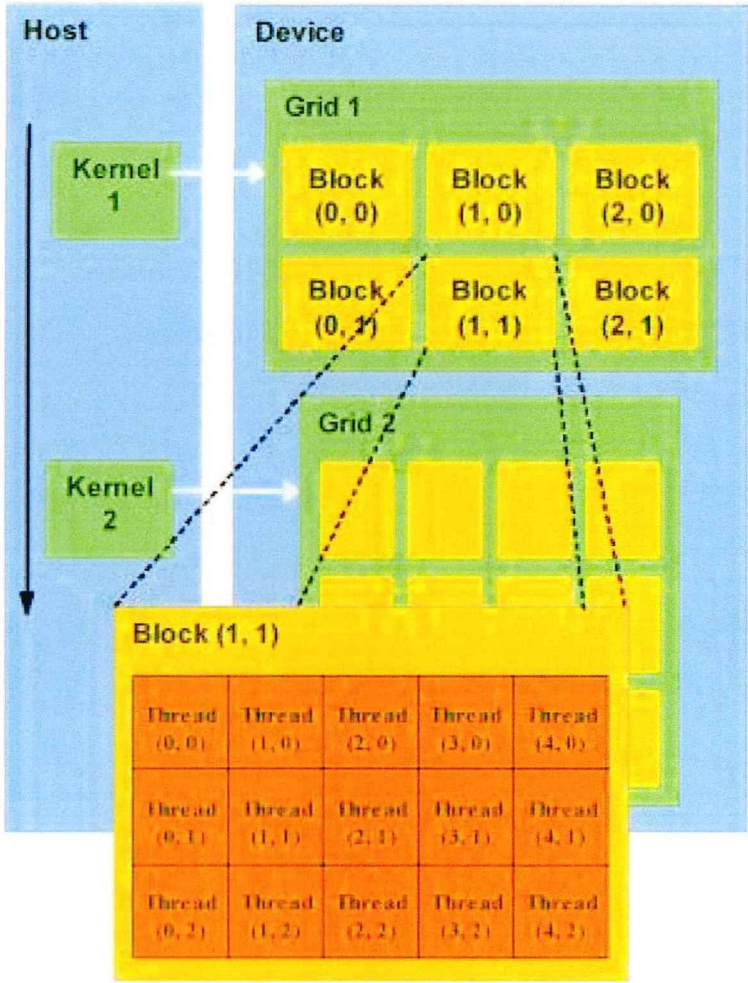


图 2.11 GPU 中的任务调度模型^[52]

构来描述任务的映射方式，每个线程负责执行一个标量操作。这种细粒度的任务划分方式虽然适用于 SIMT 架构，但无法充分发挥 SIMD 架构的并行计算优势。

2. 任务调度参数重映射

为适应 SIMD 架构的特性，本工作对 Triton 语言的调度参数进行了创新性重映射。Triton 语言采用 *grid* 和 *num_warps* 两个关键参数来描述任务的映射方式和执行粒度。在传统 GPU 实现中，*grid* 参数与线程块（Thread Block）绑定，表示程序使用 *grid* 个线程块，每个线程块包含 *num_warps* 个线程束，每个线程束中包含 32 个线程。

针对 SIMD 架构，本章节提出以下重映射方案：

- 将 *grid* 参数映射到 MLU 的三维任务空间 [*taskDimX*, *taskDimY*, *taskDimZ*]，用于描述并行任务的划分维度；
- 将 *num_warps* 参数重新定义为任务类型标识，用于指定每个任务占用的计算单元数量，比如对于 *BLOCK* 类型任务，设置 *num_warps* = 1 表示每个任务占用一个计算单元。

算法 2.2 Triton Task Scheduling Parameter Remapping Algorithm

Input : Original Triton-GPU parameters: *grid*, *num_warps*
Output: Adapted scheduling parameters for SIMD architecture: *taskDimX*, *taskDimY*, *taskDimZ*, *taskType*

```

1 if arch == SIMD then
    // Get available hardware resources for the target architecture
2    num_cores ← getAvailableHardwareResources(arch);
    // Convert SIMT grid to SIMD task dimensions
3    [taskDimX, taskDimY, taskDimZ] ←
        ConvertGridToTaskDim(grid.x, grid.y, grid.z, num_cores);
    // Determine task type based on available cores
4    if num_cores % 4 == 0 then
        // 4 cores per cluster
5        taskType ← UnionN;
6        N ← num_cores / 4;
7    else
8        taskType ← BLOCK;
9    end
    // Validate task partitioning
10   if taskDimX × taskDimY × taskDimZ > num_cores then
11       taskDimX ← num_cores / (taskDimY × taskDimZ);
12   end
13 else
    // Keep original parameters unchanged
14   return Original parameters;
15 end
16 return [taskDimX, taskDimY, taskDimZ], taskType;

```

算法 2.2 展示了 Triton 任务调度参数重映射的核心流程。首先，算法通过 *getAvailableHardwareResources* 函数获取目标架构的可用硬件资源信息，随后调用 *ConvertGridToTaskDim* 函数将 SIMT 架构下的 *grid* 参数转换为适合 SIMD 架构的三维任务空间 [*taskDimX*, *taskDimY*, *taskDimZ*]。在确定任务类型时，算法根据可用计算核心数量进行判断：当核心数量可被 4 整除时，采用 UnionN 类型任务，否则采用 BLOCK 类型任务。为确保任务划分的合理性，算法还包含任务分区验证机制，当总任务数超过可用计算核心数时，自动调整 *taskDimX* 维度以匹配硬件资源限制。

3. 性能优化案例分析

图 2.12 展示了不同架构下任务划分策略的优化效果及其对计算性能的影响。以处理 6×128 大小的数据块为例：

- GPU 架构采用细粒度任务划分策略，将数据块分解为 6 个独立任务，每个任务分配 4 个线程束（共计 128 个线程），每个线程负责处理单个标量数据
- MLU 架构采用粗粒度任务划分策略，同样将数据划分为 6 个任务，但每个任务仅分配 1 个计算单元，每个计算单元通过 SIMD 指令并行处理 128 个数据元素

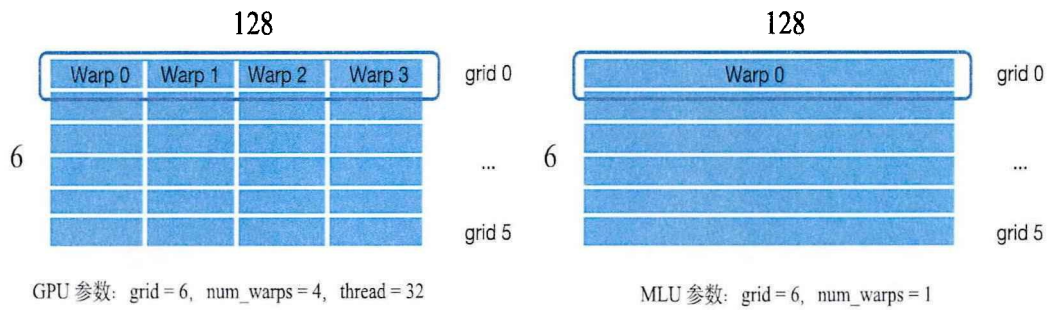


图 2.12 对于一块 6 x 128 大小的数据块，不同架构下 Triton 算子的任务划分方式

- 当计算单元的片上存储容量不足以同时处理 128 个数据时，要求用户在编写 Triton 程序时对数据块进行拆分，在计算单元内部采用循环处理机制。这种粗粒度划分策略不仅显著提升了数据并行度，还通过减少任务调度开销，使 SIMD 架构的计算资源得到最大化利用。

总结而言，本节通过对 Triton 语言任务调度参数的重新映射，成功地将 Triton 程序高效映射到 SIMD 架构上，从而提升了任务调度的效率和并行性。这种优化方案不仅适用于寒武纪 MLU 芯片，也可推广到其他 SIMD 架构的处理器上。

2.4.2 二元算子性能优化

Triton 算子因其用户友好性和高抽象层次而广受欢迎，硬件相关参数在语言层面暴露给用户的很少，因此开发过程非常便捷。然而，从性能优化的角度来看，这种设计却带来了挑战。由于硬件细节的隐藏，编译器在优化过程中缺乏直接的硬件信息，导致性能优化手段的实施变得复杂。

在多维数据的二元运算中，尤其是矩阵乘法和卷积运算中，数据交换和加载的频繁性往往成为性能瓶颈。针对减少数据交换和加载时间这一优化目标，本章提出了针对 Triton 语言编译优化的数据驻留技术和流水线处理技术。这两种技术的核心思想是通过软件层面的优化来弥补硬件信息缺失带来的性能损失，其具体实现如算法 2.3 所示。

该算法采用两阶段优化策略：第一阶段通过数据驻留技术减少数据加载频次，第二阶段通过流水线处理技术实现计算资源的充分利用。在数据驻留阶段，算法首先识别二元算子操作，然后根据数据量大小选择驻留数据块，并生成相应的数据加载策略。在流水线优化阶段，算法将计算过程划分为多个阶段，合理分配存储资源，并生成最优的流水线调度方案。

数据驻留技术通过在计算过程中驻留一方数据，减少其加载频次，从而降低 I/O 量。具体来说，在矩阵乘法运算中，可以选择驻留左乘矩阵 A 或右乘矩阵 B。以驻留矩阵 A 为例，矩阵 A 的数据块在计算过程中只加载一次，而矩阵 B 的数据块则按照一定的频次循环加载。通过这种方式，矩阵 A 的数据块可以重复使

算法 2.3 Data Residency and Pipeline Optimization for Binary Operators

```

Input : Matrix A, Matrix B
Output: Optimized computation result

// Data Residency Optimization
1 if size(A) < size(B) then
2   |   resident_matrix ← A;
3   |   loop_matrix ← B;
4 else
5   |   resident_matrix ← B;
6   |   loop_matrix ← A;
7 end

// Initialize pipeline buffers
8 buffer1 ← allocate_memory();
9 buffer2 ← allocate_memory();
// Pipeline Processing with Loop Unrolling
10 for i ← 1 to num_blocks step 2 do
11   |   // Load and compute in parallel
12   |   load(buffer1, loop_matrix[i]);
13   |   compute_with_resident(resident_matrix, buffer1);
14   |   if i + 1 ≤ num_blocks then
15   |   |   load(buffer2, loop_matrix[i + 1]);
16   |   |   store(buffer1);
17   |   |   compute_with_resident(resident_matrix, buffer2);
18   |   |   store(buffer2);
19   |   end
20 end

20 return Final computation result;

```

用多次，从而减少了矩阵 A 的加载次数。

如图 2.13 所示，以卷积计算为例，可以选择一次加载权值（图中用 W_t 表示），循环加载神经元（图中用 Act 表示），或者选择一次加载神经元，循环加载权值。这种选择的依据是通过比较神经元和权值的数据量大小进行比较，选择数据量较少的一方进行循环加载，而数据量较多的一方则驻留于内存中，从而得出一种数据搬运量最少的加载策略。通过这种方式，可以确定哪种加载策略能够最大限度地减少数据搬运的次数，从而优化数据加载过程，显著提升执行效率。

流水线处理技术通过将数据访存和计算处理并行化，实现了计算资源的充分利用。在多核计算装置中，数据访存和计算处理往往是串行进行的，这导致了计算资源的闲置和效率的降低。特别是在大规模矩阵乘法运算中，数据访存和计算处理的时间开销较大，进一步影响了整体计算效率。流水线处理技术通过在计算簇层级和处理核层级，分别配置至少两个存储区，用于支持在其中一个存储区与外部存储电路进行数据访存的同时，在另一个存储区与处理核进行数据访存。通过这种方式，数据访存和计算处理可以同时进行，从而提高了计算效率。这一技术的创新点主要体现在并行化数据访存和计算处理以及优化存储区配置两个方面。首先，通过配置多个存储区，实现了数据访存和计算处理的并行化，显著

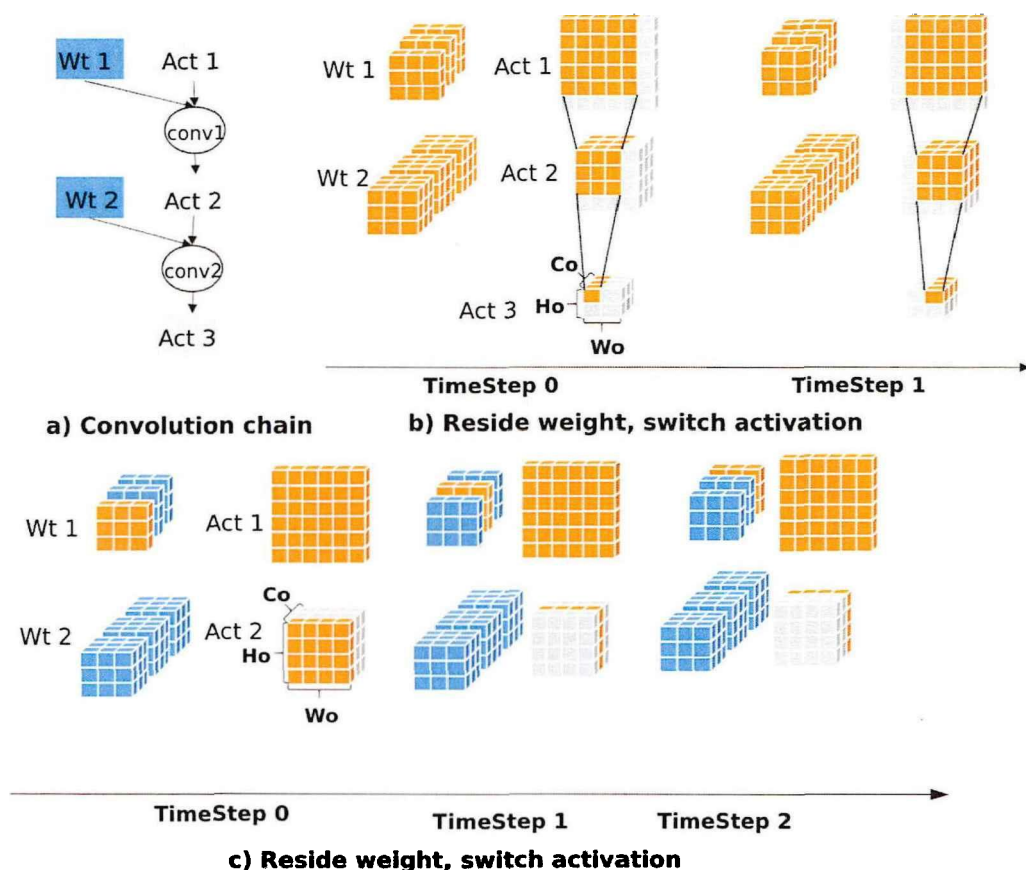


图 2.13 数据驻留技术在卷积计算中的优化方法

提高了计算效率。其次，在处理核层级，分别配置第一存储电路和第二存储电路，用于存储不同的数据块，进一步优化了数据访存和计算处理的流程。这些创新点使得流水线处理技术在实际应用中能够显著提升计算效率，特别是在大规模矩阵乘法运算中表现尤为突出。

如图 2.14 所示，二元算子的典型计算过程通常可以细分为三个主要阶段：数据读取（load，简称为 L）、计算处理（compute，简称为 C）以及结果存储（store，简称为 S）。图中的左图和右图分别展示了未采用流水线处理和采用流水线处理的计算流程对比。在处理两个拆分块的情况下，未采用流水线处理的方式中，数据访存和计算处理是串行执行的，虽然无需配置多个存储区，但是需要更多的时间片来完成整个计算过程。相比之下，采用流水线处理的方式中，数据访存和计算处理可以并行执行，这要求分配两块存储区。具体而言，在第一块存储区中进行第一个拆分块的计算处理时，可以在第二块存储区中同时执行第二个拆分块的数据读取过程。这种并行处理机制显著提高了计算效率，减少了整体计算时间。

数据驻留技术和流水线处理技术的结合，进一步提升了二元算子的性能优化效果。通过数据驻留技术减少数据加载频次，结合流水线处理技术实现数据访

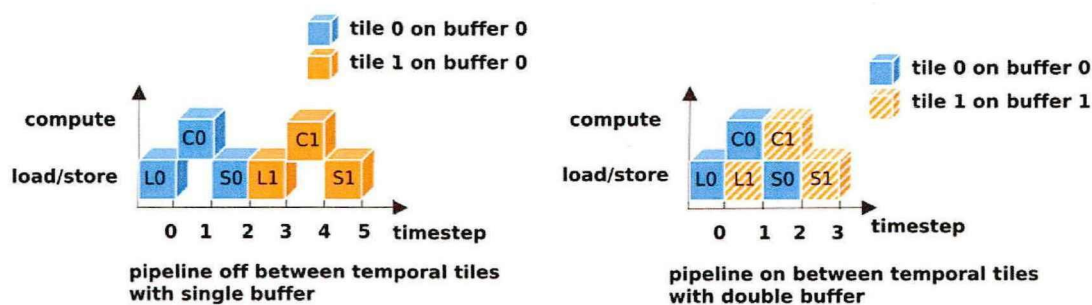


图 2.14 流水线技术在卷积计算中的优化方法

存和计算处理的并行化，两者相辅相成，共同优化了二元算子的计算流程。这种结合不仅减少了数据交换和加载的次数，还充分利用了计算资源，显著提高了计算效率。在实际应用中，这种结合能够有效应对大规模矩阵乘法运算中的性能瓶颈，为高性能计算提供了有力支持。

2.5 实验设计与评估

2.5.1 实验目标

本研究采用系统性实验设计，从编译性能优化、编程效率评估和跨平台性能表现三个维度，全面评估 Triton 语言在 SIMD 架构上的应用效果，为深度学习编译器优化提供实证依据。

在编译性能优化评估方面，本研究首先通过消融实验，分别评估数据驻留技术和流水线处理技术带来的性能收益。通过控制变量法，深入分析各项优化技术的实际效果，为后续编译优化策略的制定提供数据支持。

在编程效率评估方面，本研究通过对比分析 Triton 实现与手写算子库在 MLU 平台上的代码实现，重点考察矩阵乘法、卷积和加法等典型算子的实现差异。实验不仅关注代码量的缩减程度，更深入评估 Triton 语言在简化硬件细节处理、提升代码可维护性等方面的实际效果。同时，通过对比 Triton 算子和 BANGC 算子的性能表现，全面评估编译性能优化策略的整体效果。

在跨平台性能评估方面，本研究在相同芯片功耗条件下，对比 MLU (SIMD) 和 GPU (SIMT) 平台上 Triton 算子的性能表现。这一评估不仅验证了 Triton 语言对 SIMD 架构的良好支持，更为不同硬件平台上的性能优化提供了基准参考。

通过上述系统性实验设计，本研究构建了 Triton 语言在 SIMD 架构上的全面性能评估框架，为深度学习编译器技术的优化提供了坚实的实证基础。

2.5.2 实验环境

1. 硬件平台

本章实验采用两种主流计算架构的硬件平台进行性能评估。在 SIMD 架构方面,选用 MLU370 云端 AI 芯片作为实验平台。该芯片采用先进的 7nm 制程工艺,在性能指标方面,MLU370 的板卡功耗为 75W,峰值计算性能达到 72 TOPS (半精度浮点类型),显存带宽为 307.2 GB/s。在 SIMT 架构方面,实验选用 NVIDIA T4 GPU 作为对比平台。该 GPU 基于图灵 (Turing) 架构,同样采用 7nm 工艺制造,功耗为 70W,使用 GDDR 显存,显存带宽达到 320 GB/s,峰值计算性能为 65 TOPS (半精度浮点类型)。通过选用这两种具有代表性的硬件平台,本研究能够全面评估 Triton 语言在不同计算架构下的性能表现。

2. 实验用例

为系统评估 Triton 语言的性能特征,本研究采用开源性能评测框架 Flag-Perf^[53]作为基准测试平台,以单算子性能作为核心评估指标。基于深度学习计算范式,实验选取了具有代表性的计算密集型算子作为测试对象。根据计算特征,这些算子可被系统划分为 Reduction 类和 Pointwise 类^[49]。其中,Reduction 类算子选取了矩阵乘法 (GEMM)、标准化 (LayerNorm) 和归一化 (Softmax) 等典型操作,Pointwise 类算子则选取了加法运算 (Add) 作为代表进行消融实验。更进一步地,为了验证 Triton 语言在 MLU 平台上支持的功能和性能泛化性,围绕上述这些典型算子,进一步扩充其算法类似的算子到 15 个,从而充分验证 Triton 语言的编程效率和跨平台的性能对比。这些算子不仅构成了深度学习模型的核心计算单元,同时也是硬件架构优化的关键目标。

本研究采用多维参数空间探索策略构建实验框架,具体测试算子类型及其规模参数详见表 2.1。在测试规模设计上,采用固定高维特征、遍历低维特征的系统性测试方法,通过控制变量法对算子规模进行多维度调节,深入探究不同计算负载下的性能变化规律,从而实现对 Triton 语言计算性能特征的全面评估与量化分析。在实验环境配置方面,本章实验采用 Triton v3.0.0 版本作为 GPU 后端实现基准^[54],并基于该版本进行 MLU 后端编译器的扩展开发,确保实验结果的可靠性与可比性。

2.5.3 Triton 语言编译性能优化消融实验

1. 任务调度优化消融实验

实验方法: 本实验在 MLU 370 平台上开展任务调度优化策略的消融实验,通过对比分析 Triton-GPU 原生调度方式与针对 SIMD 架构优化后的调度方式的性能差异,深入评估调度优化策略的有效性。实验采用系统性测试方法,选取算子规模参数 $n \in [0, 15]$,对矩阵乘法、标准化、归一化和加法等典型算子进

表 2.1 测试算子类型及规模^[63]

算子类型	具体算子种类	算子参数
矩阵乘法 (GEMM) 类算子	Matmul BatchMatmul	input: (1024, 2 ⁿ), weight: (2 ⁿ , 1024)
标准化 (Layernorm) 类算子	Layernorm SkipLayernorm Groupnorm	input: (1024, 2 ⁿ)
归一化 (Softmax) 类算子	Softmax Sigmoid Argmax Cumsum	input: (1024, 2 ⁿ)
加法 (Add) 类算子	Add Silu Cosine Exponential Reciprocal Random	input: (1024, 2 ⁿ)

注：计算类型均为半精度浮点类型 (float16)，n 的取值范围从 0 到 15。

行遍历测试。其中 Triton-GPU 的调度方式中，grid 设置为 1024，与输入规模的高维一致，num_warps 设置为 $2^n / 32$ ，这是针对 GPU 调度方式效率最高的配置。MLU 调度方式中通过编译期的优化处理，将 grid 调整为硬件中计算核的个数，将 num_warps 调整为 1，意味着使用 **BLOCK** 类型的任务，使用硬件中提供的全部分算核数量。

在 MLU 平台的调度优化策略中，编译器采用多层次数据块划分机制以实现计算任务与硬件资源的精确匹配。具体而言，该策略基于片上存储容量约束，采用自底向上的递归式数据划分方法。首先，编译器从计算任务的最低维度开始进行数据块划分，通过逐维度递增的方式，直至找到一个拆分块，使得该块在片上存储空间达到饱和状态。

实验结果与分析：实验结果如图 2.15 所示，优化后的调度方式在 MLU 370 平台上展现出显著的性能优势。通过定量分析发现，不同算子类型均实现了显著的性能提升：矩阵乘法 (MatMul) 平均加速比达到 3.04 倍，标准化 (LayerNorm) 提升 4.22 倍，归一化 (Softmax) 提升 4.04 倍，加法 (Add) 运算提升 4.65 倍。这一性能提升呈现出明显的规模效应，即随着输入规模参数 n 的增大，性能加速比呈现单调递增趋势。这种趋势主要源于数据块划分策略的优化效果：随着 n 值的增加，计算任务被划分为更大的连续数据块，从而显著提升了访存局部性，降低了数据搬运开销。

进一步分析表明，不同算子类型的性能提升幅度存在显著差异。其中，以访存操作为主要瓶颈的算子（如 Add、LayerNorm 和 Softmax）获得了更高的加速比，而以计算密集型操作为主的矩阵乘法 (MatMul) 则相对较低。这一现象验证了优化后的调度策略在缓解访存瓶颈方面的有效性。值得注意的是，当 n 值超

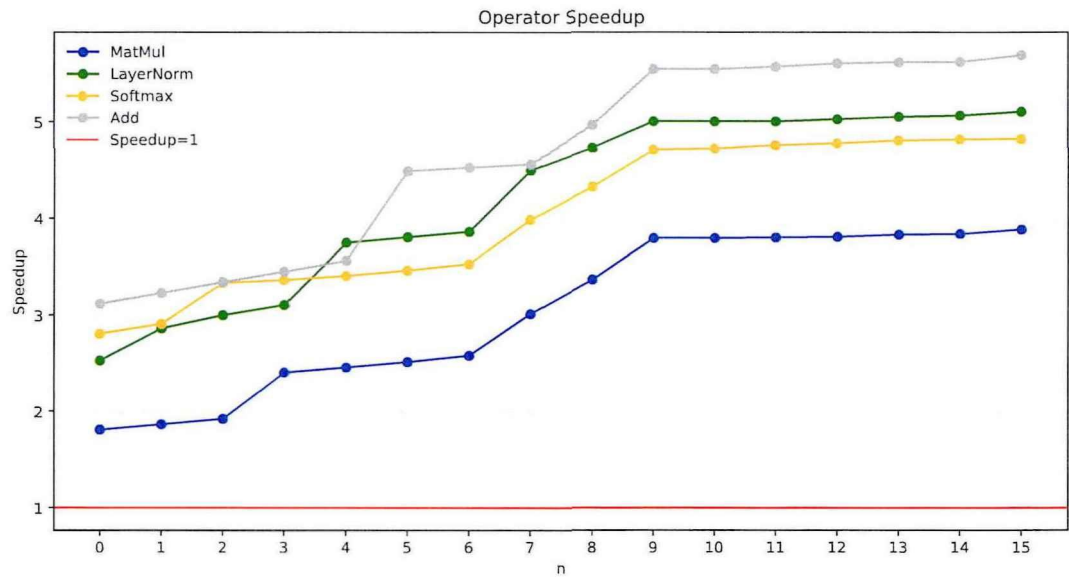


图 2.15 MLU 平台上 Triton 语言任务调度优化前后的性能对比

注：算子规模如表 2.1 所示，图中横坐标表示 n 的取值范围是 $[0, 15]$

过 10 后，性能提升趋于稳定。这是由于此时数据块大小已达到片上存储容量上限，进一步增大 n 值不会带来额外的访存优化效果。这一发现为后续优化策略的制定提供了重要参考，表明在特定硬件约束下，存在最优的任务划分粒度。

2. 二元算子性能优化消融实验

实验方法：本研究在 MLU 370 计算平台上开展二元算子性能优化策略的消融实验，系统评估数据驻留技术与流水线并行技术对计算性能的影响。实验采用全参数空间遍历法，对算子规模参数 $n \in [0, 15]$ 进行系统性测试，以全面考察优化策略在不同计算负载下的性能表现。

实验结果与分析：实验结果表明，二元算子性能优化策略在 MLU 370 平台上展现出显著的性能提升效果，其中矩阵乘法（MatMul）平均加速比达到 2.75 倍，标准化（LayerNorm）提升 1.93 倍，归一化（Softmax）提升 1.71 倍，加法（Add）运算提升 1.54 倍。通过深入分析实验结果，可以发现二元算子优化策略对不同类型的算子呈现出差异化的优化效果：计算密集型算子（如矩阵乘法）的优化效果显著优于访存密集型算子（如加法等）。

这种性能差异主要源于优化策略的作用机制：数据驻留技术和流水线并行技术对计算密集型算子具有显著的协同优化效果。具体而言，数据驻留技术通过减少数据搬运开销来提升计算效率，而流水线并行技术则通过重叠计算与访存操作来最大化硬件利用率。对于访存密集型算子，虽然数据驻留技术能够有效减少访存量，但由于其本身计算时间较短，流水线并行带来的性能收益相对有限。这一发现为后续优化策略的制定提供了重要启示：针对不同类型的算子特征，需

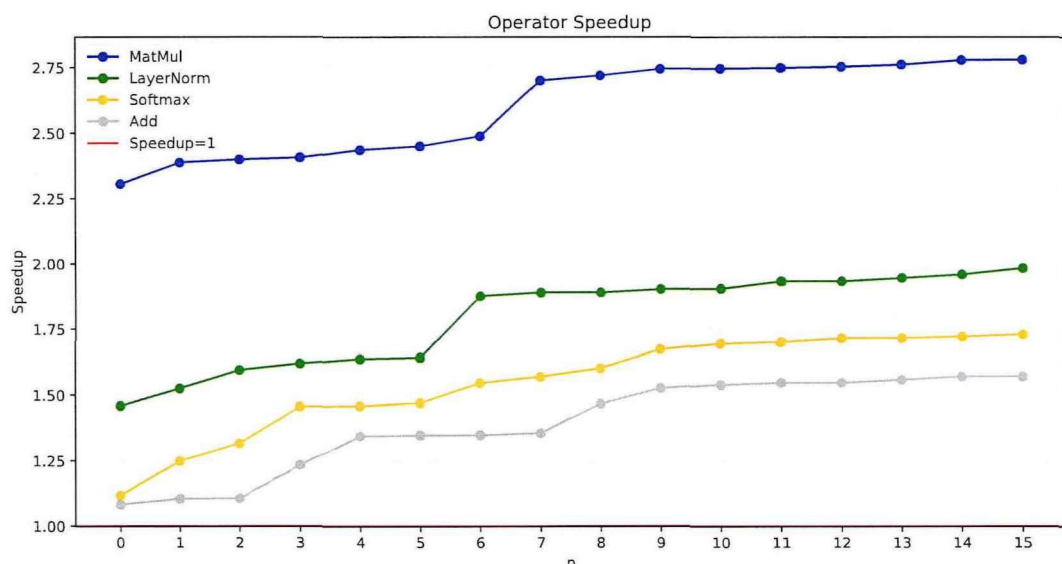


图 2.16 MLU 平台上二元算子性能优化前后的性能对比

注：算子规模如表 2.1 所示，图中横坐标表示 n 的取值范围是 $[0, 15]$

要采用差异化的优化策略组合，以实现最优的性能提升效果。实验结果充分验证了数据驻留技术与流水线并行技术在提升计算效率方面的协同效应，为深度学习编译器优化策略的制定与实施奠定了坚实的理论基础和实践依据。

2.5.4 Triton 语言在 MLU 平台上的编程效率评估

实验方法：本研究按照 Reduction 类和 PointWise 类的计算特征划分方式，从 FlagPerf^[53] 仓库中选取了 15 个算子如表 2.1 所示，以系统性评估 Triton 语言在 SIMD 架构下的性能泛化性以及编程效率。本研究采用控制变量法进行对比实验设计，在 MLU 370 计算平台上分别基于 BANGC 语言和 Triton 语言实现了这些算子。

为量化评估两种编程语言的开发效率，本研究引入代码行数（Line Of Code, LOC）作为核心评价指标，通过系统统计两种实现方案的代码规模，深入分析 Triton 语言在抽象硬件细节、降低编程复杂度等方面的实际效果。同时，本研究采用标准化的性能测试框架，对两种实现方案的计算效率进行对比分析，从开发效率和执行性能两个维度全面评估 Triton 语言在 MLU 平台上的综合表现。在实验参数设置方面，本研究选择 $n=15$ 作为基准测试规模，针对半精度浮点数（float16）这一数据类型进行实验，以确保实验结果的代表性和可靠性。

实验结果与分析：实验结果如图 2.17 所示，在相同算子实现条件下，Triton 语言展现出显著的开发效率与执行性能优势。具体而言，Triton 实现方案仅需 BANGC 语言 19.8% 的代码量，即可实现 1.4 倍的性能提升。这一结果充分验证了 Triton 语言在简化硬件编程复杂度、提升代码可维护性方面的显著优势，同时

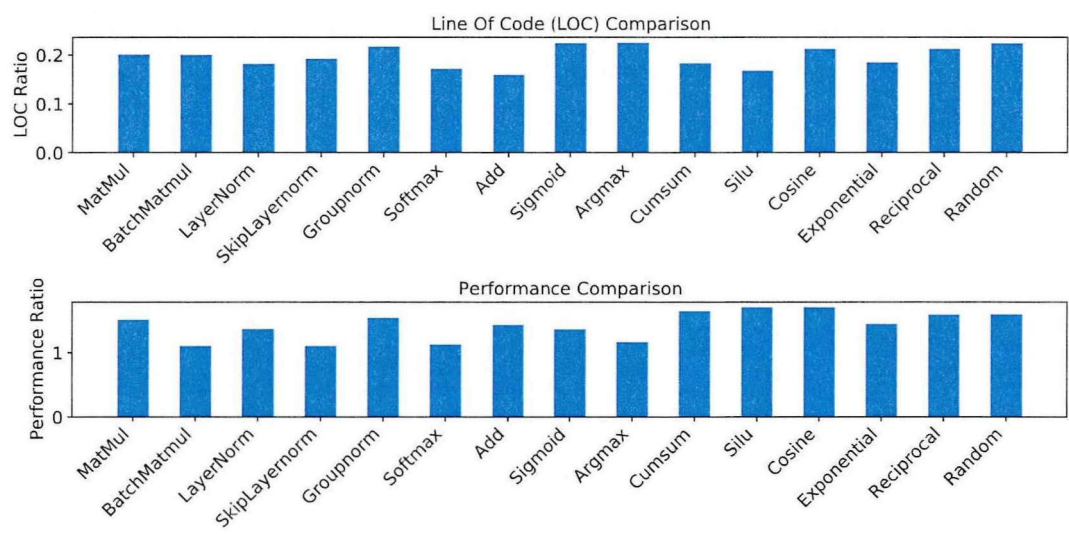


图 2.17 MLU 平台上 Triton 与 BANGC 语言的代码效率与性能对比

注：LOC Ratio 指的是 Triton / BANGC 的代码行数对比，Performance Ratio 指的是 BANGC / Triton 的执行时间对比。

其优化的编译器架构也确保了执行效率的显著提升。

2.5.5 Triton 语言跨平台的性能对比

实验方法：为全面评估 Triton 语言在不同计算架构下的性能表现，本研究采用控制变量法进行跨平台性能对比实验。实验选取 MLU370 平台作为 SIMD 架构的代表，NVIDIA T4 平台作为 SIMT 架构的代表，在相同功耗和算力条件下，通过系统性遍历矩阵乘法、标准化、归一化和加法等关键算子的规模参数，对 Triton 语言在两种架构下的性能表现进行定量分析。本实验采用的规模和数据类型对齐 2.5.4 中的实验设置。

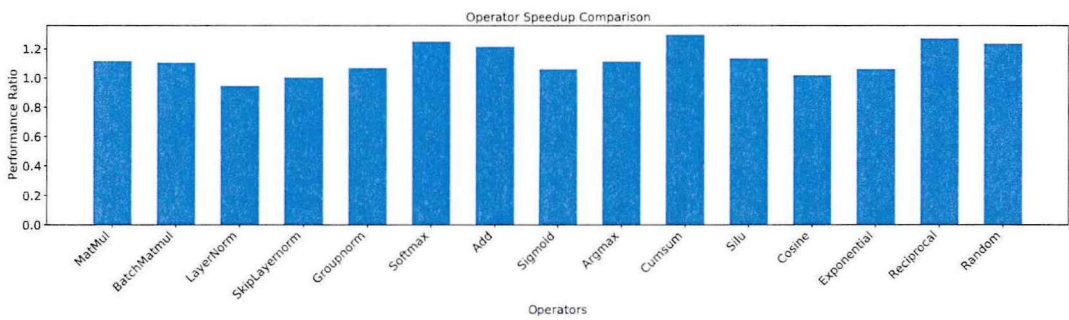


图 2.18 Triton 语言编写的算子在 MLU 与 GPU 平台上的性能对比

注：Performance Ratio 是 GPU 平台相对于 MLU 平台的执行时间（ns）比例。

实验结果与分析：实验结果如图 2.18 所示，在相同计算负载下，Triton 语言在 SIMD 架构上展现出与 SIMT 架构相当的性能表现。具体而言，在相近功耗与算力条件下，SIMD 架构可实现平均 1.1 倍的性能提升。这一发现不仅验证了

Triton 语言在 SIMD 架构下的适配完备性,更为深度学习编译器在不同计算架构上的性能优化提供了重要参考。实验结果充分证明了 Triton 语言在跨平台性能优化方面的显著优势,为后续编译器优化策略的制定提供了实证依据。

2.6 讨论

编译器进一步发掘优化潜力:基于本研究成果,Triton 编译器在性能优化与跨平台适配方面仍具有显著的提升空间。首先,在性能优化方面,当前实现方案仍存在进一步优化的潜力。具体而言,可以通过引入冗余数据拷贝消除技术来减少不必要的数据搬运开销。这种优化策略能够有效降低内存带宽压力,特别是在处理大规模张量运算时,可以显著提升计算效率。此外,还可以探索基于数据依赖分析的优化技术,通过识别和消除冗余计算来进一步提升性能。

跨平台适配与国产处理器支持:在跨平台适配方面,Triton 编译器展现出良好的可扩展性。其模块化架构设计使得硬件相关部分与硬件无关部分实现了有效解耦,这种设计模式为支持更多国产处理器提供了便利。以华为 Ascend 910 处理器为例,通过复用现有的硬件无关优化模块,仅需针对 Ascend 架构的特定硬件特性进行适配优化,即可实现快速移植。这种适配策略不仅能够显著降低开发成本,还能确保在不同硬件平台上保持一致的性能表现。

2.7 本章小结

本章针对 Triton 语言在 SIMD 架构上的优化与应用展开了系统性研究,主要取得了以下创新性成果:

首先,在任务调度优化方面,提出了一种基于硬件资源感知的任务参数重映射算法。该算法通过将 SIMT 架构的 grid 参数转换为 SIMD 架构的三维任务空间,并引入任务类型标识机制,实现了计算任务与硬件资源的精确匹配。实验结果表明,该优化策略在 MLU 370 平台上实现了显著的性能提升,其中矩阵乘法、标准化、归一化和加法运算分别获得了 3.04 倍、4.22 倍、4.04 倍和 4.65 倍的加速比。

其次,在二元算子性能优化方面,提出了数据驻留与流水线并行相结合的优化策略。通过数据驻留技术减少数据加载频次,结合流水线处理技术实现计算与访存的并行化,显著提升了计算效率。实验结果表明,该优化策略在矩阵乘法、标准化、归一化和加法运算上分别实现了 2.75 倍、1.93 倍、1.71 倍和 1.54 倍的性能提升。

在编程效率评估方面,通过对比 Triton 与 BANGC 语言的实现方案,验证了

Triton 语言在简化硬件编程复杂度方面的显著优势。实验结果表明, Triton 实现方案仅需 BANGC 语言 19.8% 的代码量, 即可实现 1.4 倍的性能提升, 充分体现了 Triton 语言在提升开发效率和执行性能方面的双重优势。

最后, 通过跨平台性能对比实验, 验证了 Triton 语言在 SIMD 架构上的适配完备性。在相同计算负载下, SIMD 架构可实现平均 1.1 倍的性能提升, 证明了 Triton 语言在跨平台性能优化方面的显著优势。

本章研究成果为深度学习编译器在 SIMD 架构上的优化提供了理论依据和实践指导, 对推动深度学习计算框架的跨平台适配具有重要意义。

第3章 面向 SIMD 架构智能处理器的算子融合数据流评估框架

3.1 引言

深度神经网络在计算机视觉和自然语言处理等领域的广泛应用,推动了专用硬件加速器的发展。从计算机视觉^[4,55-56]到自然语言处理^[9,57-58],深度神经网络展现出强大的变革性力量。然而,这些任务通常涉及大量复杂的操作和数据流交互,对计算资源提出了严峻挑战。为应对这一挑战,专用 DNN 加速器^[21,59]应运而生,这些加速器在内存层次结构和计算能力方面呈现出显著差异。随着 DNN 任务复杂性与硬件加速器多样性之间的差距不断扩大,如何在可接受代价下实现性能优化,成为当前研究的关键问题。

计算图编译器作为连接神经网络与硬件架构的关键桥梁,面临着复杂的映射和调度挑战。其主要挑战在于如何在异构硬件架构上实现高效的计算图映射和调度。具体而言,计算图编译器需要解决三个核心问题:首先,如何优化数据流以最小化内存访问开销;其次,如何在多级内存层次结构中实现高效的数据搬运;最后,如何将计算图的操作合理映射到特定的硬件资源。这些问题的解决直接影响着最终的计算性能。

数据流评估框架作为计算图编译器的核心组件,为上述挑战提供了系统性的解决方案。现有框架^[60-63]通过三个关键功能支持计算图编译器的优化决策:首先,深入分析计算图的数据依赖关系;其次,探索最优的循环分块和算子融合策略;最后,评估不同硬件映射方案的成本效益。这些功能共同构成了数据流优化的基础。然而,随着 DNN 加速器架构的不断演进,现有方法^[64-65]在硬件抽象方面仍存在不足,难以充分利用新型硬件架构的计算潜力。

现代调度框架面临的核心挑战在于如何有效平衡计算效率与内存访问开销。现有调度框架^[60-62]普遍采用循环分块技术,通过将操作数据划分为适应片上内存容量的数据块,并优化其执行顺序来提升调度性能。然而,这些框架主要局限于单个操作的分块优化,导致频繁的片外内存访问,在有限的内存带宽约束下严重制约了 DNN 加速器的性能表现。

为突破这一性能瓶颈,研究者提出了融合数据流框架^[40,66-67]。这类框架通过算子融合技术,在片上内存中保留中间计算结果,显著提升了数据局部性并减少了片外内存访问。具体而言,融合数据流框架需要解决两个关键问题:首先,框架需要遍历计算图以识别最优的融合模式,但由于 DNN 任务的复杂性,融合空间的搜索被证明是一个 NP 难问题^[68];其次,框架需要为融合后的操作确定

最佳分块策略,这要求对中间数据元素进行细粒度分析,并确保融合操作能够在相同维度上进行分块。这种分块与融合策略的联合优化进一步增加了搜索空间的复杂性,使得寻找全局最优解极具挑战性。现有方法往往局限于预定义的融合模式^[65-66]或固定分块策略^[67,69-70],难以充分探索联合优化空间。

针对上述挑战,本研究提出了 FusionFrame 框架,通过创新的三维调度空间建模和以存储为中心的表示方法,实现了 DNN 调度性能的显著提升。该框架的核心创新体现在三个方面:首先,从算子融合、循环分块和硬件映射三个维度构建了完整的调度空间,并提出了新颖的以存储为中心的表示方法,为融合数据流优化提供了系统性的解决方案(第 3.3 节);其次,开发了高效的分析模型,通过迭代探索调度空间并评估性能指标,实现了内存占用与延迟的精确估计(第 3.4 节);最后,通过全面的案例研究,深入分析了不同维度选择对性能的影响,为 DNN 调度优化提供了重要指导(第 3.5 节)。

3.2 相关背景和研究动机

3.2.1 调度表示和原语

调度表示和原语构成了深度神经网络硬件加速优化的理论基础。调度表示定义了 DNN 操作执行顺序和资源分配的设计空间,而调度原语则作为基本构建模块,实现了从高级 DNN 模型到硬件优化执行计划的转换。

传统调度方法主要采用循环级转换技术,如循环分块等,以提升数据局部性和降低内存访问延迟。Halide^[28,71]和 TVM^[22]等框架通过将调度与硬件细节解耦,实现了更灵活的优化策略。然而,这些基于循环的原语在表达操作与内存层次结构间的复杂交互时存在明显局限,特别是在处理算子融合和多级内存映射等场景时表现不足。

为突破传统方法的局限性,FusionFrame 提出了一种创新的以内存为中心的调度表示方法。该方法不仅保留了循环级转换的优势,还通过显式建模操作与内存层次结构间的关系,实现了更全面的调度空间描述。

3.2.2 融合操作的分块

分块策略对单个操作性能的影响已得到广泛研究,但在融合操作场景下的分块优化仍面临诸多挑战。现有研究表明,融合操作通常采用输出维度分块策略^[67,69-70,72],这种选择源于输入分块带来的同步复杂性,即需要协调多个输入分块与输出分块间的数据依赖关系。相比之下,输出分块具有天然的独立性,更易于实现并行化处理。

对于卷积(CONV)和池化(Pool)等滑动窗口操作,传统的高度和宽度

维度分块策略会导致显著的冗余数据开销，且这种开销随融合深度增加而急剧上升^[40,67,70]。现有框架过度关注输出高度和宽度维度，忽视了输出通道维度的优化潜力。这种局限性在 ResNet-50^[4]等深层网络中尤为突出，其中 3x3 卷积核配合 512x512 输入输出通道的配置，仅单个操作就需 2.3MB 的片上内存（int8 数据类型），对加速器提出了严峻的内存挑战^[21,59]。当前最先进的工作^[40,67,70]尚未有效解决这一关键问题。

3.2.3 硬件映射探索

DNN 工作负载在多级内存架构上的映射优化是一个复杂的系统性问题。尽管已有大量研究关注单个操作的硬件映射，如 Timeloop^[60]、MAESTRO^[61]和 ZigZag^[62]等框架，但在融合数据流的系统化映射方法方面仍存在明显不足。

当前最先进的解决方案主要存在两个局限：一是过度关注 DRAM 访问优化而忽视片上数据搬运的重要性；二是采用简单地将多余数据溢出到更高内存级别的策略^[67,70]，缺乏系统性的内存分配方案。这些局限性严重制约了 DNN 加速器在复杂任务场景下的性能表现。

3.2.4 数据流调度优化问题建模

从形式化建模的角度，DNN 数据流调度优化可被抽象为一个多维约束优化问题，涵盖调度表示、融合分块和硬件映射三个核心维度。给定一个包含 n 个算子的计算图

$$\mathcal{G} = \{o_1, o_2, \dots, o_n\} \quad (3.1)$$

其中每个算子 o_i 的输入张量维度为

$$I_i = (N_i, H_i, W_i, C_i) \quad (3.2)$$

输出张量维度为

$$O_i = (N_i, H'_i, W'_i, C'_i) \quad (3.3)$$

调度优化问题需要在三个维度上实现协同优化：

首先，在调度表示维度，需要构建统一的抽象模型

$$S = (V, E, L, P) \quad (3.4)$$

其中 V 表示算子集合， E 描述算子间的数据依赖关系， L 指定内存层次结构， P 定义分块策略集合。该模型需要支持从细粒度到粗粒度的多层次调度表示，并能够精确描述算子与内存层次结构间的交互关系。

其次，在融合分块维度，分块策略

$$B = (b_n, b_h, b_w, b_c) \quad (3.5)$$

定义了分批处理、高度、宽度和通道维度上的分块粒度。该维度需要为每个算子 o_i 构建分块映射函数

$$f_i : I_i \rightarrow \{B_1, B_2, \dots, B_k\} \quad (3.6)$$

在满足内存容量约束

$$M_{\text{input}} + M_{\text{output}} + M_{\text{weight}} \leq M_{\text{cache}} \quad (3.7)$$

的前提下，实现最优分块组合。

最后，在硬件映射维度，需要构建多级内存映射函数

$$\mathcal{M} : \mathcal{G} \rightarrow \{L_0, L_1, \dots, L_k\} \quad (3.8)$$

将计算图中的算子合理分配到不同内存级别。该映射需要综合考虑数据局部性、内存带宽和计算并行度等因素。

从优化目标的角度，调度优化可以根据具体应用场景的需求进行灵活调整。在内存受限场景下，可以采用内存占用优先的优化策略，通过最小化内存占用

$$\min(M_{\text{total}}) \quad (3.9)$$

来确保计算任务在有限资源下的可执行性；在实时性要求较高的场景下，则可以采用执行时间优先的优化策略，通过最小化总执行时间

$$\min(T_{\text{total}}) \quad (3.10)$$

来满足严格的延迟约束。这种多目标优化能力使得调度框架能够适应多样化的应用需求。

从系统设计的角度来看，有效的调度优化需要满足三个关键约束：首先，维度一致性要求融合组中所有算子的分块维度保持一致；其次，数据依赖约束要求相邻算子的输出分块与输入分块精确对齐；最后，层次化支持要求调度策略能够适应从 L0、L1 等多级内存层次结构。这些约束共同构成了 DNN 调度优化的核心挑战，也是当前研究亟待突破的关键问题。

综上所述，调度框架在算子融合、分块策略和硬件映射等方面仍面临诸多挑战。**FusionFrame** 通过构建全面的调度空间并引入以内存为中心的表示方法，为解决这些挑战提供了系统性的解决方案。

3.3 计算图融合数据流的调度空间

融合数据流的调度空间是一个多维优化问题，其核心在于协调算子融合、循环分块和硬件映射三个关键维度^[40,67]。这三个维度相互耦合，共同决定了数据流调度的性能表现。

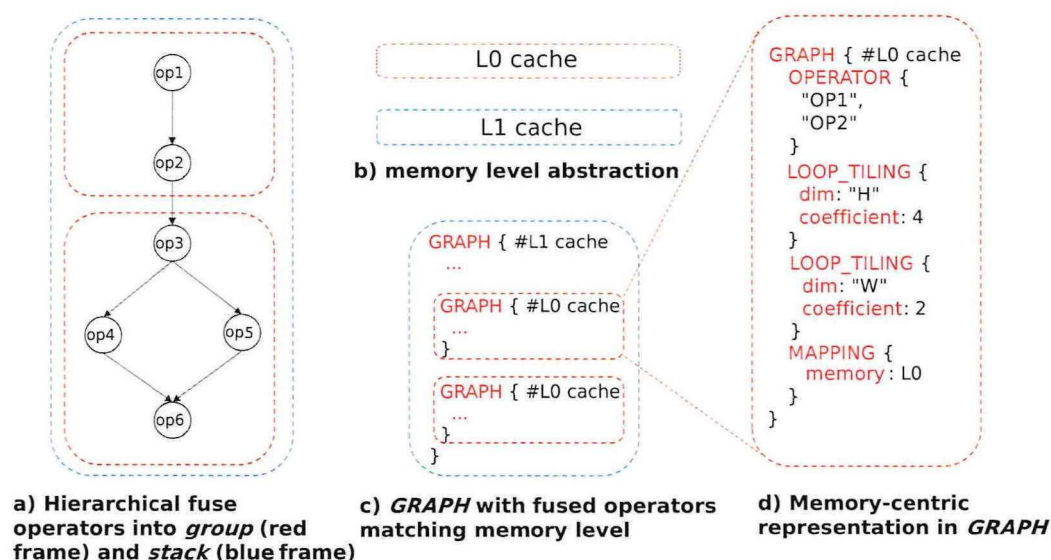


图 3.1 以存储为中心的调度空间表示

在算子融合维度，调度空间需要支持从细粒度到粗粒度的多层次融合策略。具体而言，调度框架不仅要支持单个算子的局部融合，还要能够实现多个融合组的层次化组合，从而构建完整的工作负载。如图 3.1a 所示，这种层次化融合机制能够有效提升数据局部性，减少片外内存访问。

循环分块维度则关注数据布局的优化与分块策略的灵活性。调度空间需要精确掌握数据布局特征，并为每个维度提供可调节的分块大小选项。这种灵活性使得调度器能够根据具体硬件特性和计算需求，动态调整分块策略，实现最优的数据局部性。

硬件映射维度强调对多样化硬件架构的支持能力。现代 DNN 加速器通常采用多级内存层次结构，调度空间需要能够有效探索不同内存级别之间的映射关系，充分利用硬件资源。这种多级映射能力对于实现高效的数据搬运和计算至关重要。

面对如此庞大的调度空间，需要一种结构化的表示方法来实现高效探索。本工作提出了一种创新的以存储为中心的表示方法，该方法能够全面而有效地描述调度空间，为后续的优化决策提供坚实基础。

3.3.1 以存储为中心的表示

以存储为中心的表示方法基于一个关键观察：嵌套循环结构与内存层次结构之间存在天然的对对应关系。每个内存级别都最适合特定层次的循环嵌套计算，这种对应关系为调度优化提供了重要启示。

本工作引入 *GRAPH* 概念来表示特定内存级别的调度信息。调度空间的形式

化定义为

$$S = \{G_1, G_2, \dots, G_n\} \quad (3.11)$$

其中每个 GRAPH G_i 可表示为四元组 (V_i, E_i, L_i, P_i) 。具体而言,

$$V_i = \{v_1, v_2, \dots, v_m\} \quad (3.12)$$

表示融合算子集合,

$$E_i \subseteq V_i \times V_i \quad (3.13)$$

描述算子间的拓扑关系和数据依赖, L_i 指定融合所在的存储层级 (如 L0、L1 或 DRAM 等),

$$P_i = \{p_1, p_2, \dots, p_k\} \quad (3.14)$$

则定义了分块策略集合。

分块策略 p_j 可进一步表示为二元组 (d_j, s_j) , 其中 d_j 表示分块维度 (如高度 H、宽度 W 或通道 C 等), s_j 则指定具体分块大小。这种形式化表示方法能够完整描述调度空间的三个关键维度: 算子融合、存储层级映射和循环分块。因此, 完整的调度空间可形式化为:

$$S = \{(V_i, E_i, L_i, \{(d_j, s_j)\}_{j=1}^k)\}_{i=1}^n \quad (3.15)$$

如图 3.1b 所示, 对于具有两级片上内存的硬件架构, 算子首先在 L0 缓存级别进行融合 (如红色框架所示), 形成初步的融合组。这些融合组随后在 L1 缓存级别进行更高层次的融合 (如蓝色框架所示), 形成融合算子的堆栈。这种层次化的表示方法能够自然地扩展到任意数量的内存级别, 如图 3.1c 所示。

以存储为中心的表示方法具有显著优势: 它通过显式地将内存级别分配给融合算子, 有效弥合了分块循环与硬件内存层次结构之间的鸿沟。这种表示不仅支持层次化算子融合, 还能够灵活适应各种多级内存系统, 为调度优化提供了统一的抽象框架。

与现有工作相比, 本方法在表示能力和优化效率方面具有显著优势。MAE-STRO^[61]采用以数据为中心的表示方法, 主要关注数据流模式的分析, 但难以有效处理多级内存层次结构下的算子融合问题。Timeloop^[60]采用以计算为中心的表示方法, 虽然能够精确建模计算过程, 但在描述算子间数据依赖关系时存在局限。Tenet^[73]提出以关系为中心的表示方法, 通过张量关系图描述算子间依赖, 但缺乏对硬件内存层次结构的显式建模能力。

相比之下, 本工作提出的以存储为中心的表示方法具有三个独特优势: 首先, 通过显式建模内存层次结构与算子融合的关系, 能够更准确地评估数据搬运开销; 其次, 采用层次化的表示结构, 支持从细粒度到粗粒度的多层次融合策

表 3.1 以存储为中心的表示原语

原语		描述
OPERATOR		GRAPH 中的融合算子
MAPPING	memory	指定映射的内存级别
LOOP_TILING	dim	分块维度
	coefficient	分块大小

略；最后，通过统一的抽象框架，能够灵活适应各种硬件架构。这些优势使得本方法在复杂 DNN 任务调度中展现出更好的优化效果。

3.3.2 以存储为中心的表示原语

本工作创新性地提出了基于算子融合策略描述的调度原语体系，在目标定位和功能设计上与现有工作形成显著区分。与 TVM^[22]、Halide^[28]等工作中以计算过程描述为核心的调度原语不同，本工作将设计重点转向算子融合策略的描述，这一转变源于对调度优化本质的深入理解：算子融合策略的精确描述是构建高效调度空间的关键。基于这一认知，本工作通过有限个核心原语即可完整描述整个算子融合策略优化空间，实现了调度描述的高效性和完备性的统一。具体而言，FusionFrame 基于优化空间的三个关键维度，定义了 OPERATOR、MAPPING 和 LOOP_TILING 三个核心调度原语（如表 3.1 所示），这些原语不仅为调度空间的系统化探索提供了基础支撑，同时也为后续和优化决策建立了统一的描述框架。

OPERATOR 原语负责处理 GRAPH 中的融合算子，专注于算子间的数据依赖关系优化。每个算子通过唯一标识符进行管理，并通过提升中间数据实现融合。如图 3.1c 所示，*op1* 和 *op2* 通过提升中间数据到 L0 缓存实现融合，而 *op3-op6* 则形成另一个融合组。层次化融合通过将 *op2* 和 *op3* 之间的中间数据提升到 L1 缓存实现，这种机制有效支持了从细粒度到粗粒度的多层次融合策略。

MAPPING 原语通过 *memory* 参数指定当前 GRAPH 对应的内存级别，实现了硬件映射维度的精确控制。如图 3.1d 所示，该原语明确指示了 *op1* 和 *op2* 之间的中间数据映射到 L0 缓存的具体实现。这种显式的内存级别分配机制有效弥合了分块循环与硬件内存层次结构之间的鸿沟，为多样化硬件架构提供了统一的支持。

LOOP_TILING 原语定义了特定内存级别内的循环重组策略，实现了循环分块维度的灵活优化。该原语包含两个关键参数：*dim* 表示分块维度（如高度 *H* 或宽度 *W*），*coefficient* 则指定具体分块大小。如图 3.1d 所示，*op1* 和 *op2* 的融合调度中，维度 *H* 的分块大小为 4，维度 *W* 的分块大小为 2。这种灵活的分块机制为探索不同数据布局的调度空间提供了可能，同时确保了数据局部性的优化。

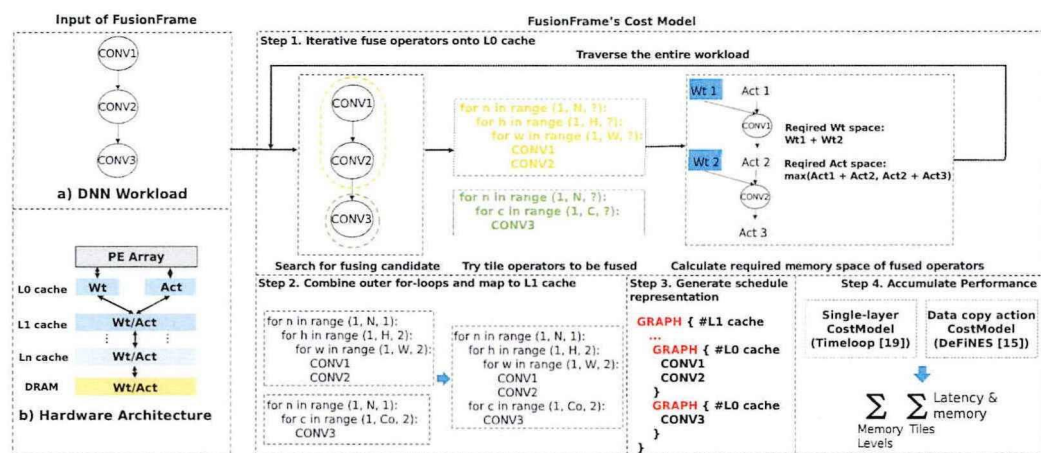


图 3.2 FusionFrame 的整体架构

3.4 基于量化分析的性能评估模型

本节描述了 FusionFrame 的量化分析成本模型，该模型接受两个输入：原始计算图和 DNN 加速器的硬件架构，如图 3.2 所示。计算图表示 DNN 工作负载，包括基于滑动窗口的算子（例如，卷积、池化）和跨各种网络架构的逐元素算子。FusionFrame 中的硬件架构将加速器抽象为 PEs 和多级内存层次结构。PEs 支持任意维度和任意大小的数组，内存层次结构被抽象为多级片上内存，存储权重（表示为 Wt ）和激活值（表示为 Act ）。成本模型探索了第 3.3 节中定义的调度空间，生成调度空间表示，并评估其性能，包括每个内存级别的延迟和内存访问。本章重点讨论密集工作负载，详细步骤如下。

3.4.1 以存储为中心的融合机制

FusionFrame 采用以存储为中心的融合机制，遍历计算图以融合算子并将算子之间的中间数据分层次映射到每个内存级别。

首先，所有算子被融合并分块，以将内部分块循环映射到最内层、最快的内存（例如，L0 缓存），形成算子组。接下来，识别这些组的外部公共 for 循环并进行组合。这些组合的外部 for 循环随后被映射到更高的内存级别。根据指定内存级别的容量和整个工作负载的计算图，算子被融合，如图 3.2 所示。

从计算图中的最后一个算子开始，融合过程搜索要融合到 L0 缓存的算子，如图 3.2 步骤 1 所示。它使用深度优先贪婪搜索算法以逆拓扑顺序遍历整个图，如 DeFiNES^[67] 中所述。这种方法可以被更高级的搜索算法^[74-76] 替代以获得更好的性能，本工作将其留待未来研究。一旦识别出要融合的算子，FusionFrame 尝试通过选择适当的分块维度和大小来分块这些算子。分块维度按布局顺序选择：首先是维度 N ，然后是维度 H 和 W ，最后是维度 C （如果遇到大权重）。分块维度和大小的选择必须遵循以下规则：

- 要融合的算子必须在同一维度上分块。FusionFrame 中支持的分块维度在第 3.4.2 节中讨论。
- 分块的中间数据总大小必须适合指定的内存级别。分块中间数据大小计算方法在第 3.4.3 节中提供。

如果 FusionFrame 找不到满足指定规则的分块维度和大小, 融合过程将失败, 并提示重新搜索其他可融合的算子。此过程继续进行, 直到遍历整个计算图。一旦 L0 缓存上的融合完成, FusionFrame 遍历所有融合组, 识别并组合它们的外部分块循环, 将这些组合的循环映射到更高的内存级别, 如 L1 缓存, 如图 3.2 步骤 2 所示。处理完所有内存级别后, FusionFrame 生成调度的以存储为中心的表示, 并通过汇总每个内存级别和每个分块的所有算子的成本来评估整体性能, 如图 3.2 步骤 3 和步骤 4 所示, 并在第 3.4.4 节中详细说明。融合过程对每个内存级别遍历整个计算图一次, 导致时间和空间复杂度均为 $O(n)$, 其中 n 是图中的节点数。对于具有单批次输入的 ResNet-50^[4], FusionFrame 在 Intel Xeon Gold 6142 CPU @ 2.6GHz 的单线程上在不到 2 分钟内完成搜索。

图 3.3a 说明了跨不同内存级别的分块数据映射。对于 DRAM 中 $3 \times 4 \times 4 \times 1$ (NHWC) 的输出形状, 算子最初通过分块 N 、 H 和 W 维度融合到 L0 缓存上, N 的分块大小为 1, H 和 W 的分块大小均为 2。这些分块对应于三个嵌套的 for 循环。 H 和 W 维度的内部循环映射到 L0 缓存, 分块大小为 $1 \times 2 \times 2 \times 1$ (NHWC)。 N 维度的外部循环映射到更高的内存级别, 即 L1 缓存。

图 3.3b 说明了三个卷积算子的工作负载的两种可能融合结果。左边的例子展示了将所有三个算子融合到一个组中, 将 H 和 W 维度的循环映射到 L0 缓存, 并将 N 维度的最外层循环映射到 L1 缓存。右边的例子展示了将算子融合到两个组中, 其中 CONV1 和 CONV2 在 H_o 和 W_o 维度上分块, CONV3 在 C_o 维度上分块。这两个组映射到 L0 缓存, 它们的最外层循环被组合并映射到 L1 缓存。

3.4.2 分块融合算子

FusionFrame 分块算子以将中间数据适应不同的内存级别, 这需要仔细选择分块维度和大小。它支持分块融合算子的所有输出维度。分块不同的输出维度对中间数据的影响不同: 在 H_o 和 W_o 维度上分块影响中间激活大小, 而在 C_o 维度上分块影响中间权重大小。

图 3.4 说明了沿不同维度的分块。为简单起见, 激活批次大小设置为 1。图 3.4a 显示了典型的卷积布局。图 3.4b 展示了一个典型的卷积链, 每个算子的激活 (表示为 Act) 和权重 (表示为 Wt)。图 3.4c 显示了在融合 CONV1 和 CONV2 时在输出 H_o 和 W_o 维度上分块所需的数据。计算 Act_3 中每个分块所需的激活和

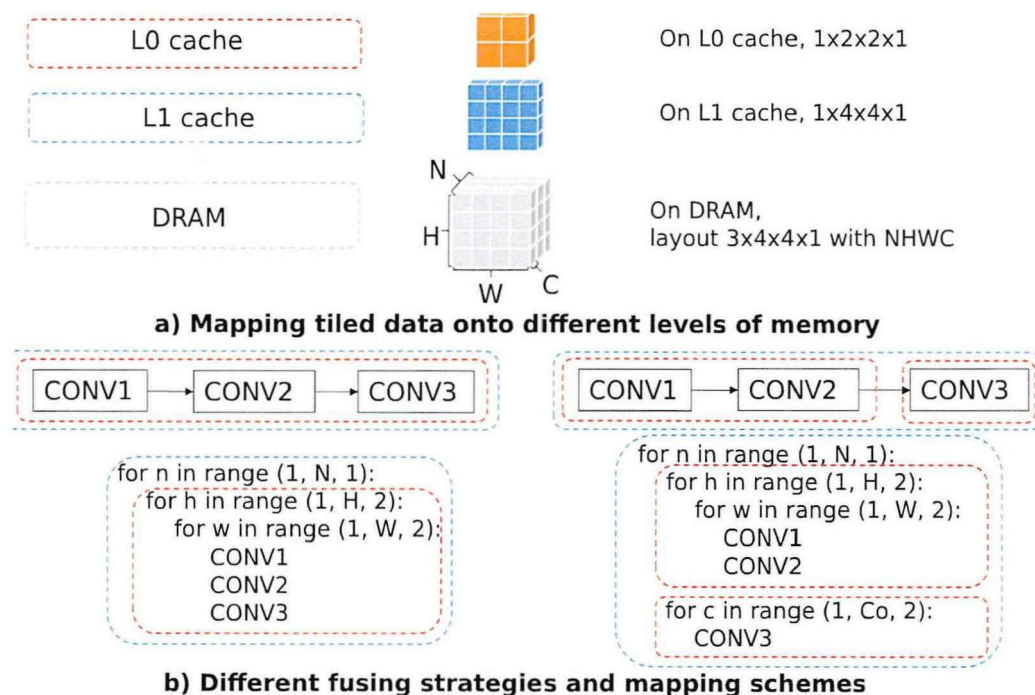


图 3.3 以存储为中心的融合机制示意图

注：橙色框架表示映射到 L0 缓存的算子或 for 循环，蓝色框架表示映射到 L1 缓存的算子或 for 循环

权重在橙色中突出显示。在此方案中，仅激活数据被分块，因此 $Wt1$ 和 $Wt2$ 驻留在 L0 缓存中， $Act1$ 、 $Act2$ 和 $Act3$ 的分块从 DRAM 加载。

图 3.4d 显示了沿输出通道维度 Co 分块的另一种情况。最内层循环的一次计算需要权重数据的分块和整个输入数据。此方案将输入数据驻留在 L0 缓存中，而分块的权重数据在片上内存和 DRAM 之间交换。这避免了冗余的分块数据传输，因为 Co 分块之间没有重叠，但需要在继续下一个算子之前收集所有输出分块。这种收集可以在多级内存架构的 L1 缓存上执行，从而能够在 Co 维度上分块，并减少融合具有大权重的算子的内存压力。

当分块可行时，选择适合内存的最大可行分块大小 (T_{max}) 使用二分搜索。分块大小可以在 1 和 T_{max} 之间进一步细化以获得最佳性能，本工作将分块大小的细粒度探索留待未来工作。

3.4.3 分块数据形状计算和内存分配

FusionFrame 沿每个分块维度遍历融合算子，以计算每个中间数据的分块形状。此遍历从分块输出开始，为每个算子建立输入和输出维度之间的映射，以推导分块输入形状。支持的算子分为两类：

- 维度无关算子：这些算子（例如，逐元素算子）的分块输入形状与其分块输出形状相同。

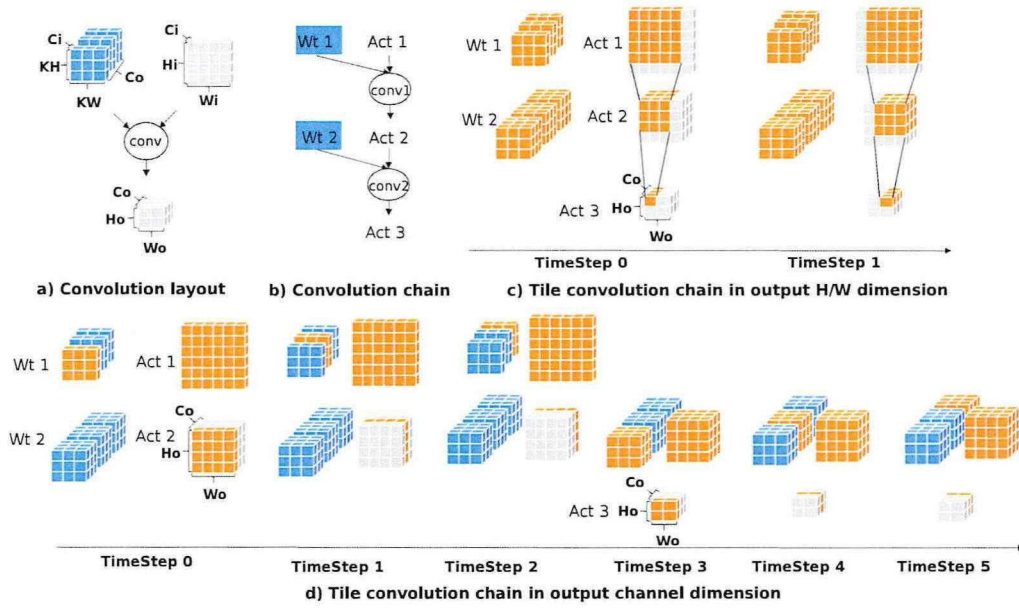


图 3.4 分块输出维度的示意图，活动分块在橙色中标记，表示为 Act_tile

表 3.2 卷积的输入形状和输出形状映射

维度	输入 (i) vs. 输出 (o)
N	$N_i = N_o$
H	$H_i = (H_o - 1) \times S_h + K_h - 2 \times P_h + D_h \times (K_h - 1) + 1$
W	$W_i = (W_o - 1) \times S_w + K_w - 2 \times P_w + D_w \times (K_w - 1) + 1$
C	C_i 和 C_o 不相关

注：映射关系中的参数： S 表示步幅 (Stride)， K 表示核 (Kernel)， P 表示填充 (Padding)， D 表示膨胀 (Dilation)

- 维度敏感算子：这些算子（例如，基于滑动窗口的算子，如卷积、池化）在输入和输出形状之间具有复杂的映射。表 3.2 显示了卷积的输入-输出映射，可以扩展到其他算子。

使用这些映射，可以确定每个中间数据的分块形状。对于分支，不同分支的数据形状被合并，如 DeFiNES^[67] 中所做的。

计算分块形状后，可以为中间数据分配内存。FusionFrame 按顺序执行融合算子，简化了同步并优化了内存使用。在此执行模型下，任何时刻所需的激活内存是当前算子的输入和输出的总和。为了计算最大内存需求，本工作遍历所有融合算子并计算：

$$M_{required} = \max_{i \in \{1, \dots, n\}} \left(\sum_{j=1}^{m_i} A_{ij} + \sum_{k=1}^{o_i} A_{ik}, \sum_{j=1}^{m_{i+1}} A_{i+1,j} + \sum_{k=1}^{o_{i+1}} A_{i+1,k} \right) \quad (3.16)$$

其中 $M_{required}$ 表示总所需的内存空间， n 表示 DNN 中的总算子数。 m_i 表示算子 i 的输入数， o_i 表示算子 i 的输出数， A_{ij} 和 A_{ik} 表示算子 i 的第 j 个和第 k 个输入的内存占用。函数 $\max$ 确保捕获执行过程中任何点的最大内存使用。内部表达式遍历每个算子并计算两个潜在的内存需求。对于图 3.4b 中的简单工作负载示例，此

卷积链中所需的激活内存空间为 $\max(Act1_tile + Act2_tile, Act2_tile + Act3_tile)$ 。

3.4.4 性能评估

FusionFrame 使用以存储为中心的方法计算每个内存级别上每个融合算子的成本。对于 L0 缓存，算子在 L0 缓存和 PE 之间执行数据搬运，并在 PE 中进行计算，使用 Timeloop^[60] 成本模型进行评估。FusionFrame 还可以集成其他 SotA 成本模型，如 ZigZag^[62] 和 MAESTRO^[61]。对于更高的内存级别，如 L1 缓存和 DRAM，仅考虑数据复制操作。FusionFrame 采用 DeFiNES^[67] 中的数据复制操作成本模型来测量在内存级别之间搬运权重和激活的延迟，考虑总数据搬运大小和内存带宽，以及潜在的非线性因素，如内存端口冲突。然后通过汇总每个内存级别的成本（包括计算和数据搬运）来计算融合程序的总成本，如图 3.2 步骤 4 所示。

总之，FusionFrame 使用以存储为中心的方法跨多个内存级别优化算子融合。它计算分块维度和大小以适应这些级别，确保高效利用片上内存层次结构。计算和数据搬运成本被评估并累加以确定最终的性能成本。

3.5 实验评估与分析

3.5.1 实验目标

本实验旨在通过系统性评估，深入探究融合数据流框架在智能处理器上的性能表现和适用性。实验主要围绕三个核心问题展开，以验证所提出方法的有效性和普适性。

首先，实验将探究处理器芯片的多级存储层次是否必然带来最优性能。尽管多级存储层次被广泛认为是提升处理器性能的关键设计，但其实际效果可能受到数据流调度策略的显著影响。本实验将通过对比不同存储层次配置下的性能表现，分析多级存储层次与数据流调度之间的相互作用，为处理器架构设计提供实证依据。

其次，实验将重点分析本工作提出的融合数据流框架相较于现有方法的优势。通过与传统数据流调度方法进行对比，实验将从计算效率、内存访问优化和硬件资源利用率等多个维度，量化评估本框架的性能提升。特别地，实验将深入分析框架中创新的以内存为中心的调度表示和分层融合算子策略对整体性能的贡献。

最后，实验将验证本数据流框架在不同硬件架构上的适用性。通过选取具有代表性的三种硬件平台（Cambricon-Acc、TPU-like 和 Tesla-NPU-like），实验将评估框架在不同内存容量、带宽和计算单元配置下的性能表现。这一评估不仅能

表 3.3 不同目标硬件规格

参数	Cambricon-Acc	TPU-like	Tesla-NPU-like
空间展开 (1024 MACs)	Ci 64 Co 64	Ci 32 Co 32	Ci 32 Wo 8 Ho 4
MAC 寄存器	W&O: 2B	W: 128B; O: 1KB	W: 1B; O: 2B
L0 缓存大小	I&O: 64KB; W: 768KB	I&O&W: 64KB	W: 64KB; I&O: 64KB
L1 缓存大小	I&O&W: 2MB	I&O&W: 2MB	W: 1MB; I&O: 1MB

够验证框架的普适性，还能为不同硬件平台上的优化策略提供指导。

3.5.2 实验环境

1. 评估准确性验证

为确保以存储为中心的融合机制的建模精度，本研究采用实际流片芯片 Cambricon-Acc 对 FusionFrame 的评估准确性进行了系统性验证。该验证基于章节 3.4.4 中所述的性能评估方法，通过多种完整 DNN 的端到端性能测试来评估框架的准确性。具体而言，Cambricon-Acc 采用 65nm TSMC 工艺制造，芯片面积为 56.24 mm^2 ，其架构特征表现为：通过类似 SIMD 的指令在暂存器内存上执行 DNN 计算，这种计算模式可被建模为映射到 L0 缓存的最内层循环。为建立准确的验证环境，本研究将 FusionFrame 的硬件参数与 Cambricon-Acc 的规格进行了精确对齐（如表 3.3 所示）。验证结果表明，FusionFrame 的性能评估与实际测量值之间的误差控制在 6% 以内（如图 3.5 所示），这一精度水平满足后续实验的要求。值得注意的是，误差主要来源于 DRAM 访问建模的局限性，具体表现为 DRAM 访问粒度差异以及小分块数据访问时的固定开销等非线性因素。

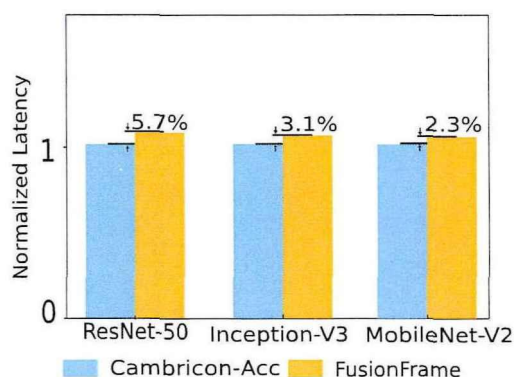


图 3.5 针对 Cambricon-Acc 的验证结果

本工作选择了 ResNet50^[4] 中具有不同参数的四个相同的残差块作为基准测试。每个选定工作负载的总权重和激活大小如图 3.6a 所示。这些工作负载从激活主导（大激活和小权重）到权重主导（小激活和大权重），激活大小逐渐减小，权重大小逐渐增加。图 3.6b 提供了每个选定残差块（表示为 *stage*）的详细信息，描绘了残差架构和实验中使用的相应参数。不同的阶段共享相同的网络架构，但在每

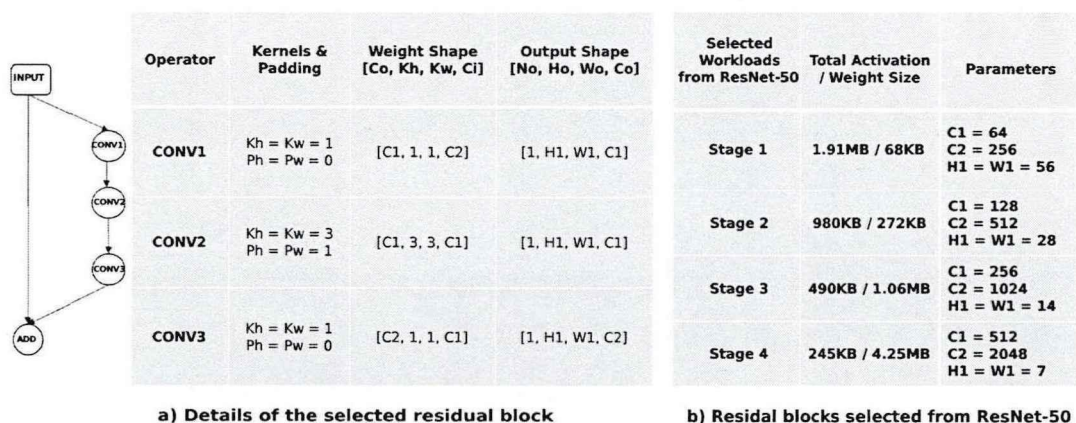


图 3.6 选定的具有相同架构和不同参数的 ResNet-50 残差块工作负载的详细说明

个卷积的特征图大小和输出通道上有所不同。为了评估 FusionFrame 的整体网络性能和适用性，本工作选择了三种具有不同架构的 DNN，并在表 3.3 中列出的三种不同硬件平台上进行了测试。选定的 DNN 包括 ResNet50^[4]、MobileNetV2^[56] 和 InceptionV3^[77]。

2. 硬件平台

本工作针对三种硬件平台，即 Cambricon-Acc^[59]、TPU-like^[19] 和 Tesla-NPU-like^[78]。这些架构在不同内存级别之间的 MAC 数量和带宽上保持一致，以进行公平比较，并保留其独特的片上内存容量、内存级别和空间展开参数。本工作将数据宽度设置为 8 位，DRAM 访问带宽设置为 64 位/周期，片上内存带宽设置为 DRAM 访问带宽的两倍，以模拟片上和片外通信瓶颈。详细的硬件规格如表 3.3 所示。

3.5.3 L1 缓存对不同融合数据流和分块维度的影响实验

本实验旨在探究处理器芯片的多级存储层次是否必然带来最优性能。尽管多级存储层次被广泛认为是提升处理器性能的关键设计，但其实际效果可能受到数据流调度策略的显著影响。通过构建具有两级片上内存的加速器架构，实验将系统研究 L1 缓存对不同融合数据流和分块策略的影响机制，重点分析多级存储层次与数据流调度之间的相互作用，为处理器架构设计提供实证依据。

L1 缓存对融合数据流的性能具有显著影响，其效果随分块维度的不同而呈现差异化特征。许多现代加速器设计了 L1 缓存^[19,21,78]以提高 DNN 工作负载的数据局部性和延迟。尽管 L1 缓存和其他内存层次结构的优势已在单算子级实验中得到验证^[61-62]，但其对不同融合数据流的影响机制仍缺乏系统性研究。本研究以图 3.6b 中详细描述的阶段 2 为目标融合工作负载，以 Cambricon-Acc 为目标硬件平台。通过基于原始 Cambricon-Acc 单层级存储层次中添加了一个 2MB 的 L1 缓存，构建具有两级片上内存层次结构的加速器（分别记为 *Acc-1L* 和 *Acc-2L*），

深入探讨了 L1 缓存在不同分块策略下的影响机制。

1. 在 Ho/Wo 维度上分块的探索

在 Ho/Wo 维度分块方面, L1 缓存显著影响了融合数据流内的数据搬运模式。本工作首先在 *Ho* 和 *Wo* 维度上对 *Acc-1L* 和 *Acc-2L* 进行分块, 以研究内存层次结构对融合数据流内数据搬运的影响。将此工作负载映射到 *Acc-2L* 的融合策略如图 3.7a 所示。该策略将所有算子融合到一个组中, 因此 L1 缓存仅影响组的输入、权重和输出。最初, 本工作独立评估输入数据的影响, 然后将分析扩展到输出和权重。为此, 本工作绕过 L1 缓存, 将输出和权重直接在 L0 缓存和 DRAM 之间搬运。

在此探索中, 本工作将分块大小从 1x1 扫描到 6x6, 以研究 L1 缓存如何影响输入数据搬运。如图 3.7 所示, 一次加载一批输入到 *Acc-2L* 的 L1 缓存中, 其中输入在 *Ho* 和 *Wo* 维度上分块并映射到 L1 缓存。相比之下, 对于 *Acc-1L*, 输入在 DRAM 上分块, 由于从 DRAM 获取重叠数据, 导致 DRAM 访问显著增加。此外, 对于 *Acc-2L*, L1 缓存访问发生是因为分块输入从 L1 缓存获取到 L0 缓存。此访问量是分块输入的总大小, 包括 *Ho* 和 *Wo* 维度分块引入的重叠数据。随着分块大小的增加, DRAM 访问开销和 L1 缓存访问减少, 这是由于分块数量减少, 从而减少了分块之间的重叠数据搬运。

此外, *Acc-2L* 的总数据搬运量大于 *Acc-1L*, 但在分块大小小于 4x4 时显示出较低的延迟。这是因为片上内存带宽大于 DRAM 访问带宽。然而, 随着分块大小的增加, 重叠数据搬运减少, 通过 L1 缓存加载输入的好处减少。因此, 需要根据可用的片上内存带宽仔细考虑输入是否应利用 L1 缓存。对于通过 L1 缓存的输入, 权衡在于减少 DRAM 访问和增加片上内存访问之间。至于输出和权重, 它们不会受到重叠数据搬运问题的影响, 因此通过 L1 缓存传递它们无法减少 DRAM 访问, 但会增加片上内存访问。因此, 对于延迟的最佳解决方案是绕过 L1 缓存传递输出和权重。

2. 在 Co 维度上分块的探索

本研究发现, 在 Co 维度分块场景下, L1 缓存对中间数据存储具有显著的优化效果。具体而言, 如图 3.7d 所示, 工作负载被划分为三个算子组, 每个组在 Co 维度上进行分块处理, 这是由于权重大小超出了 L1 缓存容量。值得注意的是, 在 Co 维度分块需要在每个卷积算子后执行数据拼接操作 (详见第 3.4.2 节), 而维度无关算子 (如 *ADD*) 则可与前置卷积算子实现融合。

实验结果表明, 对于包含 *CONV1* 的第一组算子, *Acc-1L* 和 *Acc-2L* 的关键差异体现在输出数据是否在 L1 缓存中完成拼接。通过将 *CONV1* 的输出存储在 L1 缓存中, *Acc-2L* 的 DRAM 访问量减少了 21.5%。然而, 由于输入数据未在 *Ho* 和 *Wo* 维度上进行分块 (如实验 3.5.3 所述), 通过 L1 缓存加载 *CONV1* 的

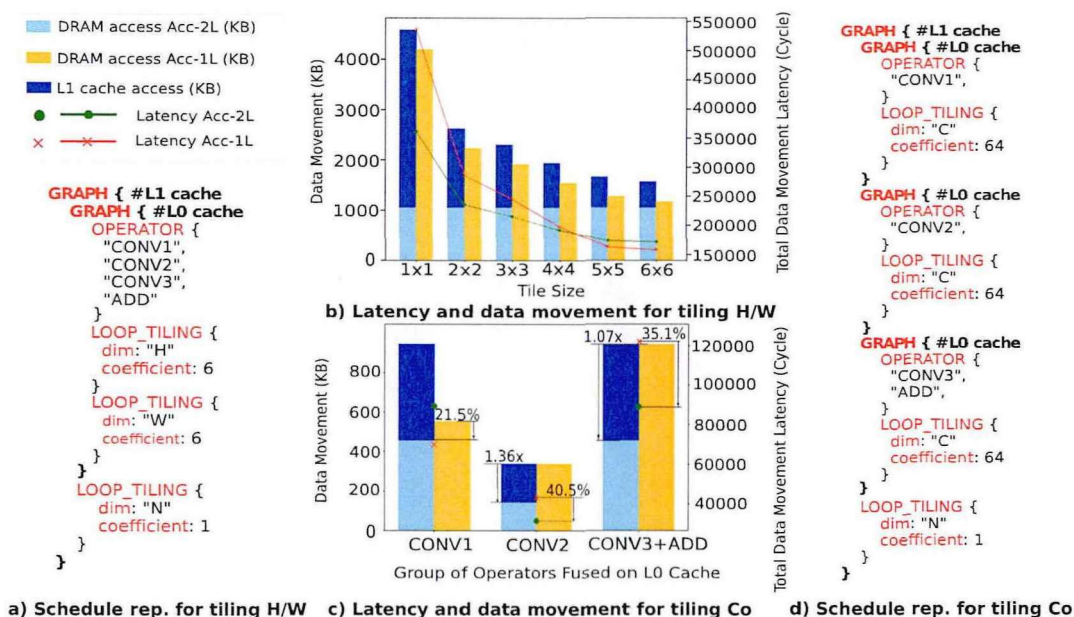


图 3.7 基于图中的调度表示，数据搬运量和延迟的实验结果

注： H_o 和 W_o 分块的表示显示所有算子在 L0 缓存上融合，最大可行分块大小为 6×6 ，批次维度映射到 L1 缓存

输入会导致总数据搬运量和延迟的增加。对于包含 $CONV2$ 和 $CONV3 + ADD$ 的算子组，尽管 Acc-2L 和 Acc-1L 的总数据搬运量相同，但得益于 L1 缓存对输入和输出中间数据的存储优化，Acc-2L 的 DRAM 访问分别减少了 1.36 倍和 1.07 倍，总数据搬运延迟分别降低了 40.5% 和 35.1%。

基于上述实验结果，本研究得出以下结论：首先，对于融合算子组，是否通过 L1 缓存加载输入数据需要综合考虑片上内存带宽以及 H_o 和 W_o 维度分块带来的重叠数据量；其次，在输出分块方案下，输出和权重数据应直接绕过 L1 缓存以避免额外的数据搬运开销；最后，L1 缓存在存储算子组间中间数据方面具有显著优势，可有效减少 DRAM 访问和数据搬运延迟。FusionFrame 框架通过其灵活的以存储为中心的表示方法和融合机制，为各类融合数据流和分块策略的分析提供了有力支持。

3.5.4 不同 SotA 融合数据流的对比实验

本实验旨在评估不同融合数据流策略在深度神经网络加速器上的性能表现，重点分析数据搬运模式和延迟特性。通过对比逐层、DeFiNES-like 和 FusionFrame 三种融合策略，探究多级内存层次结构对数据流优化的影响机制，为 FusionFrame 的调度策略优化提供实证依据。

在本案例研究中，本工作比较了三种类型的融合数据流：逐层、基于 FusionFrame 的以存储为中心和 DeFiNES^[67] 和 ACCO^[70] 中使用的深度优先 (Depth-

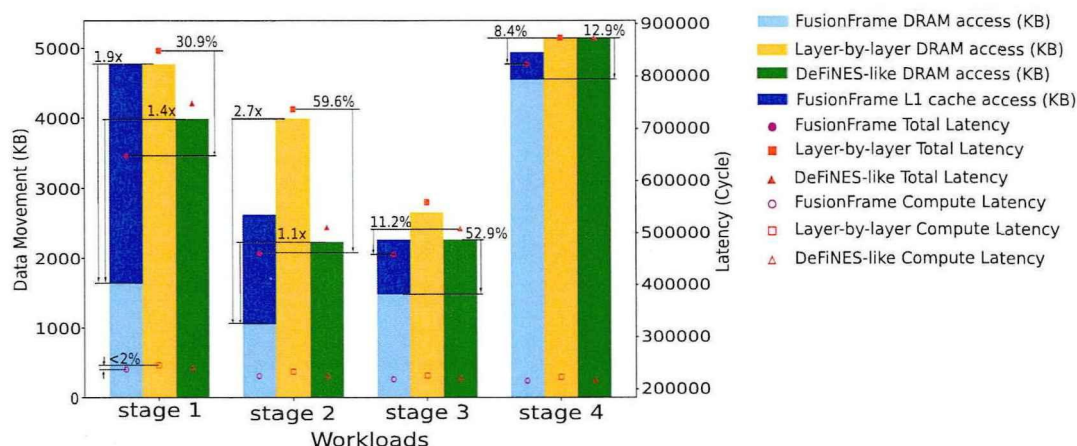


图 3.8 三种融合数据流：逐层、DeFiNES-like 和 FusionFrame 的性能对比

注：实验基于 Acc-2L 架构，对于 stage 1 和 stage 2， H_o 和 W_o 分块为 2×2 ；对于 stage 3 和 stage 4， C_o 分块为 64

First) 融合数据流。本工作在 FusionFrame 中实现了 DeFiNES 中使用的融合数据流，即 DeFiNES-like，利用 FusionFrame 的以存储为中心的抽象。此实现保留了 DeFiNES 融合数据流的关键特征：输入和输出数据优先使用 L0 缓存，仅在必要时溢出到更高内存级别，前提是权重适合 L0 缓存。此数据流的执行过程涉及直接从 DRAM 加载和存储所有类型的数据，绕过 L1 缓存，而权重在整个计算过程中保留在 L0 缓存中。相比之下，逐层数据流逐个完全评估每个算子，中间特征图在算子之间传递，使用它们适合的最低内存级别。FusionFrame 采用在实验 3.5.3 中识别的性能更好的融合数据流，该数据流涉及通过 L1 缓存加载融合堆栈的输入数据以进行 H_o 和 W_o 分块，其他类型的数据直接在 L0 缓存和 DRAM 之间搬运。不同组之间的中间特征图通过 L1 缓存传递。

本工作使用图 3.6 中引入的四个残差块在 Acc-2L 架构上评估这些融合数据流。在此探索中，本工作将不同的融合机制与相同的分块大小对齐，以说明每个融合机制的内存访问模式。结果如图 3.8 所示，详细说明了不同的数据流。不同数据流的计算延迟变化小于 2%，因为融合模式对计算影响不大。因此，本工作的分析主要集中在数据搬运和总延迟上。对于前两个阶段，FusionFrame 和 DeFiNES-like 在 L0 缓存中融合所有算子，并进行 H_o 和 W_o 分块，因为总权重适合 L0 缓存容量。FusionFrame 数据流分别减少了 1.9 倍和 2.7 倍的 DRAM 访问，并分别减少了 30.9% 和 59.6% 的延迟，相比于逐层数据流。在这些情况下，FusionFrame 和 DeFiNES-like 融合数据流采用类似的策略，除了 FusionFrame 受益于通过 L1 缓存加载输入，进一步减少了 1.4 倍和 1.1 倍的 DRAM 访问，如实验 3.5.3 所述。在阶段 3 和 4 中，融合卷积的权重大小超过了 L0 缓存容量，导致 FusionFrame 和 DeFiNES-like 的行为不同。FusionFrame 在 L1 缓存中缓存融合组之间的中间数据，而 DeFiNES-like 在权重大小超过 L0 缓存容量时无法融合

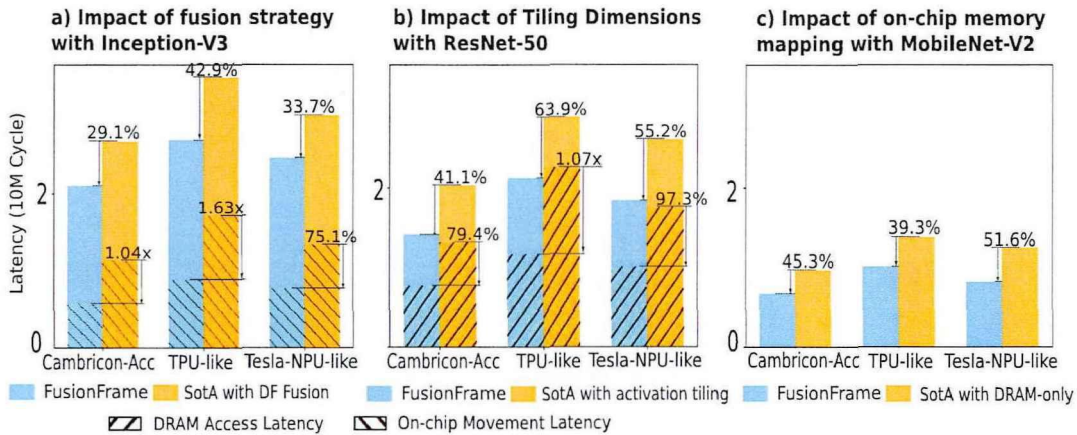


图 3.9 不同调度维度对性能影响的 SotA 比较

卷积。因此，FusionFrame 在这两个案例中分别减少了 9.8% 的延迟和 32.9% 的平均 DRAM 访问。在阶段 3 中，DeFiNES-like 数据流融合了 *ADD* 和 *CONV3*，因为 *CONV3* 的权重适合 L0 缓存容量。然而，在阶段 4 中，*CONV3* 的权重超过了 L0 缓存容量，无法与 *ADD* 融合，导致总数据搬运比 FusionFrame 增加了 12.9%。

总体而言，FusionFrame 通过优化 L1 缓存使用策略，在相同融合深度下平均减少了 80.2% 的 DRAM 访问和 14.3% 的延迟。这一性能优势主要源于两个关键因素：通过 L1 缓存加载输入以最小化 DRAM 访问，以及在 L1 缓存中缓存算子组间的中间数据。这些发现强调了在多级内存层次结构硬件上采用灵活融合策略的重要性。

3.5.5 针对调度空间的不同维度与 SotA 工作的对比实验

本实验旨在系统验证 FusionFrame 框架在不同硬件架构上的通用性和适应性。通过选取具有代表性的三种硬件平台（Cambricon-Acc、TPU-like 和 Tesla-NPU-like），实验全面评估了框架在多样化内存容量、带宽和计算单元配置下的性能表现。这一评估不仅验证了框架的跨平台适用性，还为不同硬件架构的优化策略提供了实证依据。本实验从算子融合、循环分块和硬件映射三个关键维度，系统性地评估了多种 DNN 加速器平台上的性能影响。通过对比分析，实验揭示了当前 SotA 方法在调度优化方面的局限性，充分验证了 FusionFrame 框架在性能优化方面的显著优势。

在算子融合维度，现有方法普遍存在硬件资源利用率不足的问题。以 DeFiNES^[67]、ACCO^[70] 和 ConvFusion^[64,69] 为代表的 SotA 方法采用深度优先融合策略，依赖用户提供的固定分块大小，导致融合行为次优。通过在 *Ho* 和 *Wo* 维度上为每个融合堆栈实现 *Depth-First* 策略，并使用 DeFiNES^[67] 建议的 6x6 分块大小，与 FusionFrame 使用 Inception-V3^[77] 进行比较，实验结果表明，DF

策略在 Inception-V3 工作负载下, 其片上内存访问量平均超过 FusionFrame 1.14 倍 (图 3.9a)。这种性能差距主要源于用户提供的分块大小与硬件规格不匹配, 导致中间数据被错误地分配在 L1 缓存上, 引发冗余的片上数据搬运。得益于硬件感知的融合策略, FusionFrame 能够根据硬件规格自动确定最优分块大小, 其总延迟平均比 Depth-First) 数据流低 35.2%。

在循环分块维度, 现有方法存在显著的权重大小限制问题。Genetic-A^[69]、ACCO^[70] 和其他方法^[65,79] 等 SotA 方法仅关注激活数据分块, 其性能受限于大 L0 缓存的需求。通过 ResNet-50 工作负载的实验对比发现, 仅激活分块数据流的 DRAM 访问延迟平均超过 FusionFrame 52.7% (图 3.9b)。这一问题在 TPU-like 架构中尤为突出, 其 DRAM 访问延迟开销达到 1.07 倍, 总延迟开销高达 63.9%。这是因为仅激活分块数据流以逐层方式处理 ResNet-50 中超出 L0 权重缓存大小的层, 使得 DRAM 访问在总延迟中占主导地位, 尤其是对于 L0 缓存较小的架构, 如 TPU-like 架构。FusionFrame 通过完整的输出分块方案, 有效解决了权重大小限制问题, 显著降低了 DRAM 访问开销。

在硬件映射维度, 现有方法普遍忽视了片上内存访问的优化。ConvFusion^[64] 和 Optimus^[65] 等 SotA 工作虽然致力于减少 DRAM 访问, 但未能充分考虑片上内存访问的影响。通过 MobileNet-V2^[56] 工作负载的实验评估表明, FusionFrame 在平衡片上和片外访问方面表现出色, 其平均延迟比仅优化 DRAM 访问的融合数据流提高了 45.4% (图 3.9c)。

此外, 实验还揭示了不同硬件架构的性能差异。Cambricon-Acc 凭借其较大的片上内存容量, 在支持融合数据流方面表现出最优的延迟性能, 其次是 Tesla-NPU-like 架构, 最后是 TPU-like 架构。这一性能排序充分说明了片上内存容量对融合数据流优化的重要性。

本案例研究通过系统性评估, 揭示了当前 SotA 方法在算子融合、循环分块和硬件映射方面的局限性。实验结果表明, FusionFrame 通过优化片上内存访问和分块大小, 在降低延迟方面显著优于现有方法。表 3.4 详细比较了相关 SotA 调度框架的特性。

表 3.4 相关 SotA 调度框架比较

框架	完整分块 维度支持	多级内存 映射	硬件感知 融合
DeFiNES ^[67]	×	✓	×
Genetic-A ^[69]	×	✓	×
ACCO ^[70]	×	✓	×
ConvFusion ^[64]	×	×	×
FusionFrame (ours)	✓	✓	✓

3.6 讨论

多核架构的建模。本章提出的框架主要针对单核处理器架构进行建模。然而，随着任务规模扩展至多核场景，需进一步对多核环境下的开销进行精确建模，例如将拆分块映射到不同处理器核时可能带来的额外开销。此外，系统可扩展性亦是一个关键问题，特别是在多机多卡场景下，卡间数据通信的建模对于实现更可扩展的性能评估至关重要。进一步地，在现有调度优化原语的基础上，支持更多种类的性能优化方法，如指令并发流水等，将有助于进一步提升系统性能。

Transformer 类模型的性能评估。本章的调度框架主要针对卷积神经网络 (CNN) 相关的网络进行性能评估，其显著特点是滑窗类算子，如卷积和池化操作。然而，随着大语言模型（如 Transformer）的广泛应用，针对这类模型中的矩阵乘法和层归一化（layernorm）等算子的融合可能性及其性能收益的探索显得尤为重要。这不仅有助于扩展框架的应用范围，还能为 Transformer 类模型的优化提供新的视角。

兼容现有算法框架的生态。尽管高级机器学习框架（如 TensorFlow^[23] 和 PyTorch^[49]）提供了多样化的 DNN 模型描述，但现有的调度框架^[40,60,67,80]通常采用自定义的网络模型描述格式和硬件抽象描述格式，这使得它们无法直接处理来自 TensorFlow 和 PyTorch 的模型。此外，这种自定义格式还导致当前框架的调度结果无法在其他框架中进行评估，限制了不同调度框架能力的协同使用。因此，可以进一步探索统一的中间表示，找到一种兼容高级机器学习框架的网络模型描述方式，以及一种调度结果的统一表示，将有助于接入高级机器学习框架的生态，并充分调动不同调度框架的能力。

3.7 本章小结

本章提出了一种针对深度学习计算图融合的调度框架 FusionFrame，并详细介绍了其设计与实现，重点阐述了其在优化 DNN 调度方面的创新性和有效性。首先，本章节从算子融合、循环分块和硬件映射三个维度刻画了 DNN 调度的设计空间，提出了一种以存储为中心的表示方法，该方法能够高效地表达和探索不同的融合数据流优化机会。其次，本章节引入了一个统一的分析成本模型，该模型通过迭代探索调度空间，生成以存储为中心的表示，并准确评估其性能，从而实现了对内存占用和延迟等关键性能指标的高效估计。

通过全面的案例研究，本章节进一步验证了 FusionFrame 在不同融合数据流下的优越性能。案例研究分析了在不同调度维度下的权衡，揭示了 FusionFrame 在减少 DRAM 访问和降低延迟方面的显著优势。这些研究结果不仅为优化 DNN

调度提供了新的视角，也为未来在多核架构和 Transformer 类模型中的应用奠定了基础。

在技术实现方面，FusionFrame 通过其创新的调度空间表示和分析成本模型，显著提升了 DNN 调度的优化效果。具体而言，FusionFrame 通过灵活的以存储为中心的表示方法，能够有效地捕捉和优化不同硬件平台上的内存层次结构和数据搬运成本。这种表示方法不仅提高了调度优化的精度，还显著减少了优化过程中的计算开销。此外，FusionFrame 的统一分析成本模型通过迭代探索调度空间，能够生成多种优化方案，并通过精确的性能评估，选择最优的调度策略。这种模型不仅提高了调度优化的效率，还为未来在多核架构和 Transformer 类模型的应用提供了坚实的技术支撑。

在应用前景方面，FusionFrame 的优化效果不仅体现在单核处理器架构上，还为未来多核架构和分布式计算环境中的性能优化提供了新的思路。尽管当前 FusionFrame 主要针对单核处理器架构进行建模，但其灵活的调度空间表示和分析成本模型为未来扩展到多核和分布式环境提供了理论和技术基础。

在理论贡献方面，FusionFrame 的研究成果不仅丰富了 DNN 调度的理论体系，还为实际应用中的性能优化提供了有力的工具和方法。通过全面的案例研究，FusionFrame 展示了其在减少 DRAM 访问和降低延迟方面的显著优势，为优化 DNN 调度提供了新的视角。这些研究结果不仅为学术界提供了新的研究方向，还为工业界提供了实用的优化工具。

综上所述，FusionFrame 通过其创新的调度空间表示和分析成本模型，显著提升了 DNN 调度的优化效果，为智能计算系统的进一步发展提供了坚实的技术支撑。本章的研究成果不仅丰富了 DNN 调度的理论体系，也为实际应用中的性能优化提供了有力的工具和方法。

第4章 面向 SIMD 架构智能处理器的图融合指令生成系统

4.1 引言

随着深度学习模型的不断演进,计算图的复杂度和规模呈现指数级增长,这对指令生成系统提出了严峻挑战。特别是在面向智能处理器的场景下,如何高效地生成融合指令并优化计算性能,已成为深度学习编译器领域的关键问题。当前,深度学习计算图的优化面临两个核心挑战:其一,在深度融合场景下,传统地址推导方法难以有效处理复杂算子间的数据依赖关系,导致计算效率低下;其二,在神经网络架构中,上采样操作作为恢复空间分辨率的关键组件,其特有的计算模式使得传统优化方法难以发挥效能。这两个挑战共同制约了深度学习模型的性能优化空间,亟需新的技术突破。

在地址推导方面,传统方法主要采用标量中间表示(IR)进行地址推导,这种方法在简单算子融合场景下尚可应对,但在处理深度融合时面临显著局限性。具体而言,传统方法难以有效处理滑窗类算子(如卷积、池化)的地址推导,也无法应对冗余输入爆炸问题。现有工作如 AKG^[81]、Roller^[82]等通过限制融合深度来规避冗余输入爆炸的问题,但这种方法牺牲了深度融合带来的性能提升潜力。此外,传统方法在处理带有控制流的算子(如 pad 填补算子)时,难以进行有效的形状推导,限制了其在深度学习场景中的应用。

在上采样操作优化方面,传统方法主要针对下采样操作设计,难以有效应对上采样操作中特有的数据搬运和计算模式。上采样操作中大量的零值填充和无效计算严重影响了整体性能。现有存储层次优化方案^[40,67]虽然通过优先使用高层级存储来减少存储层级间的数据搬运,但仍存在显著局限性:一方面,高层级存储的芯片面积代价较高,难以大规模扩展;另一方面,有限的存储空间往往成为性能瓶颈,特别是在处理大规模特征图时尤为明显。这些问题导致上采样操作的计算开销往往占据整个网络推理时间的显著比例,成为制约模型性能的关键瓶颈。

针对上述挑战,本章提出了一种系统性的解决方案。首先,本章提出了一种基于向量描述的地址推导系统,通过引入新的数据结构和方法论,实现了更精确、更高效的地址推导,为深度融合场景下的指令生成提供了有力支持。其次,本章设计了一种面向上采样结构的网络优化方案,通过深入分析上采样操作的计算特征,设计针对性的优化策略,显著提升了上采样结构的执行效率。通过系统化的实验验证,本章提出的方法在多个基准测试集上均取得了显著的性能提

升，为深度学习模型的部署和优化提供了有力支撑。

4.2 相关背景和研究动机

4.2.1 深度融合下的地址推导问题

传统指令生成系统在地址推导方面存在显著局限性，难以满足深度融合场景的需求。传统方法通常采用标量 IR 作为中间表示，通过标量推导公式或算子计算公式进行地址推导，这种基于标量的推导方式在深度学习场景下面临诸多挑战。

首先，传统方法在算子定义上的灵活性导致了分析过程的复杂性。由于 CPU 等通用处理器需要支持多样化的算子类型，传统编译器采用了高度灵活的算子定义方式。然而，这种灵活性使得地址推导过程变得复杂，显著增加了分析时间和编译时间。特别是在处理带有控制流的算子（如 pad 填补算子）时，传统方法难以进行有效的形状推导，限制了其在深度学习场景中的应用。其次，传统方法在算子融合支持方面存在明显不足。现有的地址推导机制主要针对点对点操作（Pointwise）类的前后算子融合方案，即点对点的运算方式，无法支持更复杂的深度融合场景如卷积、池化等滑窗类的操作。在深度学习计算图中，算子之间往往存在复杂的依赖关系和数据流动模式，传统的标量推导方法难以准确捕捉这些特征，导致无法实现高效的深度融合优化。最后，传统方法在张量处理器指令映射方面存在适配性问题。由于传统方法采用标量 IR 到张量指令的转换过程，这种间接的映射方式难以充分利用 MLU 等张量处理器的硬件特性。特别是在处理大规模张量计算时，传统方法难以实现高效的指令生成和优化，限制了硬件性能的充分发挥。

综上所述，传统指令生成系统在地址推导方面存在诸多不足，难以满足深度融合场景的需求。为了克服这些局限性，本章提出了一种新的基于向量描述的地址推导方法，以支持更复杂的算子融合模式，实现高效的时间片间数据复用，并充分利用张量处理器的硬件特性。这些改进将为深度学习计算图的优化提供更强大的技术支持。

4.2.2 深度融合下的冗余输入爆炸问题

深度学习计算图中的冗余输入爆炸问题是制约算子深度融合性能提升的关键瓶颈。该问题在包含滑窗类算子（如卷积、池化等）的深度融合场景中尤为显著，主要表现为相邻时间片间数据重叠导致的冗余计算和访存开销急剧增加。

从理论层面分析，给定一个包含 N 个算子的计算图 $G = \{O_1, O_2, \dots, O_n\}$ ，其

中每个算子 O_i 的输入数据块

$$D_i = \{d_1, d_2, \dots, d_k\}, \quad d_j \in \mathbb{R}^{H \times W \times C} \quad (4.1)$$

在时间片划分策略 $T = \{t_1, t_2, \dots, t_m\}$ 下, 相邻时间片之间的冗余输入可形式化定义为:

$$R(t_j, t_{j+1}) = D(t_j) \cap D(t_{j+1}) \quad (4.2)$$

随着融合深度 $F \in \mathbb{N}^+$ 的增加, 冗余增长率为:

$$G(F) = \frac{|R(t_j, t_{j+1})|}{|D(t_j)|} \quad (4.3)$$

该增长率呈指数增长, 这构成了优化问题的核心挑战。传统方法在处理时间片间数据复用方面存在显著缺陷, 缺乏有效的机制来识别和利用相邻时间片之间大量可复用的数据, 这不仅增加了内存带宽压力, 还导致了计算资源的浪费, 严重影响了整体性能。具体而言, 该问题可形式化为以下优化目标:

$$\text{minimize } \sum_{j=1}^{m-1} |R(t_j, t_{j+1})| \quad (4.4)$$

要求在保证计算正确性的前提下, 尽可能实现冗余数据的复用。如图 4.1 所示, 以卷积算子的融合为例, 在高度 (H) 和宽度 (W) 维度进行拆分的情况下, 相邻两个时间片 (temporal tile) 之间存在显著的数据重叠。对于每个卷积算子来说, 时间片 1 的输入数据 (蓝色) 和时间片 2 的输入数据 (橙色) 存在大量的重叠 (黑色框部分)。特别地, 对于滑窗类算子, $R(t_j, t_{j+1})$ 与卷积核大小 K 呈正相关, 且在多维数据 (H,W,C) 情况下, 冗余增长具有维度相关性。随着融合深度的增加, 每个算子需要处理的数据块数量呈爆炸性增长, 这种冗余输入的增加不仅浪费了宝贵的计算资源, 还可能导致内存带宽的瓶颈, 从而显著降低整体计算效率。此外, 冗余输入的增加还使得地址推导和内存管理变得更加复杂, 因为需要精确地跟踪和管理每个数据块的访问模式和生命周期。

为了应对这一挑战, 现有工作如 AKG^[81]、Roller^[82] 和 Chimera^[83] 等通过限制融合深度来规避冗余输入爆炸的问题。然而, 这些工作在处理任意融合深度情况下的冗余输入爆炸问题时, 效果有限。为了从根本上解决这一问题, 本章在基于向量的形状推导方法的基础上, 针对不同的数据复用模式, 提出了一种数据复用策略, 有效地避免了冗余数据在算子融合过程中的传播放大现象。这一创新方法显著提升了算子融合场景下的性能, 为解决冗余输入爆炸问题提供了强有力的技术支持。

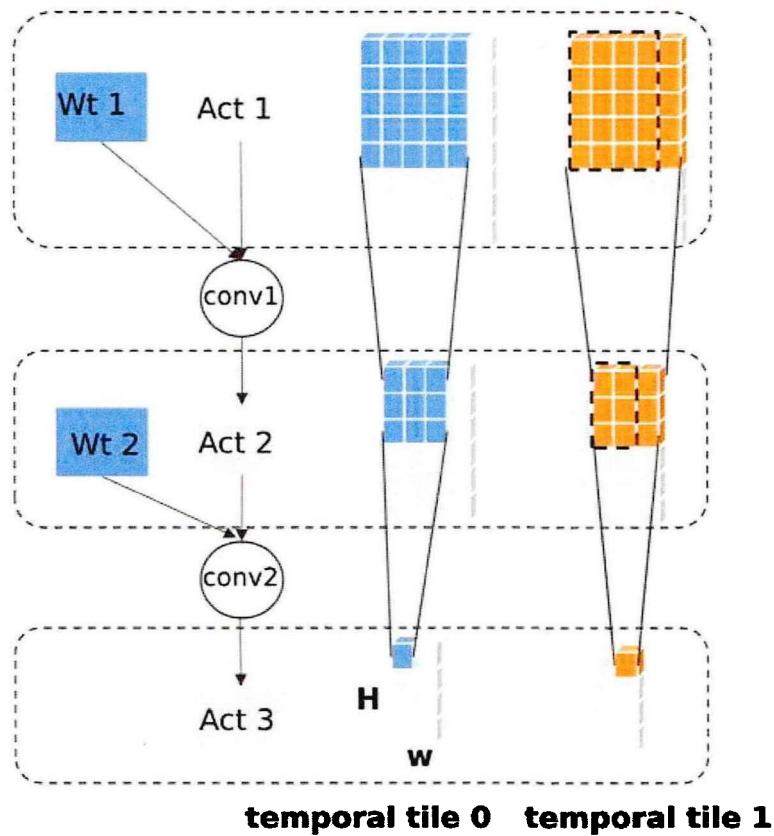


图 4.1 卷积算子融合时，相邻时间片处理的数据

注：黑色框为重复部分，随着融合深度的增加，重复部分数据量爆炸性增长

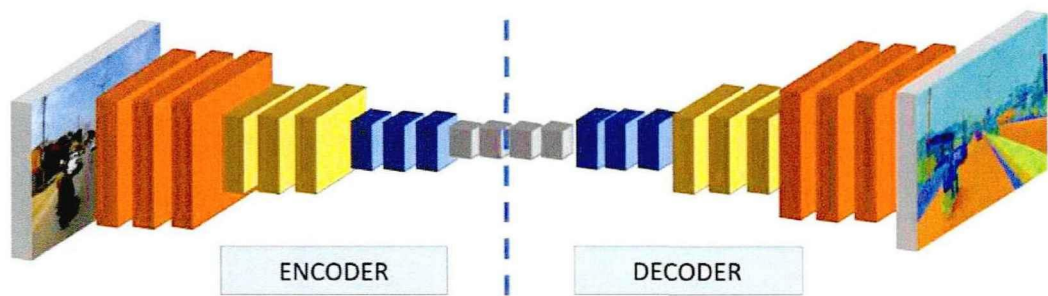
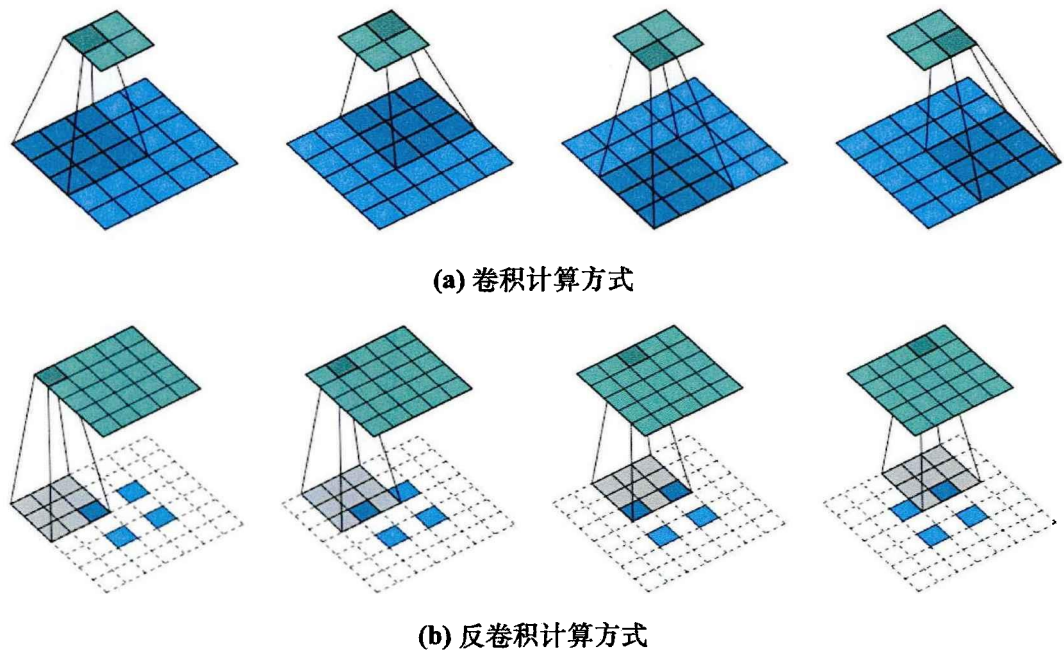


图 4.2 编码器-解码器结构^[84]

4.2.3 神经网络中上采样操作的性能优化问题

上采样操作作为卷积神经网络中恢复空间分辨率的关键组件，其性能优化面临独特的挑战。在卷积神经网络（CNN）架构中，下采样操作（如卷积和池化）作为核心计算单元，主要用于提取特征并降低特征图的空间维度。然而，在图像分割、超分辨率重建等计算机视觉任务中，为了恢复高分辨率特征图的空间信息，上采样操作成为不可或缺的关键组件。

以经典的编码器-解码器（Encoder-Decoder）结构为例，如图 4.2 所示，该架构首先通过编码器对输入特征图进行下采样以提取高层语义特征，随后在解码



注：上述例子中特征图尺寸 5×5 ，计算核尺寸 3×3 ，步幅为 2，填充为 0

图 4.3 卷积和反卷积计算方式的对比^[85]

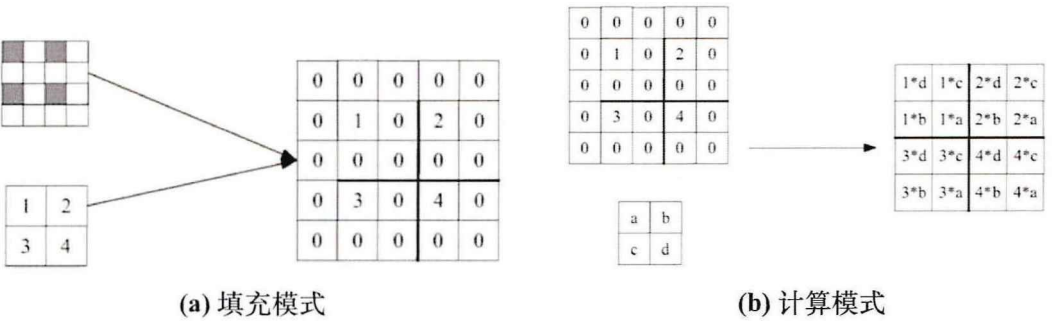


图 4.4 上采样操作中的填充和计算模式

器部分通过一系列上采样操作逐步恢复特征图的空间分辨率。这种结构在保持语义信息的同时实现空间分辨率的提升，是现代计算机视觉任务中的基础架构。

上采样操作面临的核心挑战源于其与下采样操作在计算模式上的本质差异。从计算范式来看，上采样操作旨在将低分辨率特征图映射到高分辨率空间，主要通过反卷积（Transposed Convolution）和反池化（Unpooling）等操作实现。以反卷积为例，其计算机制与标准卷积形成鲜明对比：标准卷积通过卷积核在输入特征图上滑动并计算加权和，而反卷积则采用逆向映射策略，即在输出特征图上进行卷积核操作，并需要预先对输入数据进行大量零值填充。如图 4.3 所示，这种计算模式导致上采样操作需要额外的填充步骤，显著增加了计算复杂度。

上采样操作中的填充和计算模式带来了显著的运行效率挑战。如图 4.4 所示，图 4.4a 展示了上采样操作中典型的零值填充过程，而图 4.4b 则展示了填充后数

据的卷积计算过程。从计算特征来看，填充操作需要执行大量非连续的数据搬运操作以插入零值，这不仅增加了数据移动的开销，还导致后续卷积计算中大量无效的零值参与运算。具体而言，卷积核在与填充区域进行卷积时，需要执行大量与零值的乘法运算，这些运算既不改变计算结果，又消耗了宝贵的计算资源。这种计算模式造成了严重的算力浪费，最终导致上采样操作的整体性能显著下降。

从形式化定义的角度，上采样操作的计算过程可以分解为两个关键阶段：数据填充阶段和卷积计算阶段。给定输入特征图 $X \in \mathbb{R}^{H \times W \times C}$ ，上采样因子 $s \in \mathbb{N}^+$ ，卷积核 $K \in \mathbb{R}^{k \times k \times C \times C'}$ ，其中 H, W, C 分别表示高度、宽度和通道数， C' 为输出通道数。上采样操作 $\mathcal{U}$ 可形式化定义为：

$$Y = \mathcal{U}(X, K, s) = \mathcal{P}(X, s) * K \quad (4.5)$$

其中 $\mathcal{U} : \mathbb{R}^{H \times W \times C} \times \mathbb{R}^{k \times k \times C \times C'} \times \mathbb{N}^+ \rightarrow \mathbb{R}^{sH \times sW \times C'}$ 表示上采样映射函数， $\mathcal{P}$ 表示填充操作， $*$ 表示卷积运算。该定义清晰地刻画了上采样操作从输入到输出的数学映射关系。

性能优化需要考虑两个关键因素：

1. 补0开销：填充操作 $\mathcal{P}$ 引入的零值元素数量 N_{pad} 可表示为：

$$N_{pad} = |\mathcal{P}(X, s)| - |X| \quad (4.6)$$

其中 $|\cdot|$ 表示张量元素总数。补0操作主要带来数据搬运开销，其计算复杂度与填充模式密切相关。

2. 卷积计算量：卷积运算的计算复杂度 C_{conv} 可表示为：

$$C_{conv} = sH \cdot sW \cdot C' \cdot k \cdot k \cdot C \quad (4.7)$$

这反映了实际计算所需的乘加操作数量，其计算复杂度随特征图尺寸和卷积核尺寸呈多项式增长。

因此，性能优化目标可形式化为：

$$\text{minimize } \alpha \cdot N_{pad} + \beta \cdot C_{conv} \quad \text{s.t. } \mathcal{U}(X, K, s) = Y \quad (4.8)$$

其中 α 和 β 分别为补0开销和卷积计算量的权重系数，约束条件 $\mathcal{U}(X, K, s) = Y$ 确保上采样操作的正确性。该优化目标综合考虑了数据搬运和计算两个方面的开销，能够更全面地反映上采样操作的性能特征。

上采样操作的性能优化面临独特的挑战，其核心问题在于传统优化方法难以有效解决。首先，算子融合技术主要针对 DRAM 访存优化，无法有效缓解上采样操作中由于大量零值填充带来的片上数据搬运开销。其次，现有存储层次优化方案^[40,67]虽然通过优先使用高层级存储（如 L0 cache）来减少存储层级间的

数据搬运,但仍存在显著局限性:一方面,高层级存储的芯片面积代价较高,难以大规模扩展;另一方面,有限的存储空间往往成为性能瓶颈,特别是在处理大规模特征图时尤为明显。因此,本章在指令生成层面引入新的优化策略,通过深入分析上采样操作特有的数据搬运模式,设计针对性的优化方案,以突破现有方法的性能限制。

4.3 图融合指令生成地址推导

在深度学习计算图中进行指令优化时,形状推导是一个关键步骤,它涉及如何准确表示和推导数据块在内存中的地址。本节详细介绍了一种针对深度学习工作负载的地址推导方法,该方法特别适用于可以进行深度融合的计算图。

4.3.1 访存地址表达

为了方便表示访存模式以及对访存模式进行推导,本节提出了一种数据结构,可以对数据块的拆分方式以及相应的访存地址进行表达。*Interval* 是用于表示一个循环的访存模式的数据结构,本质上是 *Range* 到 *pattern* 的映射,其中 *key* 是 *Range* (循环变量), *value* 是 *Pattern*。

- **Key:** *Range*, 表示循环中的循环变量。

```
1 struct Range {
2     int min, extent;
3 }
```

- **Value:** *Pattern*, 表示每个 *Range* 对应的地址访问模式。

```
1 // pattern数据结构表示
2 struct Pattern {
3     int min, stride, extent;
4 }
```

例如,一个长度为 14 的循环,按 3 为长度单位拆分,可以被拆分为 5 次循环,如图所示:



图 4.5 将长度为 14 的数据块按照 3 的长度拆成 5 块

使用上述数据结构,用 *Range* 描述循环次数,用 *Pattern* 描述每次循环处理的数据量可以如表格 4.1 所示,例如第 0 次循环处理的数据,是从 0 开始, *stride* 为 1,大小为 3 的数据块。

可以看出,每一次循环的起始地址和步长是成规律变化的,可以规约成表达式,用 *Interval* 的形式可以表示为表格 4.2 中所示。

同时,上述描述针对的是单个维度拆分情况下的访存模式的表达,对于多维

表 4.1 Range 和 Pattern 的对应关系

Range (循环次数)	Pattern (访存模式)
0	[0, 1, 3)
1	[3, 1, 3)
2	[6, 1, 3)
3	[9, 1, 3)
4	[12, 1, 2)

表 4.2 Range 和 Pattern 的表达式抽象

Range (循环次数)	Pattern (访存模式)
[0, 4)	[i * 3, 1, 3)
[4, 1)	[i * 3, 1, 2)

注：其中 i 表示循环变量

算法 4.1 Memory Access Pattern Derivation

Input: Computation Graph G
Output: Memory access patterns for all operations

```
1 Step 1: Select entry operator and split data;;
2 entry_op ← findLastOp(G);
3 split_data ← splitOutputData(entry_op.output);
4 Step 2: Backward traversal and memory access derivation;;
5 back_traverse(entry_op);
6 Step 3: Data reuse calculation;;
7 foreach data block b in split_data do
8   | reuse_range ← calculateReuse(b, b.next);
9   | compute_range ← calculateCompute(b, b.next);
10 end
11 Step 4: Read access derivation and final access calculation;;
12 foreach data d in G do
13   | read_pattern ← deriveReadAccess(d);
14   | final_access ← calculateFinalAccess(d, read_pattern);
15 end
```

数据块，本工作在上述数据结构的基础上进一步封装出 **Region** 这一数据结构用于表示数据在多个维度上的访存模式。**Region** 包含数据的维度顺序 (**dim_order**) 以及各个维度的访存模式 (**intervals**)，其中每个维度的访存模式使用 **Interval** 数据结构来表示。多个维度上的访存模式通过嵌套的循环（如 **for** 循环）来表示，循环的嵌套顺序与维度顺序一致。**Region** 数据结构能够将多维度的访存模式组织起来，便于进行多维度的地址推导和复用优化。

4.3.2 地址推导与数据复用整体算法概览

本工作提出了一种系统化的地址推导与数据复用优化算法，其核心思想是通过计算图的反向遍历和模式推导，实现高效的内存访问优化。该算法采用分阶段处理策略，将复杂的访存模式推导过程分解为四个关键步骤，可以通过算法 4.1 描述。

算法中的步骤如下：

入口算子选择与数据拆分：作为算法的基础阶段，该步骤通过选择计算图的最后一个操作作为入口算子，并对其输出数据进行合理拆分，为后续推导建立初始条件。数据拆分策略直接影响后续推导的精度和效率，本工作采用基于数据块的分割方法，确保每个数据块的大小与目标硬件架构的缓存特性相匹配。

反向遍历与访存模式推导：该阶段采用自底向上的推导策略，从入口算子开始，通过递归调用 `back_traverse` 函数，逐层推导计算图中每个操作的输入访存模式。这种反向推导方法能够有效捕捉数据依赖关系，确保推导结果的正确性。在推导过程中，系统会记录每个操作的访存特征，为后续优化提供必要的信息支持。

数据复用计算：本阶段重点解决数据重用问题，通过分析相邻数据块之间的重叠区域，精确计算可复用范围 (`reuse`) 和实际计算范围 (`compute`)。该计算过程考虑了数据访问的时空局部性特征，能够有效识别潜在的数据重用机会，为后续的访存优化提供量化依据。

读访问推导与最终访问计算：作为算法的最后阶段，该步骤综合所有消费者操作的需求，推导出最优的读访问模式。通过 `calculateFinalAccess` 函数，系统能够生成兼顾性能和资源利用率的最终访问方案，确保计算图的执行效率达到最优。

该算法框架具有通用性和可扩展性，能够适应不同类型的深度学习模型和硬件架构，为深度学习编译器的访存优化提供了系统化的解决方案。接下来的章节将围绕该算法具体展开描述每一步的过程。

4.3.3 地址推导的基本流程

地址推导是解决重复访存问题的核心步骤。本工作提出了一种基于图融合的地址推导方法，通过选择入口算子、数据拆分、原子操作分解以及复用优化等步骤，实现了高效的地址推导。

地址推导需要考虑每个算子的输入和输出的映射关系，从而根据算子的输出拆分方式，推导出算子的输入访存模式。本工作通过研究深度学习算法的基本算子的访存模式，提出了一种算子原子化的概念，将深度学习算法分类成多个原子化的操作类别。本工作抽象出了一系列原子推导公式，并根据原子操作的访存模式推导公式进行推导。这些基本操作包括但不限于：

- **前向滑窗操作 (Sliding Window Forward)：**用于卷积 (`conv`)、池化 (`pool`) 等操作中，通常是宽度 (`W`) 和高度 (`H`) 维度上。
- **前向填充操作 (Pad Forward)：**用于卷积、池化操作中的填充 (`padding`)，或者独立的填充操作 (填充大小为负数)。
- **后向滑窗操作 (Sliding Window Backward)：**用于反卷积 (`deconv`)、反池

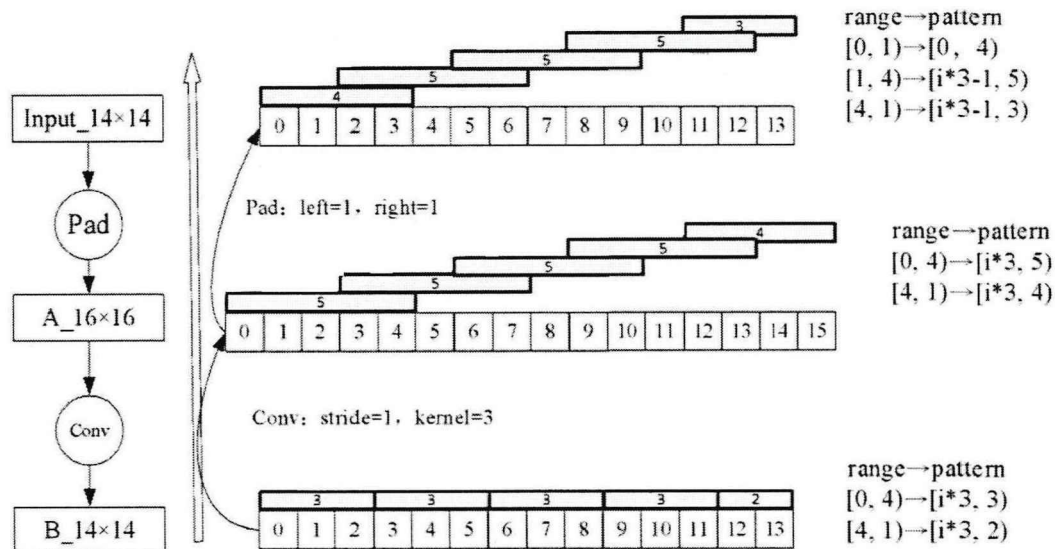


图 4.6 一个典型的神经网络片段的拆分数据块的推导及其 Interval 描述

化 (unpool) 等操作中, 通常是宽度 (W) 和高度 (H) 维度上。

- 后向填充操作 (Pad Backward): 用于反卷积中的填充, 或者独立的填充操作 (填充大小为负数)。
- 点对点操作 (Pointwise): 例如加减乘、激活函数计算等。
- 广播操作 (Broadcast): 用于类似 NumPy 的某个维度的广播操作, 如循环加 (cycle add)。
- 聚合操作 (Reduce): 用于对卷积操作的输入通道维度进行累加操作等。

每个原子操作的访存模式推导公式描述了输入数据和输出数据之间的访存地址关系。通过原子操作分解, 能够将复杂的算子操作分解为简单的原子操作, 并提供相应的访存模式推导公式, 简化了地址推导过程。

图 4.6 展示了一个访存模式推导示例。以计算图中顺序连接的两个算子: 填补 (pad) 算子和卷积 (conv) 算子为例, 采用反向推导方式, 选择卷积算子作为入口算子, 并且已知卷积算子的输出数据高度 (H 维度) 和宽度 (W 维度) 为 14, 卷积的步长为 1, 卷积核大小为 3, 此处均以宽度维度 (W 维度) 数据为例。pad 算子的参数包括: 左填补为 1, 右填补也为 1。如图 4.6 所示, 假设将卷积算子的输出数据 (图中数据块 B_14x14) 的宽度维度按拆分因子 $factor = 3$ 进行拆分, 则输出数据按照 $3 * 4 + 2$ 进行拆分, 从而拆分成 5 个数据块。这五个数据块的访存模式用 Interval 可以表示为:

$$range \rightarrow pattern : [0, 4) \rightarrow [i * 3, 3) [4, 1) \rightarrow [i * 3, 2) \quad (4.9)$$

接着, 基于卷积算子的输出数据的访存模式, 反推卷积算子的输入数据 (图中数据块 A_16x16) 的访存模式。卷积算子在 W 维度上的操作对应前向滑窗这一原子操作, 因此, 根据前向滑窗操作的访存模式推导公式, 可以推导出卷积算子的

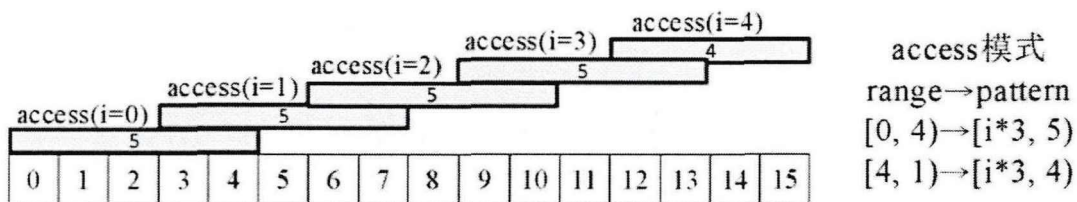


图 4.7 access 模式及其 Interval 描述

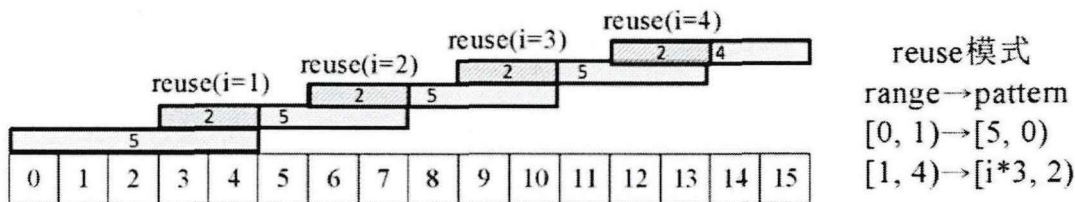


图 4.8 reuse 模式及其 Interval 描述

输入数据在 W 维度的访存模式。这一推导结果用 Interval 可以表示为:

$$range \rightarrow pattern : [0, 4) \rightarrow [i * 3, 5) [4, 1) \rightarrow [i * 3, 4) \quad (4.10)$$

同理, 可以通过前向填补操作的访存模式推导公式, 进一步推导出输入数据 (图中数据块 $Input_14 \times 14$) 的访存模式。从图 4.6 中还可以看出, 每个操作的输入数据 (如 $B_14 \times 14$ 和 $Input_14 \times 14$) 相邻数据块的访存之间存在重叠, 会导致重复读出或写入相同的数据。

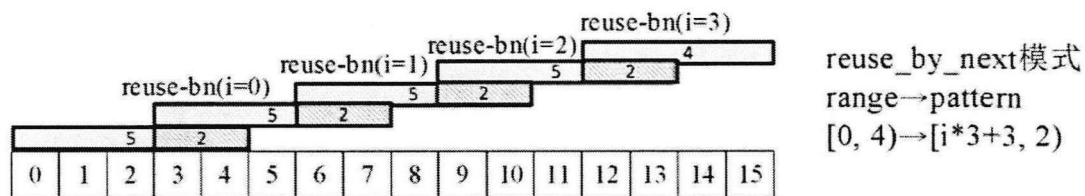
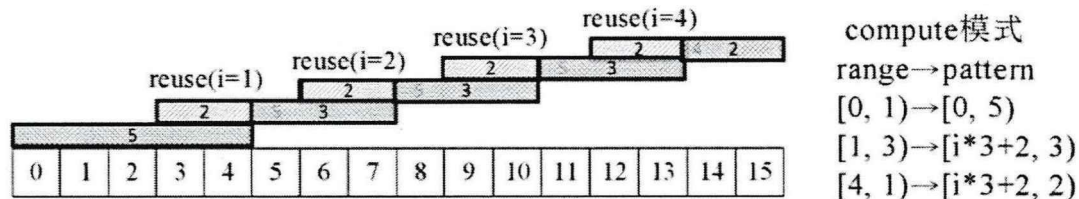
4.3.4 数据复用优化

本节旨在探讨如何通过检测相邻两次访存之间的重叠数据范围, 以及基于该重叠数据范围, 确定每次访存实际需要访问或计算的数据范围, 从而实现数据复用优化。

首先考虑一个具体的数据块示例, 如图 4.7 所示。该数据块长度为 16, 按照拆分因子 $factor=5$ 进行拆分。数据块按照 $5 * 4 + 4$ 进行拆分, 从而拆分成 5 块, 即需要 5 次循环进行访问。每次访存的步长 $stride = 3$, 前 4 次访存的大小为 5, 最后一次访存的大小为 4。可以看出, 图中所示这 5 个数据块的相邻数据块之间存在部分重叠, 因此有可能产生数据复用。本章用 *access* 描述这一拆分后的数据块的概念。

在拆分数据块的基础上, 可以有两种方式来表示可以复用的数据或对应的访存地址。一种方式是用当前数据块与前一数据块之间的数据交集来表示, 也即对应当次访存与上一次访存之间的数据交集, 其可以使用 *reuse* 来指代。换言之, *reuse* 表示当次访存中, 可以复用之前访存的范围。图 4.8 表示相邻访存之间的复用情况。可以看出, *reuse* 的计算公式可以表示如下:

$$reuse(i) = intersect(access(i), access(i - 1)) \quad (4.11)$$

图 4.9 *reuse_by_next* 模式及其 Interval 描述图 4.10 *compute* 模式及其 Interval 描述

其中, $access(i)$ 表示第 i 个数据块。 $intersect$ 表示两相邻数据块之间重复的数据范围。

另一种方式是用当前数据块与下一数据块之间的数据交集来表示, 也即对应当次访存与下一次访存之间的数据交集, 其可以使用 *reuse_by_next* 来指代, 如图 4.9 所示。换言之, *reuse_by_next* 表示当次访存中, 会被下一次访存所使用的范围。可以看出, *reuse_by_next* 的计算公式可以表示如下:

$$reuse_by_next(i) = reuse(i + 1) \quad (4.12)$$

由于相邻访存之间可以存在复用, 因此在某次访存中, 真正需要访存的范围即为整体访存部分除去复用部分后剩下的部分, 该部分可以使用 *compute* 来指代, 如图 4.10 所示。换言之, *compute* 表示的是算子真正需要计算并写入的数据部分。*compute* 的计算公式可以表示如下:

$$\begin{aligned} compute(0) &= access(0) \\ compute(i) &= access(i) - reuse(i), i > 0 \end{aligned} \quad (4.13)$$

当存储器配置有两个缓冲区 (乒乓缓冲空间与乒乓缓冲空间), 采用乒乓 (ping-pong) 流水方式支持数据的流水处理时, 上述相邻访存之间的复用可以通过将要复用的数据从当前缓冲空间 (例如 ping 空间) 搬运到下一流水的缓冲空间 (例如 pong 空间), 反之亦然。由此可以避免数据的重复计算和对外部存储器的多次访存。具体的数据复用优化的案例如图 4.11 所示, 沿用图 4.6 中的各种条件, 在此基础上应用前面描述的复用优化方案, 进一步检测访存之间可能的数据复用, 以减小访存量, 避免重复计算, 提高处理效率。从图中可以看出, 经过复用优化后, 实际需要计算的数据 (图 4.11 中 *Compute* 部分) 不存在重叠, 也即不会重复计算同一数据, 而需要复用的数据可以直接在缓冲区的乒乓空间之间搬运, 减少对外部存储器 (例如 DRAM) 的访存。

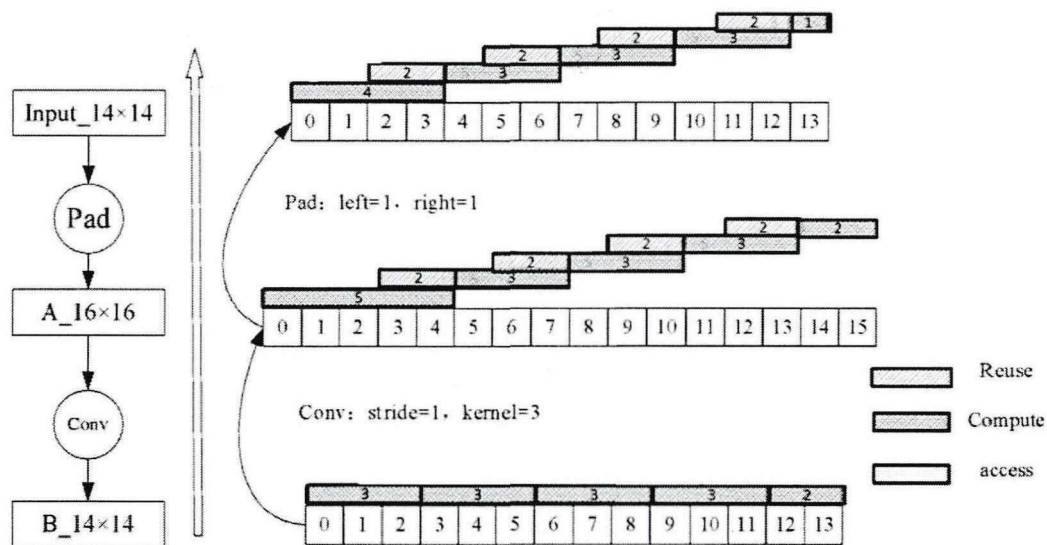


图 4.11 数据复用案例

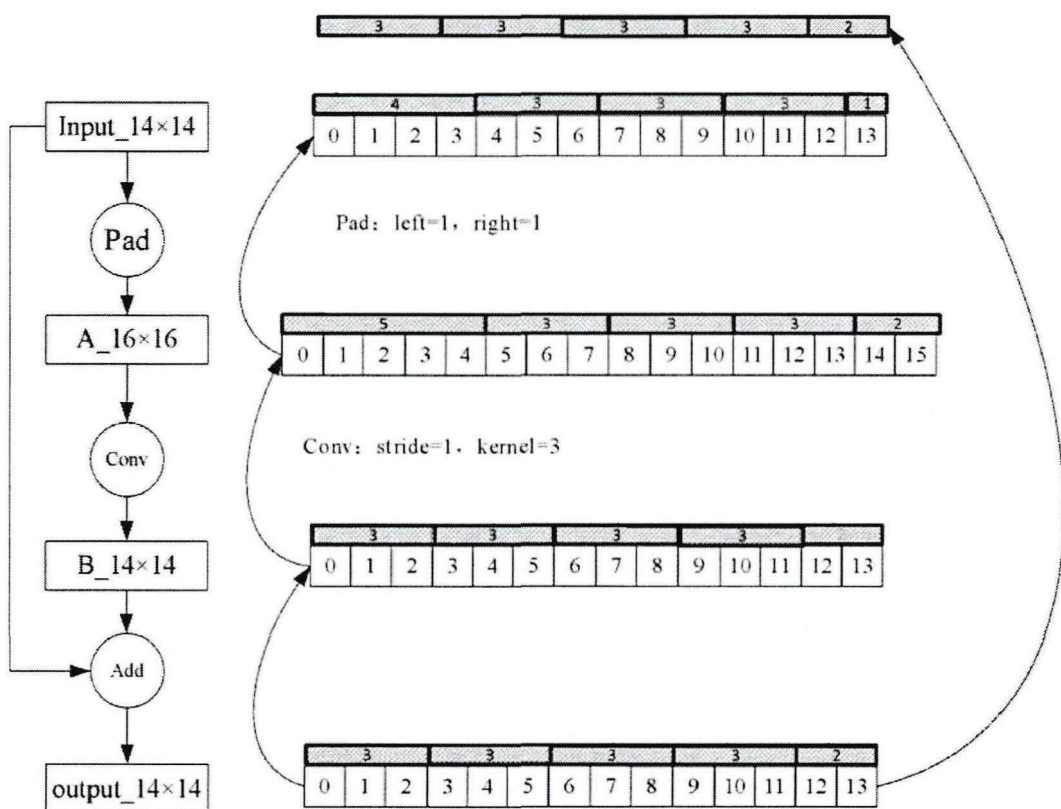


图 4.12 同一数据存在多个访存模式的示例

4.3.5 访存模式合并方案

在计算图中，分支结构的存在可能导致同一数据被多个算子使用，从而产生多个访存模式。为了优化访存并减少重复访存，本章提出了一种访存模式合并方案。该方案通过合并多个访存模式为一个最终访存模式，以确保同一数据仅需一种访存模式即可满足所有消费者的需求。

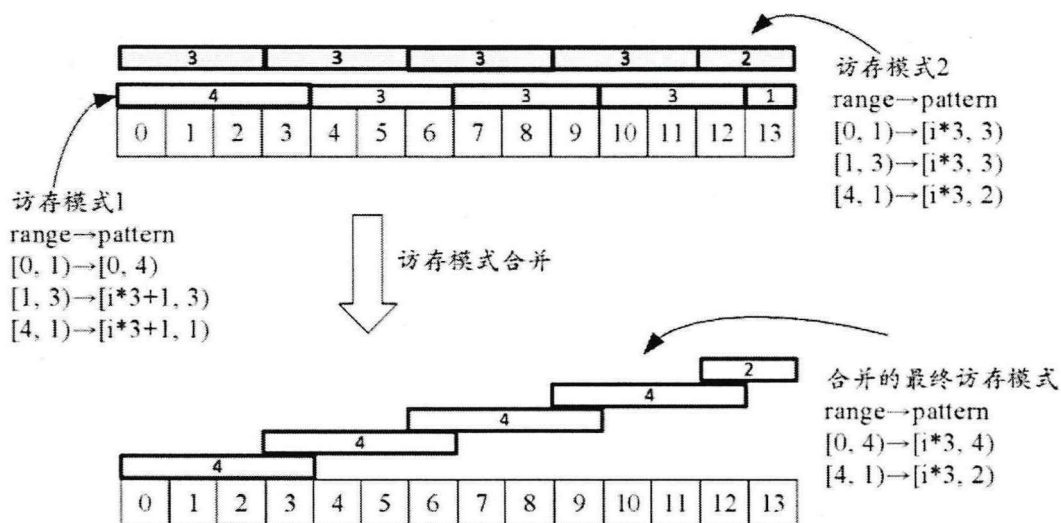


图 4.13 将不同访存模式的数据块进行合并

表 4.3 访存模式合并后的 Interval 表示

Range(i)	访存模式 1	访存模式 2	合并后最终的访存模式
[0, 1)	[0, 4)	[i * 3, 3)	[i * 3, 4)
[1, 3)	[i * 3 + 1, 3)	[i * 3, 3)	[i * 3, 4)
[4, 1)	[i * 3 + 1, 1)	[i * 3, 2)	[i * 3, 2)

图 4.12 展示了一个同一数据存在多个访存模式的示例。在此示例中，计算图除了包含填补 (pad) 算子和卷积 (conv) 算子外，还增加了一个元素对位加 (add) 算子的分支。add 算子对原始输入数据和卷积算子输出数据执行元素对位加运算。通过反向推导，可以确定计算图中各数据的访存模式。如图所示，数据块 *Input_14x14* 被 add 算子和 pad 算子读取，且它们所需的数据大小不同，因此存在两种访存模式。

针对同一数据具有多个访存模式的情况，本章提出了一种合并策略。该策略通过确定每次访存的数据范围，以覆盖所有消费者的需求，从而将多个访存模式合并为一个最终访存模式。合并后的访存模式用 *final_read_access* 表示，其计算方式如下：

$$final_read_access(i) = union(read_access(i, all_consumer_ops)) \quad (4.14)$$

其中，*read_access(i, all_consumer_ops)* 表示第 *i* 个数据在所有消费者操作中的访问模式。

图 4.13 展示了针对图 4.12 中数据的两种访存模式进行合并的过程。合并后的每次访存都覆盖了访存模式 1 和访存模式 2 的对应次访存所需的数据，即二者所需数据的并集。上述合并用 Interval 数据结构如表格 4.3 所示：

由于进行了访存模式合并，因此在采用复用优化方案时，需基于合并的最终访存模式再次确定实际复用 *reuse* 和计算 *compute* 的部分。图 4.14 展示了基于合

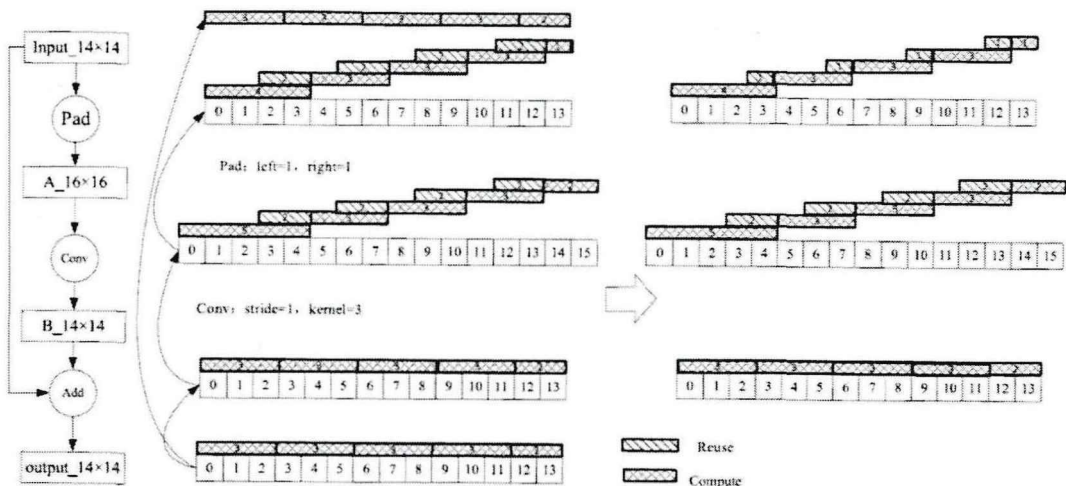


图 4.14 合并之后重新计算 reuse 和 compute，得到最终可复用的数据块访存模式

并的最终访存模式进行复用优化的效果。图中左边展示了 $Input_{14 \times 14}$ 数据块合并前的两个访存模式，右边展示了合并后的访存模式。合并后，相邻数据块的访存之间存在重叠，因此可以复用重叠数据。根据前文描述的 reuse 和 compute 部分的计算方式，可以算出最终 reuse 和 compute 部分。此外， $A_{16 \times 16}$ 数据块和 $B_{14 \times 14}$ 数据块都只有一个访存模式，因此无需合并处理，其最终的 compute 部分也与前面描述的相同。

这些步骤共同构成了一个全面的地址推导框架，为深度学习计算图中的融合操作提供了精确的访存模式，从而为高效的指令生成和空间分配奠定了基础。

4.4 上采样计算模式优化

本节提出了一种针对上采样计算模式的优化方案，通过分析上池化层与深度卷积层的计算特性，实现了计算效率的显著提升。基于上池化层的零填充特性，本章节发现深度卷积运算的结果仅与上池化层的非零输入数据相关，因此可以直接针对这些有效数据进行计算。此外，当每个卷积输出点的感受野内仅包含一个非零数据元素时，计算结果之间无需进行累加操作，从而进一步简化了计算过程。

本章节提出的优化方案主要包含两个关键步骤：首先，计算卷积核与上池化输入数据的乘积结果；其次，根据池化索引对运算结果进行重排以得到最终输出。具体而言，在计算过程中，卷积核与输入数据的运算被分解为多个轮次进行，每个轮次针对卷积核深度方向上的不同权值向量进行计算。这种分轮次计算方式能够有效利用硬件资源，提高计算并行度。

算法 4.2 详细描述了上采样计算模式优化的具体过程。该算法以输入特征图和卷积核作为输入，通过双重循环结构实现计算过程。外层循环遍历卷积核的四

算法 4.2 上采样计算模式优化算法

Input: 输入特征图 $F_{in} \in \mathbb{R}^{H \times W \times C}$, 卷积核 $K \in \mathbb{R}^{H \times W \times C}$

Output: 计算结果集合 R

```

1 初始化结果集合  $R \leftarrow \emptyset$ ;
2 for 每个权值向量位置  $(h, w) \in \{(range(0, H), range(0, W))\}$  do
3   for 每个输入向量  $v \in F_{in}$  do
4      $r \leftarrow v \odot K[h, w, :]$ ;
      // 对位乘法运算
5      $R \leftarrow R \cup \{r\}$ ;
6   end
7 end
8 return  $R$ ;

```

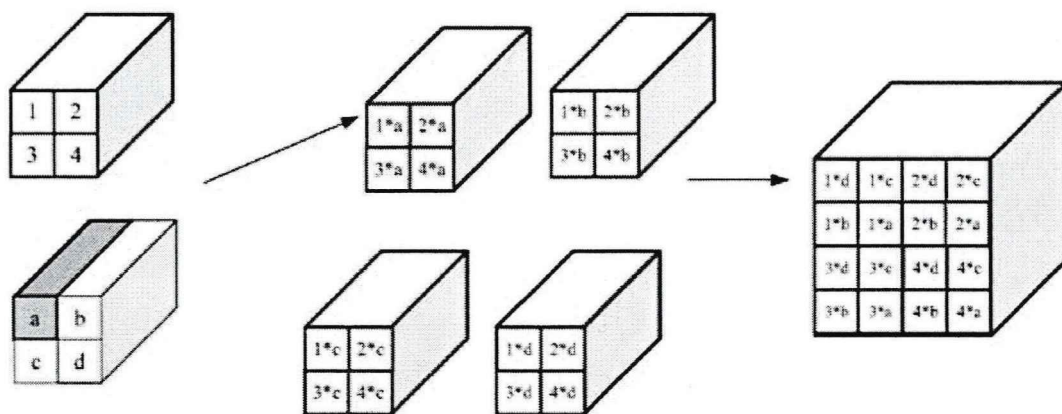


图 4.15 上采样计算模式优化

个权值向量位置，内层循环遍历输入特征图中的每个输入向量。在每次迭代中，算法执行对位乘法运算，将当前权值向量与输入向量相乘，并将结果存储在结果集合中。这种分轮次计算方式充分利用了卷积核的空间局部性特征，实现了计算过程的高效并行化。

以 $2 \times 2 \times C$ 的输入特征图和卷积核为例，如图 4.15 所示，在第一个计算轮次中，卷积核中 $H=1$ 、 $W=1$ 位置的权值向量 a 分别与输入特征图深度方向上的各个输入向量进行对位乘法运算，产生 4 个结果向量。在第二个计算轮次中，权值向量 b 与输入向量相乘产生另外 4 个结果向量。通过这种方式，依次完成卷积核中所有权值向量的计算，最终得到完整的计算结果集合。

在结果重排阶段，本章节提出了一种基于池化索引的映射机制。该机制通过建立输入向量索引与结果向量索引之间的映射关系，实现了计算结果的高效重排。具体而言，对于 2×2 卷积核，每个输入向量会依次遍历卷积核的四个位置，产生四个对应的结果向量。通过分析输入向量与卷积核中心点的固定偏移关系，可以直接基于输入向量的索引确定其与所有权值向量相乘的结果向量索引。

算法 4.3 详细描述了基于池化索引的结果重排过程。该算法以计算结果集合、池化索引和输入特征图尺寸作为输入，通过双重循环结构实现结果重排。外层循

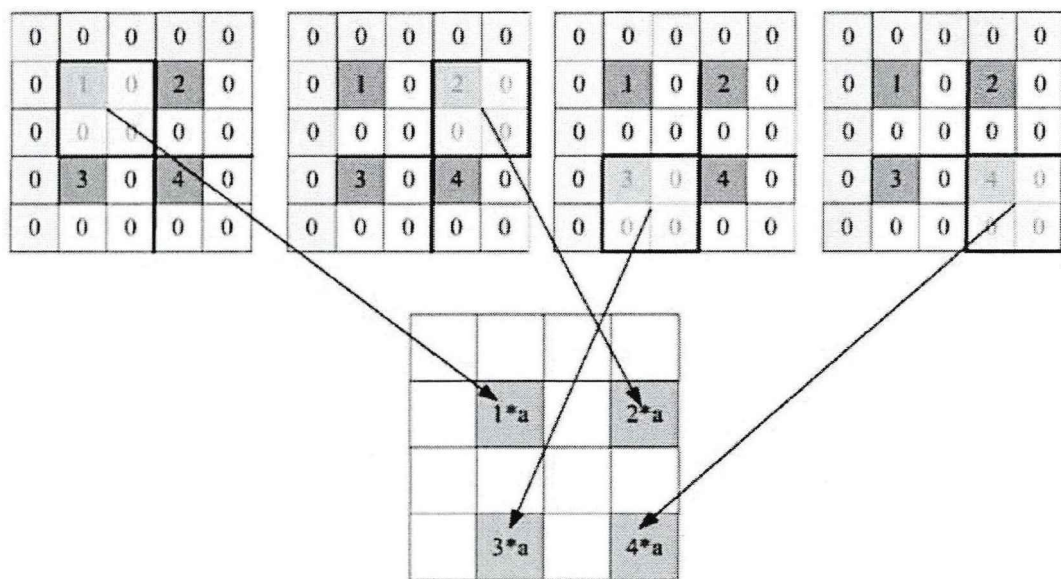


图 4.16 上采样重排优化

算法 4.3 基于池化索引的结果重排算法

Input: 计算结果集合 R , 池化索引 P , 输入特征图尺寸 (H, W) , 卷积核尺寸 (K_H, K_W)

Output: 重排后的输出特征图 $F_{out} \in \mathbb{R}^{(H+K_H-1) \times (W+K_W-1) \times C}$

```

1 初始化输出特征图  $F_{out} \leftarrow 0$ ;
2 for 每个输入向量  $v$  及其索引  $(x, y)$  do
3   for 每个权值向量位置  $(h, w) \in \{(i, j) | i \in [1, K_H], j \in [1, K_W]\}$  do
4     // 计算输出位置
4      $(x', y') \leftarrow (x + h - 1, y + w - 1)$ ;
5     // 根据池化索引调整位置
5      $(x'', y'') \leftarrow P[(x', y')]$ ;
6     // 存储结果
6      $F_{out}[x'', y'', :] \leftarrow R[v, (h, w)]$ ;
7   end
8 end
9 return  $F_{out}$ ;

```

环遍历输入特征图中的每个输入向量及其索引，内层循环遍历卷积核的四个权值向量位置。在每次迭代中，算法首先计算输出位置，然后根据池化索引调整位置，最后将结果存储到输出特征图的对应位置。这种基于池化索引的重排机制有效避免了复杂的索引计算，显著提升了结果重排的效率。

本章节系统性地研究了卷积核位置与结果向量索引之间的映射规律，揭示了卷积运算中索引映射的数学本质。如图 4.16 所示，通过理论分析和实验验证，本工作发现卷积核中不同位置的权值向量与输入向量的乘积结果在输出特征图中呈现出规律性的空间分布特征。具体而言，对于卷积核的第 1 个权值向量（池化后的索引为 (2, 2)），其与权值向量 a 的乘积结果精确映射到 4×4 卷积结果中的 (2, 2) 位置；第 2 个权值向量（池化后的索引为 (2, 4)）的乘积结果则映射到 (2, 4) 位置；而第 3 个权值向量（池化后的索引为 (4, 2)）的乘积结果对应到 (4, 2) 位

置。这种确定性的索引映射关系为结果重排提供了理论基础,使得计算过程可以避免复杂的索引计算开销。

进一步地,本章节建立了输入向量索引与结果向量索引之间的数学映射模型。研究表明,当输入向量与卷积核的第1个权值向量进行乘积运算时,其索引映射关系保持恒等变换,即输入向量索引 (x, y) 直接映射为结果向量索引 (x, y) 。对于其他权值向量,映射关系呈现出规律性的坐标偏移特征:第2个权值向量的乘积结果索引为 $(x, y - 1)$,第3个权值向量的乘积结果索引为 $(x - 1, y)$ 。这种规律性的偏移特征源于卷积核的固定空间结构,为索引计算提供了简化依据。

基于上述发现,本章节提出了基于固定偏移的索引计算优化方法。由于每个输入向量在遍历 2×2 卷积核时,其相对于卷积核中心点的偏移量保持恒定,因此可以直接根据输入向量的索引推导出所有相关乘积结果的索引。具体而言,通过建立卷积核中心点与输出点之间的索引映射关系,可以快速确定输入向量产生的每个向量乘积结果的索引位置。这种优化方法不仅简化了索引计算过程,还显著提升了结果重排的效率,为上采样计算模式的优化提供了理论支撑。

通过上述优化方案,本章节实现了上采样计算模式的显著改进。实验结果表明,该方案在保持计算精度的同时,能够有效减少计算量和数据搬运开销,为上采样操作提供了高效的计算框架。

4.5 实验评估与分析

4.5.1 实验目标

本节采用多层次实验设计,在 MLU370 云端 AI 加速芯片平台上,基于一系列 CNN 网络,系统评估融合算子指令生成系统的性能优势。首先,通过消融实验分别验证数据复用机制和上采样性能优化策略的性能增益:在相同算子融合条件下,对比分析带/不带数据复用机制的性能差异,以及带/不带上采样性能优化策略的执行效率。其次,将完整的融合算子指令生成系统与单算子执行方式和当前最先进的 Chimera^[83] 解决方案进行网络层面的性能对比,从多个维度评估系统的整体性能。通过上述实验设计,全面验证所提出系统在算子融合场景下的优化效果和先进性。

4.5.2 实验环境

本研究构建了完整的软硬件协同验证平台,采用多层次对比实验设计,系统评估融合算子指令生成系统的性能优势。在软件层面,本研究采用第3章提出的 FusionFrame 作为编译器前端,该框架通过精确的存储层次建模和分块策略优化,为后端指令生成提供了高质量的调度方案。具体而言, FusionFrame 框架实现了

计算图的自动化解析、算子融合策略的智能生成以及存储层次的优化调度,显著提升了编译器的优化能力。在硬件层面,基于寒武纪 MLU 370 云端 AI 加速芯片平台构建了完整的软硬件协同验证平台,该平台采用多核异构架构,支持大规模并行计算,为深度学习模型的加速计算提供了强大的硬件支持。通过软硬件协同优化,本实验平台能够充分验证指令生成系统在不同计算负载下的性能表现。

4.5.3 数据复用的消融实验

1. 实验方法

本节通过系统化的消融实验设计,从多个维度定量评估数据复用机制的性能优化效果。数据复用方案的核心目标在于解决深度学习模型中普遍存在的冗余数据爆炸问题,该问题直接导致 DRAM 访存量的显著增加以及计算资源的低效利用。为全面评估数据复用机制的有效性,本研究选取 DRAM 访存量、计算时间、片上数据搬运量以及执行总时间三个关键性能指标作为评估维度。其中,DRAM 访存量反映了系统对存储带宽的优化效果,计算量体现了计算资源的利用效率,片上数据搬运量是为了反映将复用的数据在片上进行搬运带来的开销,而执行总时间则综合反映了系统的整体性能表现。通过这三个维度的定量分析,能够系统性地验证数据复用机制在减少数据冗余、优化计算效率方面的实际效果。

2. 实验用例

为全面验证数据复用机制的通用性和有效性,本研究选取了计算机视觉领域具有代表性的深度神经网络架构作为基准测试集。实验用例涵盖了 CNN 架构的经典网络 ResNet50^[4]、VGG16^[3]、MobileNetV2^[56],以及 Transformer 架构的 BERT^[10]模型。这些网络架构在图像分类和自然语言处理任务中均表现出色,且具有显著不同的计算特征:ResNet50 和 VGG16 主要包含卷积、池化等滑窗类型算子,MobileNetV2 引入了深度可分离卷积等轻量化操作,而 BERT 则主要依赖矩阵乘法和注意力机制等点对点操作。通过这种多样化的测试集设计,能够充分验证指令生成系统在不同计算模式下的通用性和优化效果,为数据复用机制的评估提供可靠的实验基础。

3. 实验结果与分析

实验结果表明,本文提出的数据复用机制在存储访问、计算效率和系统性能等多个维度均实现了显著优化。在存储访问方面,DRAM 访存量平均降低 18.8%,这一优化源于数据复用机制有效消除了冗余数据的重复加载,显著缓解了外部存储器的访问压力。计算效率方面,得益于数据复用机制避免了重复计算,计算时间平均减少 14.7%,显著提升了计算资源利用率。值得注意的是,由于数据复用需要在片上存储和传输复用数据,片上数据搬运量增加了 9.3%,但这种低代

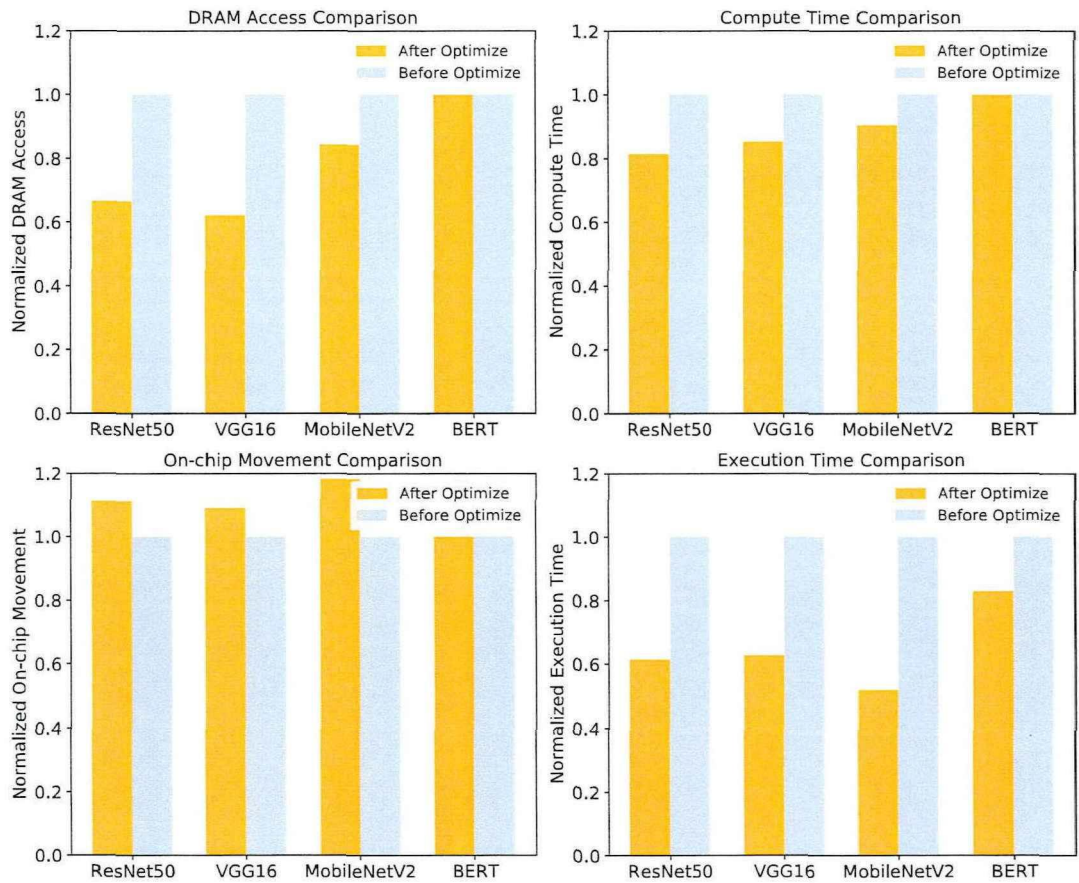


图 4.17 内存复用优化的消融实验

价的片上开销换取了 DRAM 访存和计算量的显著降低，最终使系统整体执行时间平均减少 39.3%。特别地，对于 BERT 这种不包含滑窗类操作的网络，其性能提升主要来自流水线并行优化，因此在 DRAM 访存和计算时间等指标上未表现出明显变化。实验数据充分证明，本文提出的数据复用机制在片上存储开销与全局性能优化之间实现了有效平衡，为深度学习模型的加速计算提供了切实可行的优化方案。

4.5.4 上采样性能优化的消融实验

1. 实验方法

本实验旨在系统评估上采样性能优化策略对融合算子性能的优化效果。上采样性能优化作为指令生成系统的核心优化环节，主要通过优化指令序列的逻辑结构，有效降低融合算子的片上数据搬运开销和计算开销。为全面评估上采样性能优化带来的性能收益，本研究选取片上数据搬运量、计算时间和执行时间三个关键性能指标进行定量分析。其中，片上数据搬运量反映了上采样性能优化对存储访问模式的优化效果，计算量体现了计算资源的利用效率，而执行时间则综合反映了系统的整体性能提升。通过这三个维度的定量评估，能够系统性地验证

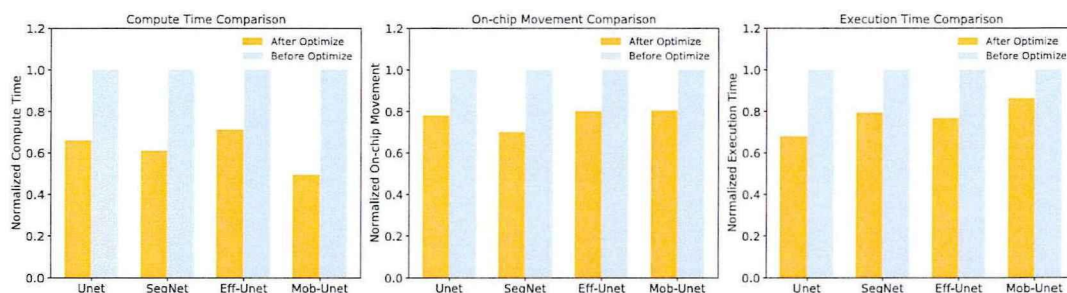


图 4.18 上采样优化的消融实验

上采样性能优化策略在优化数据局部性、提升计算效率方面的实际效果。

2. 实验用例

为验证上采样网络结构性能优化的通用性和有效性，本研究选取了图像语义分割领域具有代表性的深度神经网络架构作为基准测试集。实验用例包括 Unet^[86]、SegNet^[87]、Efficient Unet^[84]以及 Mobile Unet^[88]等典型网络架构。这些网络在医学图像分析、自动驾驶等实际应用中表现出色，且具有显著的计算特征：均包含典型的上采样结构，这些算子在传统实现方式中存在大量的数据搬运和计算问题。通过这种多样化的测试集设计，能够充分验证上采样性能优化策略在不同计算模式下的优化效果，为指令生成系统的性能评估提供可靠的实验基础。

3. 实验结果与分析

如图 4.18 所示，本工作提出的上采样优化策略在多个关键性能指标上均取得了显著的优化效果。从存储访问效率来看，片上数据搬运量减少了 21.3%，这一优化主要得益于通过重排的算法避免了填充操作中的大量补 0 带来的片上数据搬运，减少了不必要的数据搬运操作。在计算效率方面，计算时间减少了 33.7%，这主要归因于通过点对点乘法替换卷积操作，带来了大量的计算量的节省。最终，系统整体执行时间减少了 17.8%。实验结果充分证明了所提出的优化方案能够在上采样场景中有效降低融合算子的计算开销和存储访问开销，为深度学习模型的加速计算提供了可靠的优化方案。

4.5.5 融合算子指令生成系统的性能与 SotA 对比

1. 实验方法

本实验采用系统性对比评估方法，旨在全面验证融合算子指令生成系统的性能优势。为建立可靠的性能基准，实验选取当前最先进的指令生成方法 Chimera^[83]作为对比基准。在实验设计上，采用控制变量法确保对比的公平性：在相同的调度策略和硬件环境下，对比分析不同指令生成算法在 DRAM 访存量、执行时间方面的性能差异。

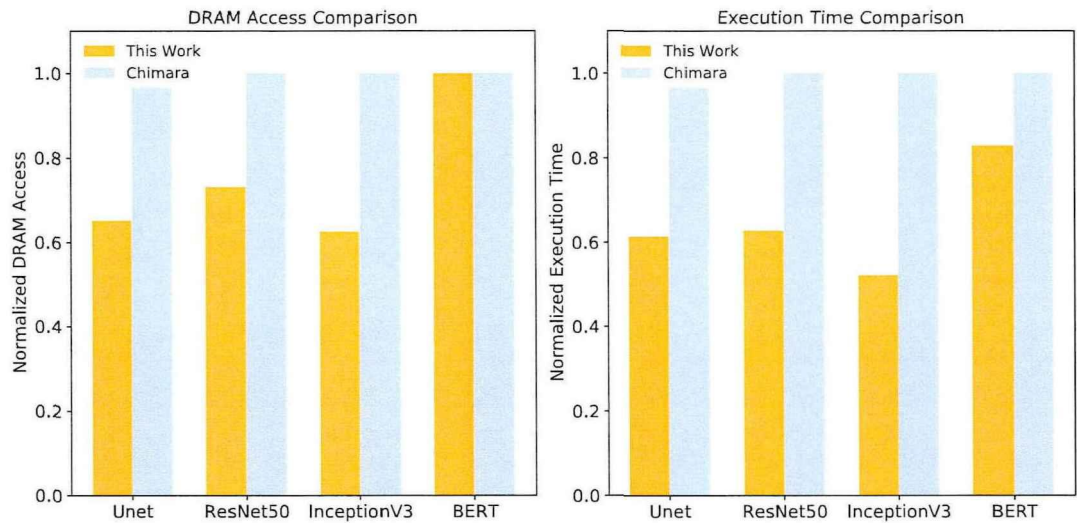


图 4.19 本工作与 SotA Chimera 的指令生成性能对比

2. 实验用例

为全面验证系统的通用性和优化效果，本实验构建了多维度、多场景的测试基准集。实验用例涵盖三大类深度学习网络架构：首先，选取计算机视觉领域具有代表性的 CNN 网络 ResNet50^[4]和 InceptionV3^[77]，这些网络包含丰富的卷积、池化等计算密集型算子；其次，选取自然语言处理领域的典型 Transformer 架构 BERT^[10]，该网络主要依赖矩阵乘法和注意力机制等点对点操作；最后，选取图像语义分割领域的代表网络 Unet^[86]，该网络包含典型的上池化与卷积融合操作。通过这种多样化的测试集设计，能够充分验证指令生成系统在不同计算模式下的通用性和优化效果。

3. 实验结果与分析

如图 4.19所示，本工作提出的融合算子指令生成系统在存储访问效率和计算性能方面均显著优于 Chimera 基准方案，展现出优异的优化效果。在存储访问效率方面，得益于优化的数据复用机制和指令选择策略，系统的 DRAM 访存量仅平均为 Chimera 方案的 76.6%。在计算性能方面，系统整体执行时间仅为 Chimera 方案的 63.4%。性能提升主要源于三个关键优化：对于 Unet 这一包含上采样操作的网络，通过优化上采样执行模式，减少了 38.2% 的执行时间开销；对于 ResNet 等卷积网络，数据复用机制有效减少了平均约 40% 的执行时间，显著提高了计算资源利用率；对于 BERT 等 Transformer 架构，由于引入流水线并行优化，实现了约 27.1% 的性能提升，而 Chimera 由于缺乏流水线并行支持，无法获得此类优化收益。值得注意的是，不同网络架构的优化效果存在显著差异：对于包含典型滑窗算子的 CNN 网络，性能提升主要来自于数据复用机制；而对于 BERT 等点对点操作占主导的网络，性能提升主要依赖于流水线并行优化。实验结果充分证明了所提出的融合算子指令生成系统在降低存储访问开销和提升计

算效率方面的显著优势。

4.6 讨论

本工作为深度学习计算图的指令生成提供了系统性的优化方案，但仍存在进一步扩展和优化的空间。

硬件适配的进一步扩展：当前系统主要针对寒武纪 MLU 系列芯片进行优化，未来可进一步扩展至更多硬件平台。具体而言，可通过抽象硬件特征描述、设计通用中间表示等方式，实现对不同架构 AI 加速器的支持，包括华为 Ascend NPU、google TPU 等主流计算平台。这种扩展不仅能够提升系统的通用性，还能异构计算环境下的指令生成提供统一解决方案。

应用场景的进一步扩展：当前优化方案主要针对卷积神经网络，随着大语言模型的快速发展，亟需探索更高效的指令生成策略。具体而言，可针对大语言模型的计算特征，设计专门的算子融合策略和内存优化方案。例如，针对 attention 机制中的矩阵计算，可探索基于稀疏计算的指令优化；针对长序列处理，可研究基于分块计算的指令生成技术。这些优化将显著提升大语言模型在 AI 加速器上的计算效率，为大规模模型部署提供技术支持。

4.7 本章小结

本章针对深度学习计算图中的关键性能瓶颈，系统性地提出了融合算子指令生成系统的优化方案。首先，通过引入基于向量描述的地址推导系统，有效解决了深度融合场景下的地址推导难题，显著提升了指令生成的效率和精度。其次，针对上采样操作的特殊计算模式，提出了基于池化索引的重排机制和固定偏移的索引计算优化方法，实现了上采样计算模式的高效优化。实验结果表明，所提出的优化方案在多个基准测试集上均取得了显著的性能提升：数据复用机制使系统整体执行时间平均减少 39.3%，上采样优化策略使计算时间减少 33.7%，整体执行时间减少 17.8%。与当前最先进的 Chimera 方案相比，本系统在 DRAM 访存量和执行时间方面分别实现了 23.4% 和 36.6% 的优化。这些优化成果为深度学习模型的部署和加速提供了可靠的技术支撑，同时也为后续研究指明了方向。未来工作将重点关注更复杂的网络架构和更大规模的计算图优化，以进一步提升系统的通用性和优化效果。

第5章 基于 MLU 平台的深度学习系统级设计与验证

5.1 引言

深度学习系统软件的优化是提升 AI 计算效率的关键环节。寒武纪作为国产 AI 计算平台的代表，其系统软件架构设计面临着深度学习框架多样性、计算图复杂性以及系统易用性等多重挑战。本文提出的创新性工作已在寒武纪深度学习系统中得到实际应用，有效解决了系统软件优化中的关键问题。

在寒武纪系统软件的整体架构中，本文工作主要应用于算子编译层和图编译层两个核心模块。在算子编译层，本文提出的基于 Triton 的编译优化框架已被集成到寒武纪加速库中，实现了从高级语言到机器指令的高效转换。在图编译层，本文设计的 MagicMind 图编译器通过计算图优化和指令生成技术，显著提升了系统整体性能。这些创新性工作为寒武纪深度学习系统提供了重要的技术支持。

本文工作主要解决了寒武纪系统软件中的三个关键问题。首先，针对深度学习框架的多样性，本文通过 Triton 编译器和 MagicMind 图编译器的协同工作，实现了对主流 AI 框架的良好支持。其次，针对计算图的复杂性，本文提出的图优化和指令生成技术有效提升了计算效率。最后，在保证性能的同时，本文通过模块化设计和优化策略，提高了系统软件的易用性和可扩展性。

本章将详细介绍本文工作如何应用于寒武纪深度学习系统软件。首先，本章节将阐述寒武纪系统软件的整体架构，重点介绍本文工作所涉及的模块。随后，本章节将详细分析算子编译层和图编译层的架构设计，以及本文提出的方法如何解决系统优化中的关键问题。最后，通过实验评估验证这些方法在实际系统中的有效性。

5.2 寒武纪深度学习系统软件设计

5.2.1 寒武纪系统软件整体架构

寒武纪深度学习系统采用分层架构设计，从应用层到底层硬件实现了完整的软件栈支持。如图 5.1 所示，在应用层面，系统针对不同的人工智能场景提供了多样化的解决方案，包括训练 (AI Training)、推理 (AI Inference) 等核心应用场景。这些应用场景的实现依赖于主流的 AI 框架和开源生态系统，如 PyTorch、TensorFlow、飞桨等深度学习框架。这些框架通过构建神经网络计算图，调用寒武纪提供的加速库来实现模型在 MLU 芯片上的高效执行。

寒武纪加速库作为连接 AI 框架与硬件的重要中间层，提供了完整的计算加



图 5.1 寒武纪系统软件整体架构

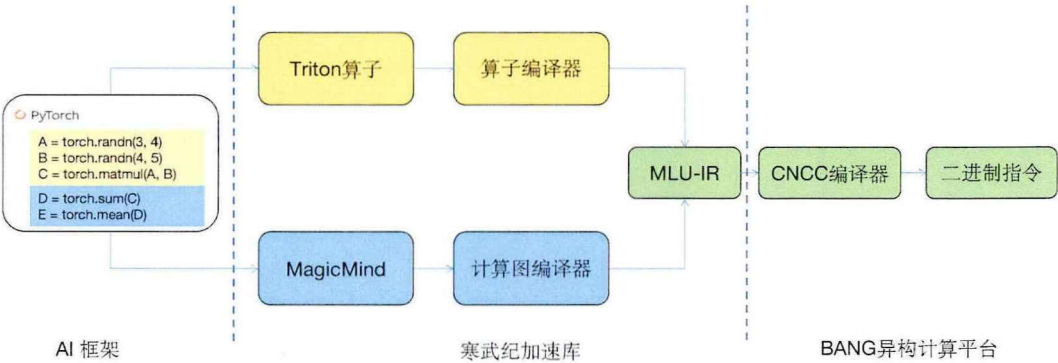


图 5.2 本工作应用场景的整体工作流程

注：本工作第 2 章对应算子编译器的应用场景，第 3 章和第 4 章共同构成计算图编译器的应用场景。

速支持。加速库的核心组件包括 MagicMind 推理计算图融合编译器、CNL 和 TRITON 算子库等。此外，加速库还集成了多个辅助模块：CNServing 提供高效的推理服务支持，CNCL 实现训练场景下的卡间通信，CNLight 则用于性能测量与优化。在底层实现上，加速库依赖于 BANG 异构计算平台工具链，将算子或计算图转换为二进制指令，最终通过 MLU 多平台驱动调度到 MLU 端云一体芯片上执行。这种分层架构设计确保了系统的高效性和可扩展性。

本研究的核心应用场景聚焦于 MagicMind 推理计算图融合编译器和 TRITON 算子库的协同工作。MagicMind 图编译器负责将 AI 框架中的计算图通过一

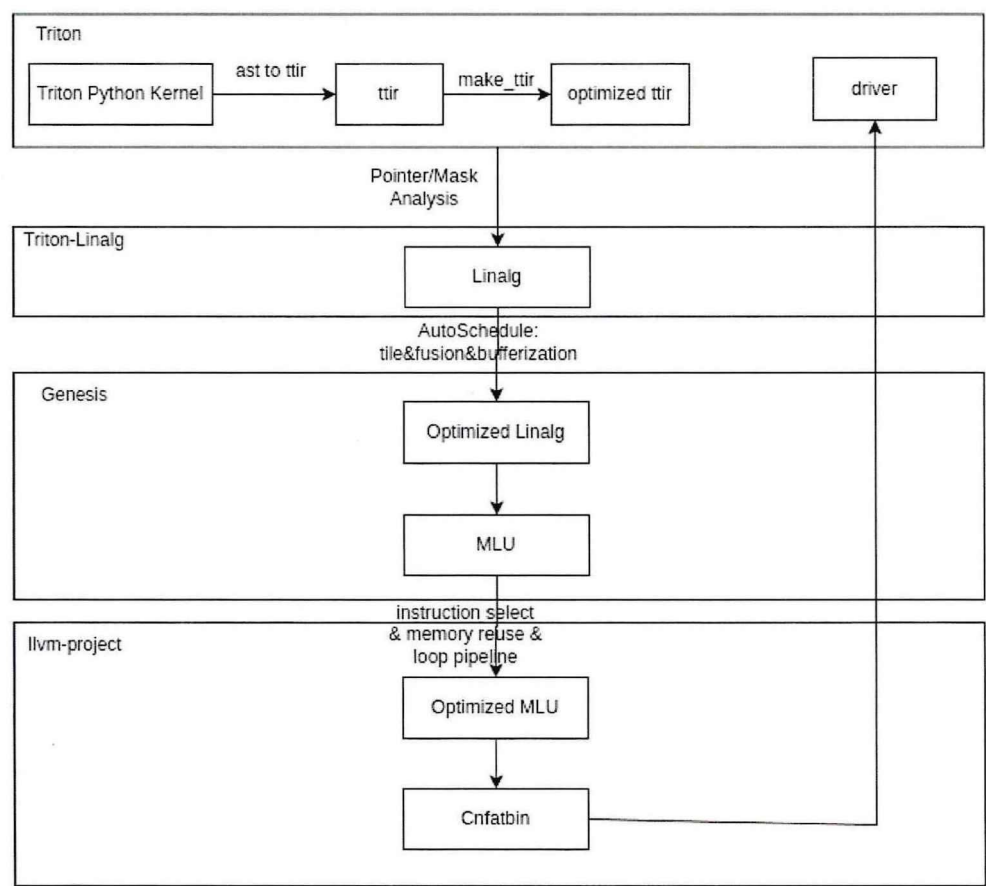


图 5.3 Triton 编译器整体架构

系列编译优化技术转换为 MLU-IR (MLU Intermediate Representation) 中间表示。TRITON 算子库则实现了将用户编写的 Triton 程序转换为 MLU-IR 表示的功能。基于 MLU-IR，BANG 异构计算平台的编译工具链（如 CNCC 等）可以进一步将其编译链接为可执行的二进制机器指令。如图 5.2 所示，以 PyTorch 为例，系统在处理完整计算图时采用混合执行策略：部分算子通过 TRITON 算子库以单算子方式执行，另一部分算子则通过 MagicMind 图编译器进行算子融合优化后，以计算图为单位执行。这种混合执行策略在保证灵活性的同时，最大化了计算效率。

5.2.2 算子编译层架构

Triton 算子编译器采用多阶段优化架构，实现了从高级语言到机器指令的高效转换。如图 5.3 所示，编译器首先将用户编写的 Triton-Python 算子通过抽象语法树分析转换为 Triton-IR 中间表示。随后，通过指针分析等优化技术生成硬件无关的 Linalg 表示。在 Linalg 层面，编译器执行拆分、融合、缓存化等优化操作，并将其转换为 MLU-IR。最后，基于 MLU-IR 进行指令选择、内存复用等传统编译优化，生成二进制指令并封装在 fatbin 文件中，通过驱动层调度在 MLU

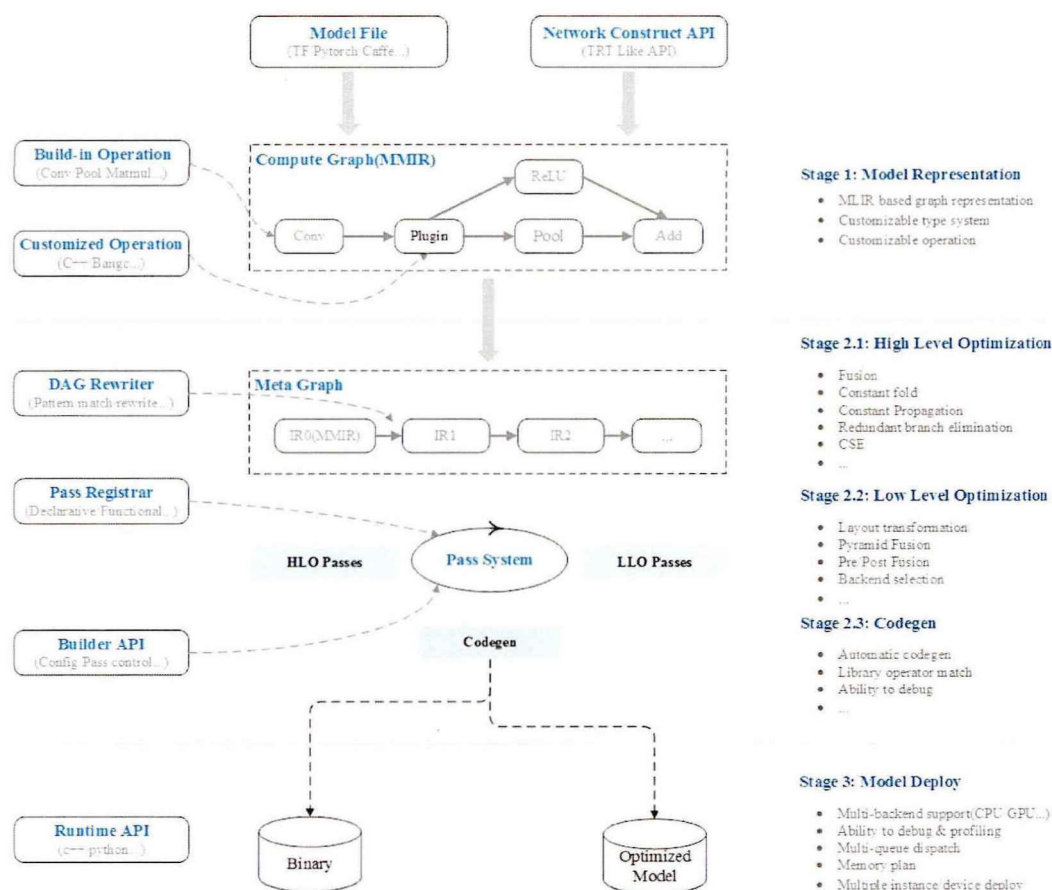


图 5.4 MagicMind 图编译器整体架构

芯片上执行。

本研究在算子编译层架构中做出了多项创新性贡献。首先，在硬件无关表示转换方面，设计了高效的指针分析算法和计算语义的 Linalg 层次抽象，显著优化了访存和计算性能。其次，在 Linalg 到 MLU-IR 的转换过程中，创新性地设计了拆分、缓存化和提升等下降步骤，确保了功能完备性，为后续优化奠定了基础。最后，针对常见二元算子，提出了数据驻留和指令流水两项关键优化技术，在实际应用中取得了显著的性能提升。

5.2.3 图编译层架构

寒武纪图编译系统采用模块化设计，实现了从模型描述到指令生成的全流程优化。如图 5.4 所示，系统首先将 AI 框架提供的算法模型转换为自定义的 MMIR (MagicMind Intermediate Representation) 计算图表示。在 MMIR 层面，系统执行算法级别的图优化，通过简化计算流程来提升性能。随后，系统基于优化后的 MMIR 进行图搜索和调度优化，结合性能评估模型和调度算法，确定算子融合策略和拆分方案。最后，系统根据调度和拆分信息生成图融合指令，完成整个编译过程。

本工作在图编译层架构中取得了多项重要突破。首先,设计并实现了一套完整的融合算子数据流框架,为 MagicMind 系统中的图搜索和调度优化提供了基础支持。其次,在算子融合指令生成过程中,创新性地解决了地址推导等关键技术难题。最后,在算子融合场景下,探索了上采样性能优化策略,在图像分割等领域的典型场景中取得了显著的性能提升。

5.3 系统层面收益评估与分析

本节从系统层面评估寒武纪深度学习系统软件在大语言模型场景下的性能表现。评估实验基于寒武纪 MLU 370 芯片平台和 NVIDIA T4 GPU 平台,采用 PyTorch 深度学习框架的 2.1 版本^[49],重点考察 Triton 算子库和 MagicMind 图编译器在典型大语言模型上的性能优化效果,并进行系统层面的性能对比分析。

实验采用 HuggingFace^[89]开源模型库作为基准测试集,该库集成了当前主流的大语言模型架构。为全面评估系统性能,本章节选取了具有代表性的六种大语言模型: Bart、Bert_large、DistilBert、GPT2、Longformer 和 T5。在传统 CNN 领域,也选取了有代表性的六种网络: ResNet50, MobileNetV2, InceptionV3, Unet, VGG16, SegNet。

在模型选择方面,本章节采用分层抽样策略以确保评估的全面性。对于大语言模型,选取的六个模型涵盖了 Transformer 架构的主要变体: Bart 代表序列到序列模型, Bert_large 和 DistilBert 分别代表大规模和轻量级编码器模型, GPT2 代表自回归生成模型, Longformer 代表长序列处理模型, T5 代表统一文本到文本转换模型。这些模型在参数量、计算模式和任务特性上具有显著差异,能够全面反映系统在不同计算模式下的性能表现。

在传统 CNN 模型方面,选取的六个网络代表了计算机视觉领域的主流架构: ResNet50 作为残差网络的典型代表, MobileNetV2 代表轻量级网络, InceptionV3 代表多分支结构, Unet 代表编码器-解码器架构, VGG16 代表经典深层网络, SegNet 代表图像分割专用网络。这些网络在计算复杂度、内存占用和计算模式上具有显著差异,能够全面评估系统在传统卷积神经网络上的性能表现。

实验设置方面,本章节采用 batch size 为 4 的配置方案,这一选择基于以下考虑:首先,较大的 batch size 可能导致内存溢出,特别是在处理大语言模型时;其次,较小的 batch size 会降低计算效率,无法充分利用硬件并行计算能力;最后, batch size 为 4 能够在计算效率和内存占用之间取得良好平衡,确保评估结果的可靠性。评估方法采用端到端的性能测试方案,通过评估模型加载、图优化到指令生成的全流程的执行时间进行性能分析。

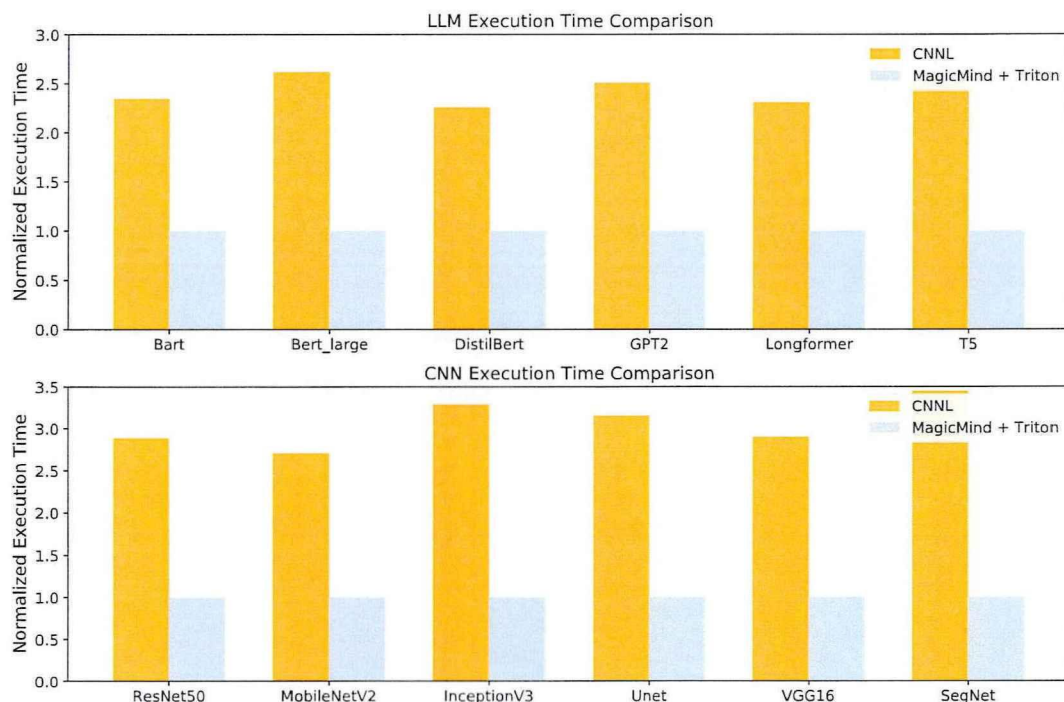


图 5.5 MLU 370 平台上手写算子库 (CNNL) 与基于 Triton 和 MagicMind 加速库的性能对比

5.3.1 MLU 架构下的系统软件收益评估

实验结果表明, 基于 MLU 370 平台, 本研究提出的 Triton+MagicMind 加速方案相比传统手写算子库 CNNL 在深度学习任务中取得了显著的性能提升。如图 5.5 所示, 在卷积神经网络 (CNN) 任务中实现了 3.1 倍的加速比, 在大语言模型 (LLM) 任务中实现了 2.43 倍的加速比。这一性能提升主要源于系统架构的多层次优化: 在 CNN 任务中, 性能增益主要来自两个关键优化点: 首先, 通过算子融合技术有效减少了数据访存开销, 降低了内存带宽压力; 其次, 在指令生成阶段对计算量进行了深度优化, 提高了计算效率。对于 LLM 任务, 性能提升则主要得益于 Triton 算子编译器的优化效果, 其通过高效的中间表示转换和硬件指令生成显著提升了单算子性能; 同时, 算子融合技术有效减少了算子间的调度开销, 进一步提升了整体执行效率。

5.3.2 跨处理器平台的收益评估

本评估采用 MLU 370 与 GPU T4 针对不同类型的深度学习算法任务进行性能对比评估, 体现出本研究工作显著提升了 MLU 平台在深度学习任务中的性能表现。如图 5.6 所示, 在大语言模型领域, MLU 平台相比 GPU 平台实现了 33.64% 的端到端执行时间缩减, 这一性能提升主要得益于 Triton 算子的性能优化, 以及流水线并行这一优化。在卷积神经网络领域, MLU 平台展现出更为显著的性能

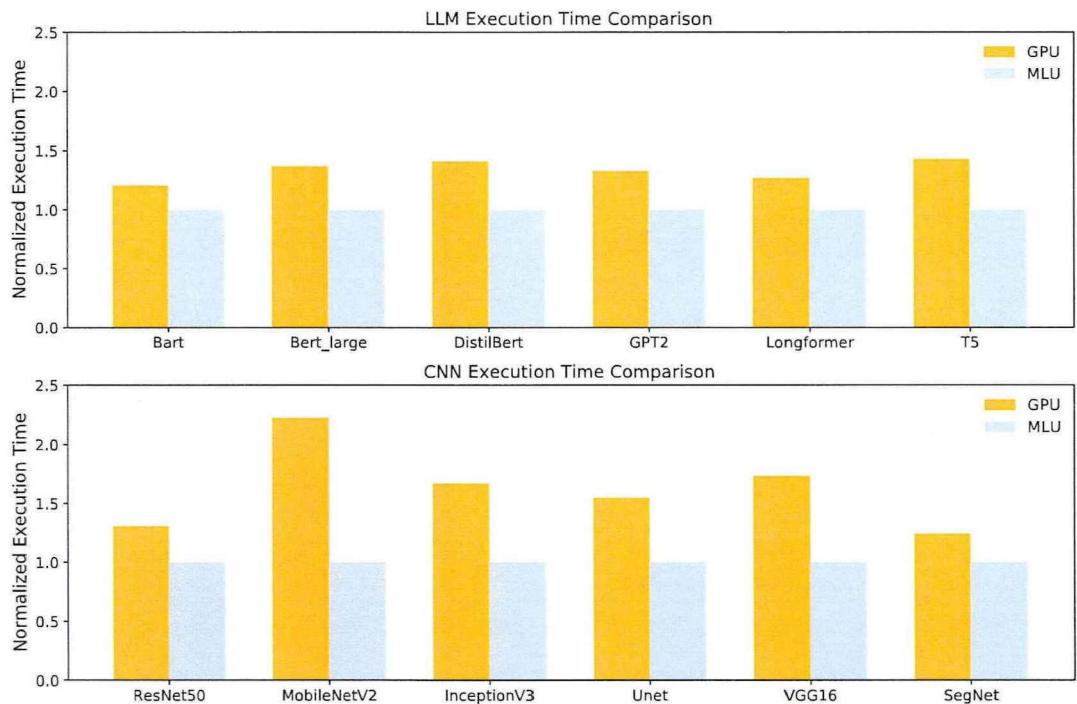


图 5.6 MLU 370 与 GPU T4 的端到端执行对比

优势，性能提升达到 62.18%。这一结果主要归功于本工作提出的指令生成中的内存复用技术和上采样结构的优化，这些优化对于滑窗类算子的场景有显著的性能收益。值得注意的是，大语言模型领域的性能收益相对卷积神经网络领域较低，这主要是由于大语言模型场景下不存在滑窗类型的计算，且没有上采样的网络结构，因此数据复用以及上采样等性能优化手段对于大语言模型类的网络没有性能收益。

5.4 本章小结

本章详细阐述了基于 MLU 平台的深度学习系统级设计与验证工作。通过构建分层架构的寒武纪系统软件，本工作实现了从 AI 框架到硬件指令的高效转换。在算子编译层，基于 Triton 的编译优化框架显著提升了算子执行效率；在图编译层，MagicMind 图编译器通过计算图优化和指令生成技术，实现了系统性能的全面提升。实验结果表明，本工作在大语言模型和传统卷积神经网络场景下均取得了显著的性能提升，其中在大语言模型领域实现了 33.64% 的端到端执行时间缩减，在卷积神经网络领域性能提升达到 62.18%。这些成果验证了本文提出的系统优化方案的有效性，为国产 AI 计算平台的性能优化提供了重要参考。

第6章 总结与展望

6.1 研究总结

随着深度学习技术的快速发展,智能处理器作为支撑人工智能计算的核心硬件平台,其性能优化已成为学术界和工业界共同关注的焦点。人工智能技术的进步主要得益于算法创新、算力提升和数据积累三大要素的协同发展。在算法层面,从卷积神经网络到 Transformer 架构的演进不断突破传统方法的性能瓶颈;在硬件层面,智能处理器通过 SIMD、SIMT 等新型架构显著提升了深度学习模型的训练和推理效率;在数据层面,海量数据的积累为模型训练提供了丰富的素材。然而,要充分发挥这些要素的潜力,实现人工智能技术的广泛应用,仍面临诸多挑战。

其中,智能处理器的编译优化问题尤为突出。在算子编译层面, SIMD 架构的处理器在任务调度和运算指令的操作数粒度上与传统的 SIMT 架构存在显著差异,现有的领域特定语言和编译器在适配 SIMD 架构时存在明显不足。在计算图优化层面,深度神经网络复杂的计算流程和数据依赖关系对硬件加速器提出了严峻挑战,特别是在内存层次结构设计和计算能力优化方面。这些挑战不仅制约了智能处理器的性能发挥,也限制了深度学习技术的广泛应用。

因此,如何实现高效的算子编译和计算图优化,已成为当前面临的两大核心挑战。本研究针对这些问题展开系统性研究,提出了一系列创新性解决方案,旨在突破智能处理器编译优化的技术瓶颈,为深度学习技术的广泛应用提供强有力的编译支持。

6.1.1 算子编译器的研究内容与本文贡献

在算子编译层面, SIMD 架构的智能处理器面临着与 SIMT 架构显著不同的编译优化挑战。具体而言, SIMD 架构的指令粒度更粗,要求编译器能够有效分析和优化数据的连续性,以充分利用其访存优势。针对这一问题,本研究提出了面向 SIMD 架构的 Triton 语言编译优化方案,主要包含以下创新性成果:在任务调度优化方面,提出了一种基于硬件资源感知的任务参数重映射算法。该算法通过将 SIMT 架构的 grid 参数转换为 SIMD 架构的三维任务空间,并引入任务类型标识机制,实现了计算任务与硬件资源的精确匹配。实验结果表明,该优化策略在 MLU 370 平台上实现了显著的性能提升,其中矩阵乘法、标准化、归一化和加法运算分别获得了 3.04 倍、4.22 倍、4.04 倍和 4.65 倍的加速比。在二元算子性能优化方面,提出了数据驻留与流水线并行相结合的优化策略。通过数据驻留技术减少数据加载频次,结合流水线处理技术实现计算与访存的并行化,显著提升

了计算效率。实验结果表明,该优化策略在矩阵乘法、标准化、归一化和加法运算上分别实现了 2.75 倍、1.93 倍、1.71 倍和 1.54 倍的性能提升。在编程效率提升方面,通过对比 Triton 与 BANGC 语言的实现方案,验证了 Triton 语言在简化硬件编程复杂度方面的显著优势。实验结果表明, Triton 实现方案仅需 BANGC 语言 19.8% 的代码量,即可实现 1.4 倍的性能提升,充分体现了 Triton 语言在提升开发效率和执行性能方面的双重优势。在跨平台性能优化方面,通过跨平台性能对比实验,验证了 Triton 语言在 SIMD 架构上的适配完备性。在相同计算负载下, SIMD 架构可实现平均 1.1 倍的性能提升,证明了 Triton 语言在跨平台性能优化方面的显著优势。这些成果为深度学习编译器在 SIMD 架构上的优化提供了理论依据和实践指导,对推动深度学习计算框架的跨平台适配具有重要意义。

6.1.2 计算图编译器的研究内容与本文贡献

在计算图编译层面,本研究提出了完整的优化解决方案,包括算子融合数据流评估框架和融合算子指令生成优化两个核心部分。

1. 融合数据流评估框架的研究内容与本文贡献

本研究针对深度学习计算图的复杂数据流特征,提出了面向 SIMD 架构智能处理器的算子融合数据流评估框架 FusionFrame,通过创新的调度空间表示和分析成本模型,显著提升了 DNN 调度的优化效果。该框架在三个关键维度实现了突破性创新:首先,在算子融合优化方面, FusionFrame 创新性地提出了以存储为中心的融合机制,支持从细粒度到粗粒度的多层次融合策略。该机制通过优化数据局部性,有效减少了片外内存访问,为复杂计算图的优化提供了新的解决方案。其次,在循环分块策略方面, FusionFrame 突破了传统方法的局限性,提出了完整的输出分块方案。这一创新性设计有效解决了长期困扰研究者的权重大小限制问题,显著优化了数据布局,为深度学习计算图的优化提供了新的技术路径。最后,在硬件映射优化方面, FusionFrame 通过硬件感知的融合策略,实现了片上和片外访问的智能平衡。该框架能够根据硬件规格自动确定最优分块大小,避免了传统方法中因固定分块导致的性能损失,为智能处理器的硬件适配提供了新的优化范式。

实验结果表明, FusionFrame 框架在支持融合数据流方面表现出色。在 ResNet-50 工作负载下,相比仅神经元分块的数据流, FusionFrame 的 DRAM 访问延迟平均降低 52.7%,在 TPU-like 架构中总延迟优化达到 63.9%。横向对比不同融合数据流方法, FusionFrame 的平均延迟比仅优化 DRAM 访问的融合数据流提高 45.4%。跨平台的研究结果表明,在 Cambricon-Acc、TPU-like 和 Tesla-NPU-like 三种硬件平台上, FusionFrame 相比现有方法在延迟优化方面平均提升 35.2%,充分验证了其在不同硬件架构下的适用性和优越性。FusionFrame 的创

新性主要体现在三个方面：多维调度空间表示方法、统一的分析成本模型和硬件感知的自动优化机制。这些创新不仅显著提升了 DNN 调度的优化效果，还为未来在多核架构和 Transformer 类模型中的应用奠定了坚实基础，为智能计算系统的进一步发展提供了有力的技术支撑。

2. 融合算子指令生成优化的研究内容与本文贡献

针对深度融合场景下的指令生成难题，本研究提出了系统性的优化方案。首先，针对地址推导难题，提出了基于向量描述的地址推导系统，通过引入新的数据结构和方法论，实现了更精确、更高效的地址推导。其次，针对上采样操作的特殊计算模式，提出了基于池化索引的重排机制和固定偏移的索引计算优化方法，有效解决了上采样操作中的性能瓶颈。此外，针对冗余输入爆炸问题，提出了数据复用策略，通过优化数据搬运和计算模式，显著提升了计算效率。

实验结果表明，所提出的优化方案在多个基准测试集上均取得了显著的性能提升。具体而言，数据复用机制使系统整体执行时间平均减少 39.3%，上采样优化策略使计算时间减少 33.7%。与当前最先进的 Chimera 方案相比，本系统在 DRAM 访存量和执行时间方面分别实现了 23.4% 和 36.6% 的优化。这些优化成果为深度学习模型的部署和加速提供了可靠的技术支撑，同时也为后续研究指明了方向。未来工作将重点关注更复杂的网络架构和更大规模的计算图优化，以进一步提升系统的通用性和优化效果。

6.1.3 系统级应用与评估

在系统级设计与验证方面，本研究应用于寒武纪 MLU 平台的深度学习系统。通过将本工作集成进算子编译优化框架和图编译器，实现了从深度学习算法框架到硬件指令的高效转换。实验结果表明，基于 MLU 370 平台，本研究提出的 Triton+MagicMind 加速方案相比传统手写算子库 CNNL 在深度学习任务中取得了显著的性能提升。在卷积神经网络（CNN）任务中实现了 3.1 倍的加速比，在大语言模型（LLM）任务中实现了 2.43 倍的加速比。与 GPU 相比，该系统在大语言模型和传统卷积神经网络场景下均取得了显著的性能提升，其中在大语言模型领域实现了 33.64% 的端到端执行时间缩减，在卷积神经网络领域性能提升达到 62.18%。

这一性能提升主要源于系统架构的多层次优化：在 CNN 任务中，性能增益主要来自两个关键优化点：首先，通过算子融合技术有效减少了数据访存开销，降低了内存带宽压力；其次，在指令生成阶段对计算量进行了深度优化，提高了计算效率。对于 LLM 任务，性能提升则主要得益于 Triton 算子编译器的优化效果，其通过高效的中间表示转换和硬件指令生成显著提升了单算子性能；同时，算子融合技术有效减少了算子间的调度开销，进一步提升了整体执行效率。

这些研究成果不仅丰富了深度学习编译优化的理论体系，也为智能处理器的实际应用提供了重要的技术支撑。未来研究可进一步探索在更广泛硬件平台上的适应性优化，以及面向新兴 AI 模型的编译优化技术，持续推动深度学习计算框架的跨平台适配和性能优化。

6.2 研究展望

本文围绕智能计算系统的跨平台编译优化这一核心问题展开研究，针对当前深度学习编译器在 SIMD 架构适配、计算图优化和指令生成等方面存在的挑战，系统性地将问题拆解为三个相互关联又相对独立的子问题：面向 SIMD 架构的 Triton 语言编译优化、面向 SIMD 架构智能处理器的算子融合数据流评估框架、以及面向 SIMD 架构智能处理器的图融合指令生成系统。通过理论创新与实践验证相结合的研究方法，本文在多个维度取得了突破性进展，为智能处理器的编译优化提供了新的技术路径。然而，随着深度学习技术的快速演进和硬件架构的持续创新，本研究仍存在诸多值得深入探索的方向。基于当前研究成果和领域发展趋势，本工作从技术深度、应用广度和系统完备性三个维度进行了深入讨论：

面向 SIMD 架构智能处理器的 Triton 语言编译性能优化：未来研究将围绕性能优化与跨平台适配两个维度展开深入探索。在性能优化方面，重点突破冗余数据拷贝消除技术和数据依赖分析优化技术，通过降低内存带宽压力提升大规模张量运算效率；同时，将研究范围扩展至多核架构，探索基于数据局部性的并行计算优化方案，实现计算资源的充分利用。在跨平台适配方面，着力构建统一的硬件抽象层，通过模块化架构设计实现硬件相关与无关部分的解耦，为不同国产处理器提供快速移植方案，推动国产处理器生态建设。

面向 SIMD 架构智能处理器的算子融合数据流评估框架：未来研究方向将聚焦于多核架构精确建模与框架应用范围扩展。在多核架构建模方面，重点研究任务拆分与核间映射优化策略，建立多机多卡场景下的通信开销模型，提升系统可扩展性；针对 Transformer 类模型，深入研究矩阵乘法和层归一化等算子的融合优化策略，开发面向大语言模型的专用评估模块。此外，将致力于构建统一的中间表示，实现与主流机器学习框架的生态兼容，探索调度结果的统一表示方法，促进不同调度框架的协同优化，提升框架的通用性和实用性。

面向 SIMD 架构智能处理器的图融合指令生成系统：未来研究将重点突破硬件适配与应用场景扩展两个关键方向。在硬件适配方面，构建通用的硬件特征描述框架，设计跨平台的中间表示，实现对华为 Ascend NPU、google TPU 等异构计算平台的统一支持，提升系统的硬件兼容性。针对大语言模型的计算特征，

开发专门的指令生成策略，包括基于稀疏计算的 **attention** 机制优化和面向长序列处理的分块计算技术，提升系统在复杂场景下的性能表现。同时，探索动态指令生成技术，实现基于运行时信息的自适应优化，增强系统的智能化水平。

参考文献

- [1] LECUN Y, BOTTOU L, BENGIO Y, et al. Gradient-based learning applied to document recognition[J]. Proceedings of the IEEE, 1998, 86(11): 2278-2324.
- [2] KRIZHEVSKY A, SUTSKEVER I, HINTON G E. Imagenet classification with deep convolutional neural networks[J]. Advances in Neural Information Processing Systems, 2012, 25.
- [3] SIMONYAN K, ZISSERMAN A. Very deep convolutional networks for large-scale image recognition[J]. arXiv preprint arXiv:1409.1556, 2014.
- [4] HE K, ZHANG X, REN S, et al. Deep residual learning for image recognition[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2016.
- [5] GIRSHICK R, DONAHUE J, DARRELL T, et al. Rich feature hierarchies for accurate object detection and semantic segmentation[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2014.
- [6] LONG J, SELHAMER E, DARRELL T. Fully convolutional networks for semantic segmentation[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2015.
- [7] LITJENS G, KOOI T, BEJNORDI B E, et al. A survey on deep learning in medical image analysis[J]. Medical image analysis, 2017, 42: 60-88.
- [8] BOJARSKI M. End to end learning for self-driving cars[J]. arXiv preprint arXiv:1604.07316, 2016.
- [9] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need[J]. Advances in Neural Information Processing Systems, 2017, 30.
- [10] DEVLIN J. Bert: Pre-training of deep bidirectional transformers for language understanding [J]. arXiv preprint arXiv:1810.04805, 2018.
- [11] DOSOVITSKIY A. An image is worth 16x16 words: Transformers for image recognition at scale[J]. arXiv preprint arXiv:2010.11929, 2020.
- [12] LIU Z, LIN Y, CAO Y, et al. Swin transformer: Hierarchical vision transformer using shifted windows[C]//Proceedings of IEEE/CVF International Conference on Computer Vision (ICCV). 2021.
- [13] BROWN T, MANN B, RYDER N, et al. Language models are few-shot learners[J]. Advances in Neural Information Processing Systems, 2020, 33: 1877-1901.
- [14] RADFORD A, KIM J W, HALLACY C, et al. Learning transferable visual models from natural language supervision[C]//International conference on machine learning. 2021.

- [15] NARAYANAN D, SHOEYBI M, CASPER J, et al. Efficient large-scale language model training on gpu clusters using megatron-lm[C]//Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. 2021.
- [16] CHILD R, GRAY S, RADFORD A, et al. Generating long sequences with sparse transformers [J]. arXiv preprint arXiv:1904.10509, 2019.
- [17] DU Y, LIU Z, LI J, et al. A survey of vision-language pre-trained models[J]. arXiv preprint arXiv:2202.10936, 2022.
- [18] HENNESSY J L, PATTERSON D A. Computer architecture: a quantitative approach[M]. Elsevier, 2011.
- [19] JOUPPI N P, YOUNG C, PATIL N, et al. In-datacenter performance analysis of a tensor processing unit[C]//Proceedings of the International Symposium on Computer Architecture (ISCA). 2017.
- [20] LINDHOLM E, NICKOLLS J, OBERMAN S, et al. NVIDIA Tesla: A unified graphics and computing architecture[J]. IEEE Micro, 2008, 28(2): 39-55.
- [21] CHEN Y, CHEN T, XU Z, et al. DianNao family: energy-efficient hardware accelerators for machine learning[J]. Communications of the ACM, 2016.
- [22] CHEN T, MOREAU T, JIANG Z, et al. TVM: An automated End-to-End optimizing compiler for deep learning[C]//USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2018.
- [23] ABADI M, BARHAM P, CHEN J, et al. TensorFlow: A system for Large-Scale machine learning[C]//USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2016: 265-283.
- [24] PASZKE A, GROSS S, MASSA F, et al. Pytorch: An imperative style, high-performance deep learning library[J]. Advances in Neural Information Processing Systems, 2019, 32.
- [25] VASILACHE N, ZINENKO O, THEODORIDIS T, et al. Tensor comprehensions: Framework-agnostic high-performance machine learning abstractions[J]. arXiv preprint arXiv:1802.04730, 2018.
- [26] VERDOOLAEGE S, GROSSER T. Polyhedral extraction tool[C]//Second International Workshop on Polyhedral Compilation Techniques (IMPACT), Paris, France: volume 141. 2012.
- [27] ADAMS A, MA K, ANDERSON L, et al. Learning to optimize halide with tree search and random programs[J]. ACM Transactions on Graphics (TOG), 2019, 38(4): 1-12.
- [28] RAGAN-KELLEY J, BARNES C, ADAMS A, et al. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines[J]. Acm Sigplan Notices, 2013, 48(6): 519-530.

- [29] FUNG W W, SHAM I, YUAN G, et al. Dynamic warp formation and scheduling for efficient GPU control flow[C]//Proceedings of the IEEE/ACM International Symposium on Microarchitecture (MICRO). IEEE, 2007: 407-420.
- [30] JIA Z, MAGGIONI M, STAIGER B, et al. Dissecting the NVIDIA volta GPU architecture via microbenchmarking[J]. arXiv preprint arXiv:1804.06826, 2018.
- [31] ZHANG Y, OWENS J D. A quantitative performance analysis model for GPU architectures [C]//2011 IEEE 17th international symposium on high performance computer architecture. IEEE, 2011: 382-393.
- [32] KAYIRAN O, JOG A, KANDEMIR M T, et al. Neither more nor less: Optimizing thread-level parallelism for gpgpus[C]//Proceedings of the 22nd international conference on Parallel architectures and compilation techniques. IEEE, 2013: 157-166.
- [33] ROGERS T G, O'CONNOR M, AAMODT T M. Cache-conscious wavefront scheduling [C]//Proceedings of the IEEE/ACM International Symposium on Microarchitecture (MICRO). IEEE, 2012: 72-83.
- [34] CHEN T, ZHENG L, YAN E, et al. Learning to optimize tensor programs[J]. Advances in Neural Information Processing Systems, 2018, 31.
- [35] ZHENG L, JIA C, SUN M, et al. Ansor: Generating {High-Performance} tensor programs for deep learning[C]//USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2020.
- [36] TILLET P, KUNG H T, COX D. Triton: an intermediate language and compiler for tiled neural network computations[C]//Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages. 2019.
- [37] SABNE A. Xla: Compiling machine learning for peak performance[J]. Google Res, 2020.
- [38] ZHAO J, GAO X, XIA R, et al. Apollo: Automatic partition-based operator fusion through layer by layer optimization[J]. Proceedings of Machine Learning and Systems, 2022, 4: 1-19.
- [39] SYMONS A, MEI L, COLLEMAN S, et al. Stream: Design space exploration of layer-fused dnns on heterogeneous dataflow accelerators[J]. IEEE Transactions on Computers, 2024, 74 (1): 237-249.
- [40] ZHENG S, CHEN S, GAO S, et al. Tileflow: A framework for modeling fusion dataflow via tree-based analysis[C]//Proceedings of the IEEE/ACM International Symposium on Microarchitecture (MICRO). 2023.
- [41] WOLFE M. More iteration space tiling[C]//Proceedings of the ACM/IEEE Conference on Supercomputing. 1989.
- [42] DARTE A. On the complexity of loop fusion[C]//International Conference on Parallel Architectures and Compilation Techniques. 1999.

- [43] ALLEN J R, KENNEDY K. Automatic loop interchange[C]//Proceedings of the SIGPLAN Symposium on Compiler Construction. 1984.
- [44] GIRBAL S, VASILACHE N, BASTOUL C, et al. Semi-automatic composition of loop transformations for deep parallelism and memory hierarchies[J]. International Journal of Parallel Programming, 2006, 34: 261-317.
- [45] SATO Y, YUKI T, ENDO T. An autotuning framework for scalable execution of tiled code via iterative polyhedral compilation[J]. ACM Transactions on Architecture and Code Optimization (TACO), 2019, 15(4): 1-23.
- [46] MULLAPUDI R T, ADAMS A, SHARLET D, et al. Automatically scheduling halide image processing pipelines[J]. ACM Transactions on Graphics (TOG), 2016, 35(4): 1-11.
- [47] GROSSER T, GROESSLINGER A, LENGAUER C. Polly—performing polyhedral optimizations on a low-level intermediate representation[J]. Parallel Processing Letters, 2012, 22(04): 1250010.
- [48] LIU H I C, BREHLER M, RAVISHANKAR M, et al. Tinyiree: An ml execution environment for embedded systems from compilation to deployment[J]. IEEE Micro, 2022, 42(5): 9-16.
- [49] ANSEL J, YANG E, HE H, et al. Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation[C]//Proceedings of the ACM International Conference on Architectural Support for Programming Languages and Operating Systems. 2024.
- [50] Triton language programming guide[EB/OL]. 2020[2020-06-25]. <https://triton-lang.org/main/programming-guide/chapter-1/introduction.html>.
- [51] BANG C Language Developer Guide[EB/OL]. 2008[2020-02-04]. <https://forum.cambricon.com/uploadfile/user/file/20201125/1606289569710855.pdf>.
- [52] NVIDIA CUDA Programming Guide[EB/OL]. 2008[2008-01-25]. https://www.nvidia.cn/docs/IO/51635/NVIDIA_CUDA_Programming_Guide_1.1_chs.pdf.
- [53] FlagPerf - A Performance Benchmark Suite for Flag Open Source Projects[EB/OL]. 2023[2023-01-25]. <https://github.com/FlagOpen/FlagPerf>.
- [54] Triton-lang v3.0.0[EB/OL]. 2023[2024-01-25]. <https://github.com/triton-lang/triton>.
- [55] SZEGEDY C, LIU W, JIA Y, et al. Going deeper with convolutions[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2015.
- [56] HOWARD A G, ZHU M, CHEN B, et al. Mobilenets: Efficient convolutional neural networks for mobile vision applications[J]. arXiv preprint arXiv:1704.04861, 2017.
- [57] RADFORD A, KIM J W, XU T, et al. Robust speech recognition via large-scale weak supervision[C]//International Conference on Machine Learning. 2023.
- [58] LIANG P, TANG Y, ZHANG X, et al. A survey on auto-parallelism of large-scale deep learning

- training[J]. IEEE Transactions on Parallel and Distributed Systems, 2023, 34(8): 2377-2390.
- [59] CHEN Y, LAN H, DU Z, et al. An instruction set architecture for machine learning[J]. ACM Transactions on Computer Systems (TOCS), 2019, 36(3): 1-35.
- [60] PARASHAR A, RAINA P, SHAO Y S, et al. Timeloop: A systematic approach to dnn accelerator evaluation[C]//IEEE international symposium on performance analysis of systems and software (ISPASS). 2019.
- [61] KWON H, CHATARASI P, SARKAR V, et al. Maestro: A data-centric approach to understand reuse, performance, and hardware cost of dnn mappings[J]. IEEE micro, 2020, 40(3): 20-29.
- [62] MEI L, HOUSHMAND P, JAIN V, et al. Zigzag: Enlarging joint architecture-mapping design space exploration for dnn accelerators[J]. IEEE Transactions on Computers, 2021, 70(8): 1160-1174.
- [63] YEUNG G, BOROWIEC D, YANG R, et al. Horus: Interference-aware and prediction-based scheduling in deep learning systems[J]. IEEE Transactions on Parallel and Distributed Systems, 2021, 33(1): 88-100.
- [64] WAEIJEN L, SIOUTAS S, PEEMEN M, et al. Convfusion: A model for layer fusion in convolutional neural networks[J]. IEEE Access, 2021, 9: 168245-168267.
- [65] CAI X, WANG Y, ZHANG L. Optimus: An operator fusion framework for deep neural networks[J]. ACM Transactions on Embedded Computing Systems, 2022, 22(1): 1-26.
- [66] XING Y, LIANG S, SUI L, et al. DNNVM: End-to-end compiler leveraging operation fusion on FPGA-based CNN accelerators[C]//Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. 2019.
- [67] MEI L, GOETSCHALCKX K, SYMONS A, et al. DeFiNES: Enabling Fast Exploration of the Depth-first Scheduling Space for DNN Accelerators through Analytical Modeling[C]//IEEE International Symposium on High-Performance Computer Architecture (HPCA). 2023.
- [68] MOREIRA O, POPP M, SCHULZ C. Graph partitioning with acyclicity constraints[J]. arXiv preprint arXiv:1704.00705, 2017.
- [69] KARL S, SYMONS A, FASFOUS N, et al. Genetic Algorithm-based Framework for Layer-Fused Scheduling of Multiple DNNs on Multi-core Systems[C]//Design, Automation & Test in Europe Conference & Exhibition (DATE). 2023.
- [70] YIN J, MEI L, GUNTORO A, et al. Acco: Automated causal cnn scheduling optimizer for real-time edge accelerators[C]//IEEE International Conference on Computer Design (ICCD). 2023.
- [71] YANG X, GAO M, LIU Q, et al. Interstellar: Using halide's scheduling language to analyze dnn accelerators[C]//Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems. 2020.

- [72] MEI L, LIU H, WU T, et al. A uniform latency model for dnn accelerators with diverse architectures and dataflows[C]//Design, Automation & Test in Europe Conference & Exhibition (DATE). 2022.
- [73] LU L, GUAN N, WANG Y, et al. Tenet: A framework for modeling tensor dataflow based on relation-centric notation[C]//Proceedings of the International Symposium on Computer Architecture (ISCA). IEEE, 2021: 720-733.
- [74] HEGDE K, TSAI P A, HUANG S, et al. Mind mappings: enabling efficient algorithm-accelerator mapping space search[C]//Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems. 2021.
- [75] KAO S C, KRISHNA T. Magma: An optimization framework for mapping multiple dnns on multiple accelerator cores[C]//IEEE International Symposium on High-Performance Computer Architecture (HPCA). 2022.
- [76] KAO S C, JEONG G, KRISHNA T. Confucius: Autonomous hardware resource assignment for dnn accelerators using reinforcement learning[C]//Proceedings of the IEEE/ACM International Symposium on Microarchitecture (MICRO). 2020.
- [77] SZEGEDY C, VANHOUCKE V, IOFFE S, et al. Rethinking the inception architecture for computer vision[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2016.
- [78] TALPES E, SARMA D D, VENKATARAMANAN G, et al. Compute solution for tesla's full self-driving computer[J]. IEEE Micro, 2020, 40(2): 25-35.
- [79] KAO S C, HUANG X, KRISHNA T. DNNFuser: Generative pre-trained transformer as a generalized mapper for layer fusion in dnn accelerators[J]. arXiv preprint arXiv:2201.11218, 2022.
- [80] ZHENG S, CHEN R, JIN Y, et al. Neoflow: A flexible framework for enabling efficient compilation for high performance dnn training[J]. IEEE Transactions on Parallel and Distributed Systems, 2022, 33(11): 3220-3232.
- [81] ZHAO J, LI B, NIE W, et al. AKG: Automatic kernel generation for neural processing units using polyhedral transformations[C]//Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation. 2021.
- [82] ZHU H, WU R, DIAO Y, et al. ROLLER: Fast and efficient tensor compilation for deep learning[C]//USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2022.
- [83] ZHENG S, CHEN S, SONG P, et al. Chimera: An analytical optimizing framework for effective compute-intensive operators fusion[C]//IEEE International Symposium on High-Performance Computer Architecture (HPCA). 2023.

- [84] BAHETI B, INNANI S, GAJRE S, et al. Eff-unet: A novel architecture for semantic segmentation in unstructured environment[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2020: 358-359.
- [85] DUMOULIN V, VISIN F. A guide to convolution arithmetic for deep learning[J]. arXiv preprint arXiv:1603.07285, 2016.
- [86] RONNEBERGER O, FISCHER P, BROX T. U-net: Convolutional networks for biomedical image segmentation[C]//Medical image computing and computer-assisted intervention. Springer, 2015: 234-241.
- [87] BADRINARAYANAN V, KENDALL A, CIPOLLA R. Segnet: A deep convolutional encoder-decoder architecture for image segmentation[J]. IEEE transactions on pattern analysis and machine intelligence, 2017, 39(12): 2481-2495.
- [88] JING J, WANG Z, RÄTSCHE M, et al. Mobile-Unet: An efficient convolutional neural network for fabric defect detection[J]. Textile Research Journal, 2022, 92(1-2): 30-42.
- [89] Hugging Face Models[EB/OL]. 2024[2024-06-25]. <https://huggingface.co/models>.